# NUMERICAL SOLUTIONS TO PARTIAL DIFFERENTIAL EQUATIONS

**Acknowledgement of Funding**
Development of this textbook was supported by grants from the U.S. National Science Foundation (grant numbers 1752531 and 2306329) and a Zero Textbook Cost (ZTC) initiative funded by the Colorado Department of Higher Education Open Educational Resources Grant Program, which supports the creation of openly accessible learning materials to reduce or eliminate textbook costs for students.

**Use of AI-assisted editing tools**
The author acknowledges the use of *Claude* (Anthropic) and *ChatGPT* (OpenAI) as editing tools in the preparation of this textbook. Both AI tools were used *extensively* to identify grammatical issues, brainstorm clearer phrasing, verify consistent notation, resolve formatting issues in LaTeX, and compress image files. Any sample computer codes in this text were likely passed through an AI tool to check against best coding practices. To the author's best knowledge, the use of AI tools did not extend to original content generation. Furthermore, all figures and images in this text were either created by the author (using primarily Inkscape and Matlab) or third-party sources who are credited under Creative Commons licenses.

**Creative Commons Licensing**



This work is licensed under Creative Commons Attribution-NonCommercial 4.0 International. To view a copy of this license, visit
https://creativecommons.org/licenses/by-nc/4.0/

# Contents




1 Introductions and Basics . . . . . 6

1.1 Partial Differential Equations Overview 6

1.2 How to Visually Communicate 7

2 Finite Difference Methods . . . . . 23

2.1 The Basics 23

2.2 First Derivative Approximations 24

2.3 Second Derivative Approximations 26

3 One-Dimensional FDMs in Practice . . . . . 27

3.1 Type I Grids 28

3.2 Type II Grids 29

3.3 Boundary Conditions 29

3.4 Basic Numerical Implementation 33

4 Two-Dimensional FDMs in Practice . . . . . 37

4.1 The Five-Point Laplacian 38

4.2 Ordering the Unknowns 39

4.3 Assembling Operators with Kronecker Products 40

4.4 Type I and Type II Grids in Two Dimensions 41

4.5 Boundary Conditions in Two Dimensions 42
4.6 Assembling the Discrete System 45
4.7 Basic Numerical Implementation 45

5 Staggered Grids . . . . . 51
5.1 Coupled Unknowns on a Single Grid 51
5.2 Staggering in One Dimension 52
5.3 The Marker-and-Cell Grid 53
5.4 The Stokes Equations 56

6 Krylov Methods and GMRES . . . . . 60
6.1 A Model Problem: Poisson's Equation in Three Dimensions 60
6.2 Krylov Subspaces 62
6.3 The GMRES Algorithm 63
6.4 Initial Guesses 64
6.5 Choosing a Tolerance 65
6.6 Looking Ahead: Preconditioning 65
6.7 Numerical Implementation 66

7 Immersed Boundary Methods . . . . . 70
7.1 A Model Problem 70
7.2 Regularized Delta Functions 73
7.3 Spreading and Interpolation 74
7.4 The Saddle Point System 74
7.5 Solving Without a Matrix 75
7.6 Numerical Implementation 76

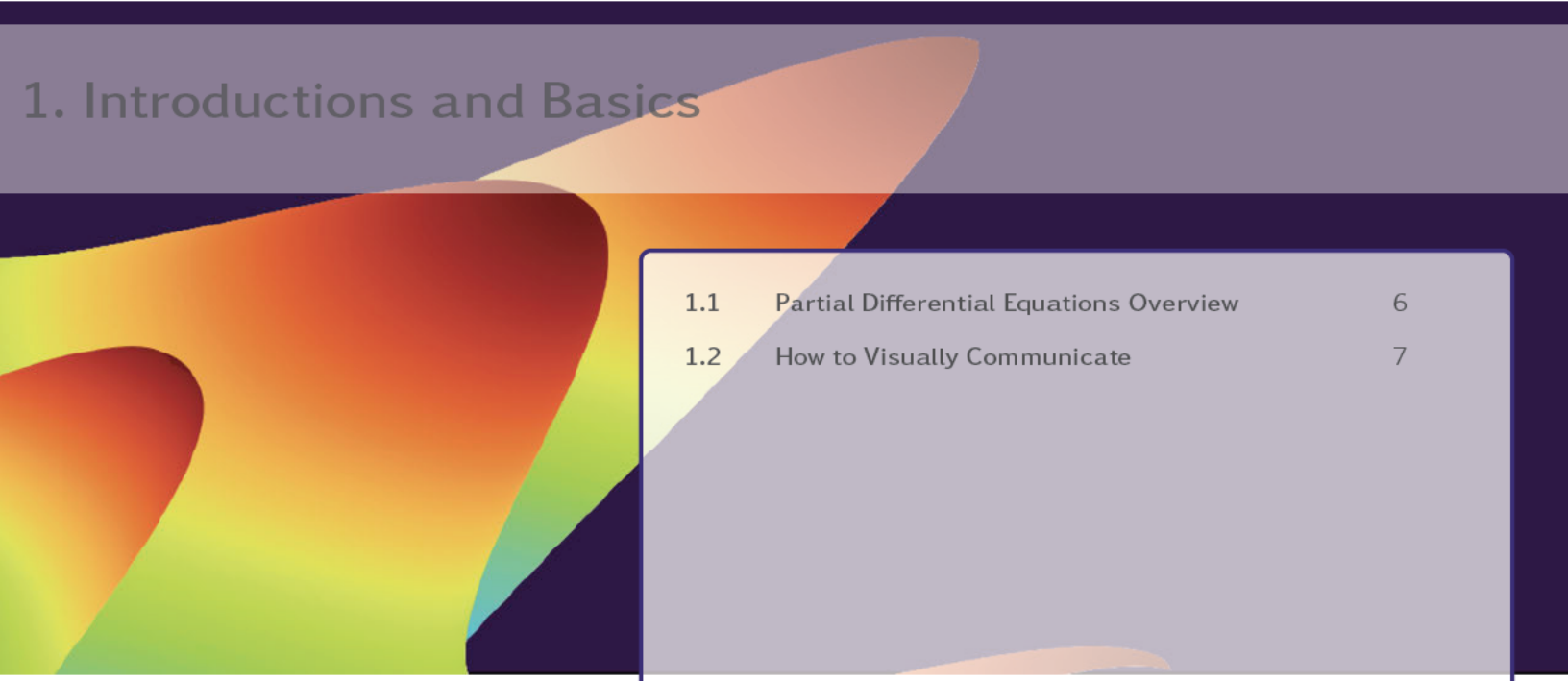
8 Time Stepping: Accuracy and Stability . . . . . 81
8.1 The Heat Equation 81
8.2 Exact Solution 84
8.3 Forward and Backward Euler 85
8.4 Accuracy 86
8.5 Stability 87
8.6 Higher-Order Time Steppers 89
8.7 Solving the Implicit Systems Efficiently 90

8.8 Numerical Implementation 91

Bibliography . . . . . 95

Index . . . . . 96

# 1. Introductions and Basics




1.1 Partial Differential Equations Overview 6

1.2 How to Visually Communicate 7


## 1.1 Partial Differential Equations Overview

There are three classic partial differential equations (PDEs):

**I.** Wave equation: $\nabla^2\phi = \frac{1}{c^2}\frac{\partial^2\phi}{\partial t^2}$ *(hyperbolic)*

**II.** Heat/diffusion equation: $\nabla^2\phi = \frac{1}{k}\frac{\partial\phi}{\partial t}$ *(parabolic)*

**III.** Laplace's equation: $\nabla^2\phi = 0$ *(elliptic)*

Here $\phi$ denotes some property (i.e., velocity, pressure), $t$ time, and $c, k \in \mathbb{R}$ are constants. $\nabla$ is the nabla operator and its definition depends on the context. Generally, $\nabla$ is an $n$-dimensional vector operator whose components contain a partial derivative operation with respect to the dimension of their index. For example,

$$\begin{aligned} \text{n} = 1: \quad & \nabla = \left[\frac{\partial}{\partial x}\right] \\ \text{n} = 2: \quad & \nabla = \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}\right] \\ \text{n} = 3: \quad & \nabla = \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z}\right] \end{aligned}$$

Let's consider a two-dimensional ($n = 2$) domain $\Omega$ and the property velocity $\mathbf{u} = [u(x,y), v(x,y)]$, where $(x,y) \in \Omega$. Then,

$$\nabla \cdot \mathbf{u} = \left[\frac{\partial}{\partial x}, \frac{\partial}{\partial y}\right] \cdot [u(x,y), v(x,y)] = \frac{\partial u(x,y)}{\partial x} + \frac{\partial v(x,y)}{\partial y}.$$

This is simply the divergence of $\Omega$. The $\nabla$ operator also appears for calculations of gradients ($\nabla\mathbf{u}$) and curl ($\nabla \times \mathbf{u}$).

Each of the three classic PDEs have their own set of applications. The wave equation **I** can help model the motion of specific waves, such as sound and electromagnetic, and how they interact with solids. Heat transfer and basic diffusion of chemicals, such as air pollutants, can be modeled with the diffusion equation **II**. Laplace's equation **III** can be applied to gravitational attraction and electrostatics. All of these equations have many more applications than those listed here, but these give applications provide intuition on how they can be used to better understand dynamical systems.

Note that all three of these equations are linear, second-order, homogeneous PDEs. Recall that a PDE is linear if no unknowns are multiplied together, second-order because the highest derivative of $\mathbf{u}$ is two, and homogeneous because it is only an equation of unknowns. In all three, $\mathbf{u} = \mathbf{0}$ is always a solution and the principle of superposition can be applied. These properties allow us to construct many solutions of the PDE by separation of variables. All three equations also contain $\nabla^2$, which follows from physical assumptions. Many modeled phenomena are independent of coordinate system, meaning that the choice or origin and orientation of axes does not change the outcome. $\nabla^2$ is also independent of the coordinate system, making it a standard choice when modeling these phenomena.

Inhomogeneous versions of classic PDEs are of interest as they allow us to understand how a system behaves when forced. One notable example is Poisson's equation

$$\nabla^2 \phi = \mathbf{f},$$

where $f$ is a given function and $\phi$ is unknown. Often we cannot solve these by hand, leading to the development of numerical methods. Numerical methods fall into two main categories: colocation and Galerkin. Colocation methods solve for $\phi$ at discrete points across the domain. Galerkin methods rely on known basis functions and solve for $\phi$ in terms of these bases. In this class, we will consider

1. Finite difference methods (Colocation)
2. Finite element methods (Galerkin)
3. pseudo-spectral methods (either)
4. boundary integral methods (either)

## 1.2 How to Visually Communicate

Pictures, figures, images, plots, etc. are all forms of visual communication. Any visual communication should be made in the most *effective* and *accessible* way possible. This chapter will discuss what it means to visually communicate effectively and ensure accessibility when creating line and field plots from data. General implementation of these practices in python and MATLAB is provided in Section 1.2.2.

### 1.2.1 Effective and Accessible Communication

Visual communication is effective when the reader immediately understands what is being presented. Labels, such as those in legends and on axes, informs the reader without the need for questions, assumptions or guessing. Units for the axes set the scale of the problem,

which allows the reader to understand physical or numerical implications without explanation. Captions describe the visualization in words and include what the reader needs to understand the plot's labels, such as variable definitions. Note that conclusions or analysis *should not* be discussed in captions. Plots can imply if the data are discrete or continuous through style; data are generally discrete and should be plotted using symbols, while theory/fits should be plotted as continuous curves. All of these choices discussed make visual communication more effective.

For visual communication to be completely accessible, no reader should be at a disadvantage due to physical abilities or technological access. Colorblindness can make data difficult to discern if the colors do not contrast, especially when considering those who are completely colorblind or when work is printed in black and white. Efforts to use more than color, such as markers, and to select contrasting colors ensure general readability no matter the case. Assistive technology is capable of converting visual communication to non-visual communication by reading alternative text of images. Although out of the scope of this class, it is important to include and use descriptive alternative text so that any visual communication can be understood without seeing the image or figure.

The next two subsections will discuss class standards on visual communication for line and field plots. Code used to produce Figures 1.1 and 1.3 is provided in Section 1.2.2.

### Line Plots

Consider the presentation of data in Figure 1.1. The figure is arranged in two columns: column (1), on the left, shows a poorly made plot, while column (2), on the right, shows a well-constructed plot. The three rows *(a)*, *(b)*, and *(c)* correspond to three different color spectrums: full RGB, that related to protanopia (red-green colorblind), and grayscale. Table 1.1 compares each flaw of plot (1) with its correction in column (2), with a description of how the correction increases effectiveness in column (3).

| *Bad plot traits (seen in 1)* | *Good plot traits (seen in 2)* | *Impact to reader* |
|---|---|---|
| No data symbols | Large, differentiable data symbols | Shows where the data actually is. **Symbols represent data; curves represent theory/fits.** Symbols also keep data distinguishable in the legend and plot when color fails (e.g., poor projector or paper printout). |
| Small tick labels (numbers on axes) | Large labels everywhere | **Make everything larger than you think!** Easier for the viewer to read and interpret the data. |
| No labels for data, plot, or axes | Fully labeled plot **with units** | Tells the reader what is being presented and the scale of the problem. |
| Similar colors for different data | Unique, contrasting colors for each dataset | Keeps data distinguishable for colorblind readers and in grayscale, shown in plots (b) and (c). Similar RGB colors can become identical in both cases. |
| Linearly scaled axes | Logarithmically scaled axes | Reveals power-law behavior. |
| No reference lines | Reference lines for $n$-order convergence | Confirms to the reader that the claimed order of convergence is actually achieved. |
| Programs default font (mismatched with document) | Font matching the document body (e.g., Latin Modern) | Makes plot text consistent with the surrounding LaTeX type, so figures don't look pasted in. **Computer/Latin Modern or Helvetica/Free Sans are strongly recommended.** |

*Table 1.1: Description of differences in plots shown in Figure 1.1 and their impact on readability.*

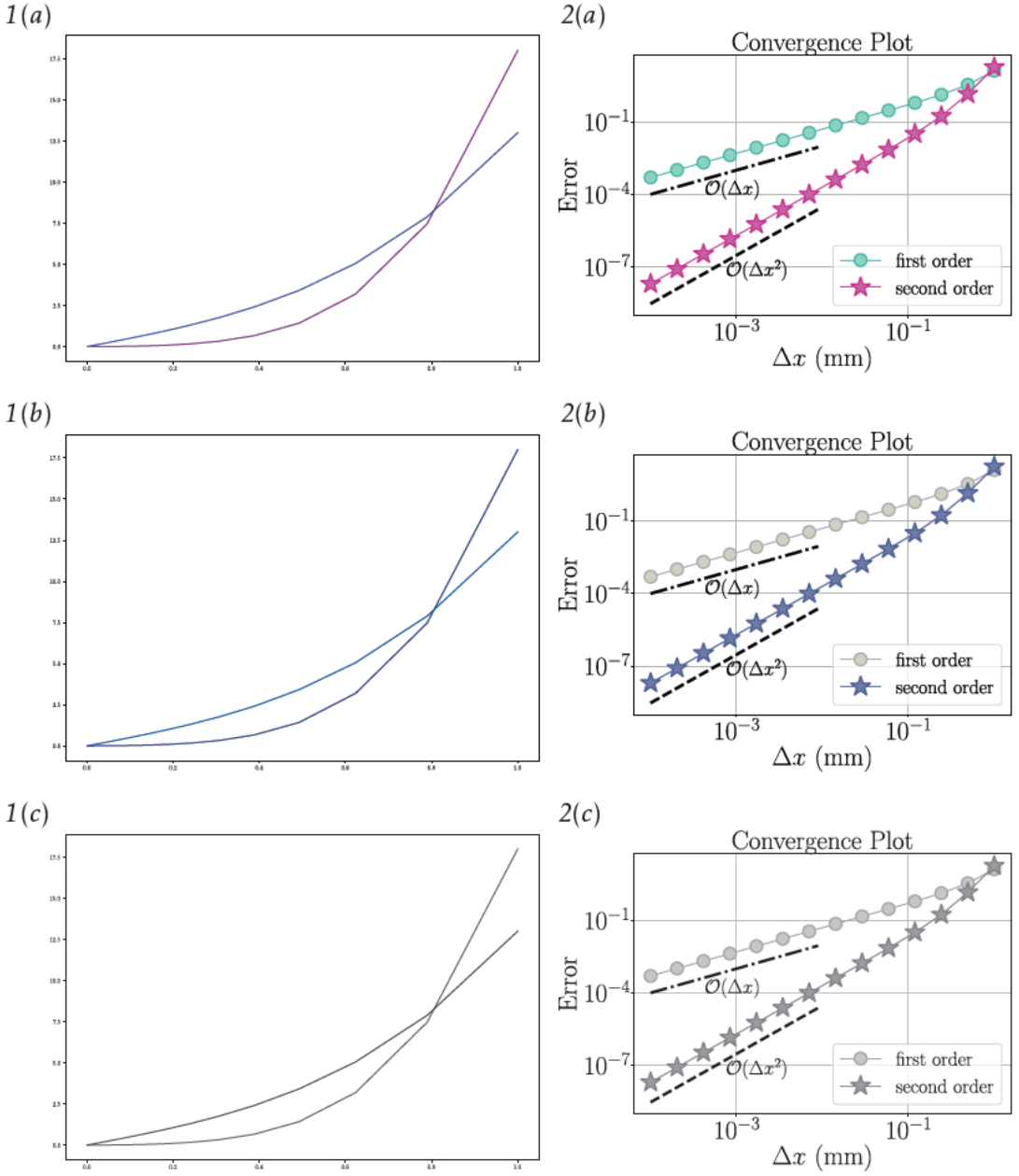



*Figure 1.1: Column 1 and Column 2 represent line plots of the same dataset with different plotting styles. The three rows* (a), (b), *and* (c) *correspond to full RGB color, color as experienced with protanopia (red–green colorblindness), and grayscale, respectively.*

### Field Plots

Plotting fields is an effective way to understand what a system is doing at an instance of time. Efforts to make fields more effective and accessible are focused on ensuring the labels are clear and a good colormap is selected.



*Figure 1.2: A sequential colormap (blues) on top and a diverging colormap (curl) on bottom. Images from Sipkens and Pasquier [1].*

A colormap is a gradient of color, as seen in Figure 1.2. Colormaps map numerical values of a field to colors along a gradient, which allows for spatial visualization of variations of the field. Different styles of colormap suit different applications; here we consider *sequential* and *divergent* maps.

Sequential colormaps change monotonically from one end to the other, like the *blues* map at the top of Figure 1.2, which runs from dark blue to bright white. Sequential maps are effective for fields where growth or subtle changes in magnitude across the field are important to discern.

Divergent colormaps change monotonically from each end toward the middle, where two dark colors meet at a bright center, as in the *curl* [2] map shown at the bottom of Figure 1.2. Divergent maps are effective for highlighting variation from means in fields that diverge or cycle around a value (i.e., sinusoidal).

Consider the two-dimensional fields shown in Figure 1.3. The field in column 1 is considered sequential, as the data grows monotonically to peaks at different points in the field. The field is column two contains discontinuous oscillations about $f(x,y) = 0$. A divergent color map clearly shows where these oscillations are and the general pattern they have across the field, while highlighting the mean. This pattern is much more apparent in the two-dimensional (2D) plot (2b), whereas the three-dimensional (3D) plot (2b) does not accurately reflect the field. In this case, the field at the center is hidden by the large magnitude oscillations along the $x$ axis. In 1(b), it is not clear what is happening behind the peaks of the field. These examples highlight the importance of angle at which the data is plot, as the surface can hid information when certain angles are chosen. Generally, it is best to plot in 2D to ensure a full view of the data field, as seen in the (a) row of Figure 1.3.

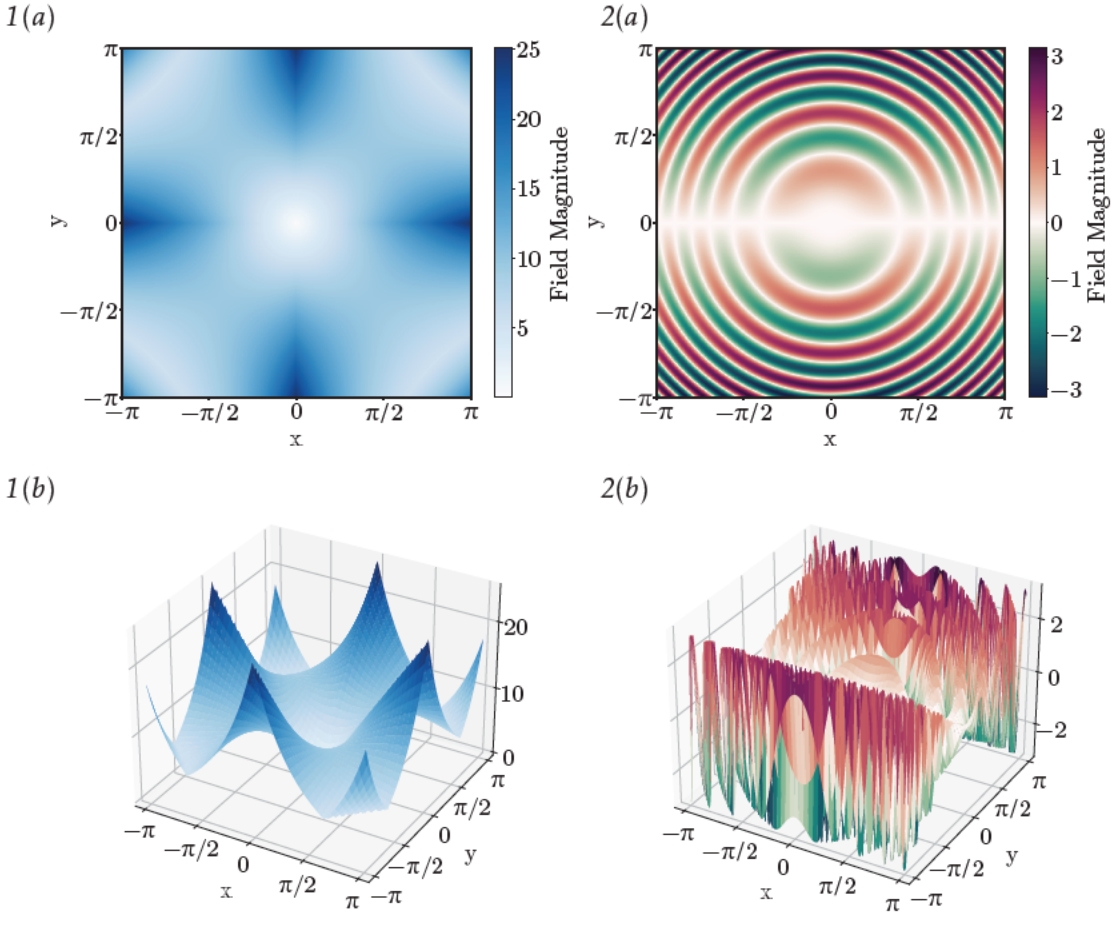



*Figure 1.3: Column 1 and Column 2 represent field plots of the same dataset with different plotting styles. The two rows* (a) *and* (b) *correspond to flat pseudocolor plots and 3D plots, respectively.*

What makes a colormap good? Color can mean more than just showcasing subtle changes or disparities around a mean. It can help showcase landscapes, simulations of ocean surfaces, statistical disparities, and much more. Many colormaps are born from a need to have both effective and accessible visual communication in situations that call for certain color schemes or styles. Open-source colormap repositories include:

- cmocean [2, 3]: for presenting oceanography data,
- *CMasher* [4]: aimed for making accessible, informative, and *cmashing* plots
- seaborn [5]: for statistical data visualization
- cmap [1]: perceptually uniform colormaps for MATLAB, compiled from multiple sources.
- scientific [6]: accurate data visualization needs and accurate scales readable by everyone.

Some colormaps can fall into more than one category as well, making them effective tools in general cases. *turbo*[7] is one of the most popular colormaps because it can be applied to many different styles of data effectively and was designed with accessibility in mind.



*Figure 1.4: jet (top) and turbo (bottom), both sequential and diverging colormaps. turbo image from Sipkens and Pasquier [1].*

*turbo*, shown at the bottom of Figure 1.4, is a form of rainbow colormap. *tubo* was created to address major faults in *jet*, the top color map of Figure 1.4. *jet* is high in contrast and has distinct color bands which manifest on data, making certain areas seem less smooth than they are. This phenomena is known as `false detail` and should be considered when picking a colormap. The left tail on *jet* is a broad, almost-uniform blue gradient that manifests as one color in grayscale and for the colorblind, making it inaccessible. *turbo* addresses these problems with a perceptually uniform rainbow with well-defined tails, effectively removing the false detail, color bands, and inaccessibility concerns of *jet*.

These rainbow maps are unique as they serve as both sequential and divergent colormaps. They are particularly popular showing details across a range of magnitudes, as the combination of style highlights extremes at both end points and in the middle. Application of rainbow plots to study depth in images allows information at the far-field and near-field to be highlighted without overlooking what lies between.

We analyze *jet* and *turbo* in Figure 1.5, which showcases an image of a road in three sets of three. In each set, three colormaps are applied to the image: *cool*, *jet*, and *turbo*. Each set is then displayed in three different color spectrums: full RGB, that related to protanopia (red-green colorblind), and grayscale. Consider a case where it is important to have information up-close and far away, but also to clearly see when something stands out. One example

might be a self-driving car analyzing its path ahead, where it needs to know where it is, whats coming ahead, and if there is something along the path, like a person crossing the road or another car.

Looking at the full RGB set (set 1), it is clear that *cool* (top image) captures only the closest cars, with everything at a distance either vague or washed out. The ability of *jet* (middle) to capture detail is clear, with street posts and cars even at long-distance standing out. However strong color bands are seen on the closer cars and everything at a certain distance has similar colors of blue. In comparison, *turbo* (bottom) has smooth gradients on the closest cars, the further perceived depth, and builds on the sharp detail of *jet* for all distances.

Sets 2 and 3 showcase the same images as set 1 but for the color scale of protanopia (red-green colorblindness) and a grayscale, respectively. In set 2, *cool* does not change much, which follows as there is minimal red and green, but shows its flaws in the grayscale of set 3 where the image is completely gray with no detail. *jet* has more detail stand out at a distance for set 2, with the blues becoming more defined than in RBG, but the color bars on the closet cars expand. At grayscale, *jet* is able to maintain a lot of detail but fails to maintain a uniform gradient, making it hard to tell depth without perspective of the objects on the street. *turbo* is able to maintain detail at a distance in both set 2 and 3, but loses some of its uniform gradient at grayscale with closer details blending together.

This discussion of colormaps highlights that certain colormaps are effective for specific data analysis and considerations for unconventional RBG scales makes visual communication of that data more accessible to all. In this class, data will generally be either sequential or divergent. Choosing colors that relate to the problem (i.e., *Blues* for water) should be considered, while keeping in mind that not all colors are accessible.

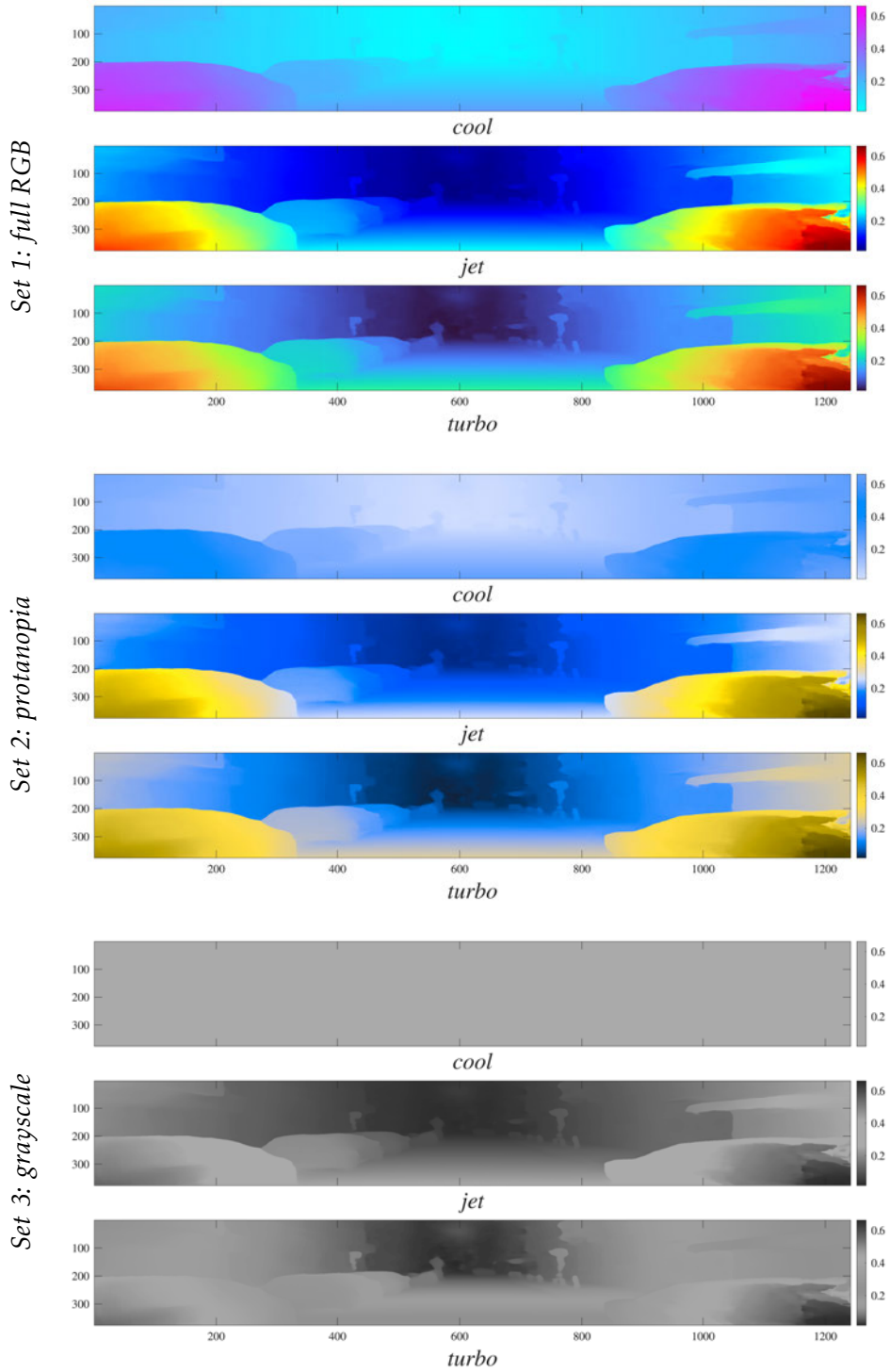



*Figure 1.5: An image of a road in three sets of three. The main set is three colormaps applied to the image: cool, jet, and turbo. This set is then showcased for three different color spectrums: full RGB (Set 1), that related to protanopia/red-green colorblind (Set 2), and grayscale (Set 3).*

### 1.2.2 Implementations

**Lines in python**

```
# plotting package import
import matplotlib.pyplot as plt
# numerics package import
import numpy as np

# LaTeX rendering in Latin Modern
plt.rcParams['text.usetex'] = True
plt.rcParams['text.latex.preamble'] = r'\usepackage{lmodern}'
plt.rcParams['font.family'] = 'serif'

# generate data
dxs = np.logspace(-4, 0, 40)  # grid spacings, 10^-4 to 10^0
y1 = 5.0*(dxs) + 8.0*(dxs**3)     # first order error curve
y2 = 2.0*(dxs**2) + 16.0*(dxs**4) # second order error curve

# reference lines (order one ref. is just dxs itself)
ref2 = dxs**2  # order two reference line

fs = 50  # set font size variable

# this is how you define non-standard colors!
# all these values are the RBG #/255. this accepts both the
# calculation (67/255) and the fraction (0.2627) for RBG triplets (see below)
badCols = np.array([
    [0.5, 0.000, 0.5],  # purple
    [0, 0, 0.9],        # blue
])

cols = np.array([
    [67/255, 179/255, 174/255],   # teal (marker edge/line 1)
    [136/255, 216/255, 192/255],  # light green (marker face 1)
    [198/255, 97/255, 166/255],   # light magenta (marker face 2)
    [202/255, 44/255, 146/255],   # magenta (marker edge/line 2)
])

# define figure, axes, and size
fig, ax = plt.subplots(figsize=(13, 9))

# bad color choice example (uses badCols)
# ax.plot(dxs, y1, '-', color=badCols[0, :])
# ax.plot(dxs, y2, '-', color=badCols[1, :])

# reference lines (excluded from legend)
ax.plot(dxs[:20], dxs[:20], '-.', color='black',
        linewidth=5,
        label='_nolegend_') # this excludes this line from the legend
ax.plot(dxs[:20], 0.3*ref2[:20], '--', color='black',
        linewidth=5, label='_nolegend_')

# plots y1 vs dxs every third point as solid line with markers
ax.plot(dxs[::3], y1[::3], '-o',
        color=cols[0, :],            # set line color
        linewidth=2,                 # set line width
        markersize=24,               # set marker size
        markeredgewidth=3,           # set marker outline width
```

```
        markeredgecolor=cols[0, :],  # set marker outline color
        markerfacecolor=cols[1, :],  # set marker fill color
        label='first order')         # set legend entry

ax.plot(dxs[::3], y2[::3], '-*', color=cols[3, :], linewidth=2,
        markersize=40, markeredgewidth=3, markeredgecolor=cols[3, :],
        markerfacecolor=cols[2, :], label='second order')

# label reference slopes directly on the plot
ax.annotate(r'$\mathcal{O}(\Delta x)$',
            (1.3*dxs[5], 0.3*dxs[5]), fontsize=0.75*fs)
ax.annotate(r'$\mathcal{O}(\Delta x^2)$',
            (2.3*dxs[5], 0.4*(dxs[5]**2)), fontsize=0.75*fs)

# the multiplications you see above are simply translating the position to get
# the labels off the line but close to it!

for spine in ax.spines.values():  # set axes line width
    spine.set_linewidth(3)
ax.tick_params(width=3, labelsize=fs)  # set tick width and font size
ax.tick_params(axis='x', pad=10)       # pad ticks away from axis (preventing
    overlap)

ax.set_xlabel(r'$\Delta x$  (mm)', fontsize=fs, labelpad=2)
ax.set_ylabel(r'Error', fontsize=fs)

ax.legend(fontsize=0.75*fs, loc='lower right')
ax.set_title('Convergence Plot', fontsize=fs)

# set the axis scale to log log
ax.set_xscale('log')
ax.set_yscale('log')
ax.grid() # sets major grid lines - can have settings here

plt.show()

## note: fig will not save if plt.show() is before plt.savefig(...)!!!!
# plt.savefig(linePlotExample.pdf, dpi=800)  # uncomment to export figure (dpi sets
    raster resolution)
```

### Fields in python

```
# plotting package import
import matplotlib.pyplot as plt
# numerics package import
import numpy as np
# import nicer colormaps
import cmocean

# LaTeX rendering in Latin Modern
plt.rcParams['text.usetex'] = True
plt.rcParams['text.latex.preamble'] = r'\usepackage{lmodern}'
plt.rcParams['font.family'] = 'serif'

# generate data
dx = 0.01
x = np.arange(-np.pi, np.pi + dx, dx)  # + dx includes endpoint
y = x
```

```
X, Y = np.meshgrid(x, y) # creates mesh of x and y (2D fields)

# diverging colormap example (use cmap='cmo.curl' in pcolormesh)
#############################################################
R = np.sqrt(X**2 + Y**2)
f = Y * np.sin(2.0 * R**2)
#############################################################

# sequential colormap example (use cmap='Blues' in pcolormesh)
#############################################################
# f = R * 1.3**(1.01*(abs(abs(X)*abs(Y) - 2*np.pi)) + np.pi/2)
#############################################################

fs = 35  # set font size variable
cm = 'cmo.curl' # 'Blues'

fig, ax = plt.subplots(figsize=(12, 9))  # set figure size

ax.set_aspect('equal')  # force square aspect ratio

# pseudocolor plot of f across X and Y
# gouraud shading interpolates between cells (no mesh lines)
pc = ax.pcolormesh(X, Y, f, cmap=cm, shading='gouraud')

ax.set_xlabel('$x$', fontsize=fs)  # label x-axis in latex
ax.set_ylabel('$y$', fontsize=fs)  # label y-axis in latex

for spine in ax.spines.values():  # set axes line width
    spine.set_linewidth(3)
ax.tick_params(width=3, labelsize=fs)  # set tick width and font size

axesLims = [-np.pi, np.pi]  # define axes limits
ax.set_xlim(axesLims)       # set x axis limits
ax.set_ylim(axesLims)       # set y axis limits

cb = fig.colorbar(pc, ax=ax)                  # add colorbar
cb.ax.tick_params(labelsize=fs)               # set colorbar tick size
cb.set_label('Field Magnitude', fontsize=fs)  # label colorbar

# tick locations and latex labels
ticks = [-np.pi, -np.pi/2, 0, np.pi/2, np.pi]
tickLabel = ['$-\\pi$', '$-\\pi/2$', '$0$', '$\\pi/2$', '$\\pi$']
ax.set_xticks(ticks)           # set x tick locations
ax.set_yticks(ticks)           # set y tick locations
ax.set_xticklabels(tickLabel)  # set x tick labels to tex strings
ax.set_yticklabels(tickLabel)  # set y tick labels to tex strings

# plt.show()  # uncomment to view interactively

## note: fig will not save if plt.show() is before plt.savefig(...)!!!!
# plt.savefig(divergencePlotExample.pdf, dpi=800)  # uncomment to export figure
# (dpi sets raster resolution)

#### 3D Plot

fig, ax = plt.subplots(subplot_kw={projection: 3d}, figsize=(12,9))

ax.plot_surface(X, Y, f, cmap=cm)
```

```
fs = 0.8*fs

ticks = [-np.pi, -np.pi/2, 0, np.pi/2, np.pi]  # defining location of ticks
tickLabel = ['$-\\pi$', '$-\\pi/2$', '$0$', '$\\pi/2$', '$\\pi$']  # defining
    labels for ticks

ax.set_xticks(ticks)            # set x ticks to ticks array
ax.set_yticks(ticks)            # set y ticks to ticks array
ax.set_xticklabels(tickLabel)   # set labels to tex strings in tickLabel
ax.set_yticklabels(tickLabel)   # set labels to tex strings in tickLabel

ax.set_xlabel('$x$', fontsize=fs, labelpad=20)  # label x-axis as x in latex
ax.set_ylabel('$y$', fontsize=fs, labelpad=20)
ax.set_ylabel('$f(x,y)$', fontsize=fs, labelpad=20)

for spine in ax.spines.values():   # set axes linewidth
    spine.set_linewidth(3)

ax.tick_params(width=3, labelsize=fs)  # set tick width and font size
axesLims = [-np.pi - 0.3, np.pi + 0.3]  # define axes limits
ax.set_xlim(axesLims)       # set x axis limits
ax.set_ylim(axesLims)       # set y axis limits

plt.show()

# plt.savefig(seq3D.pdf, dpi=800)
```

**Lines in MATLAB**

```
% generate data
dxs = logspace(-4, 0, 40);         % grid spacings, 10^-4 to 10^0

y1 = 5.0*dxs + 8.0*dxs.^3;         % first order error curve
y2 = 2.0*dxs.^2 + 16.0*dxs.^4;     % second order error curve

% reference lines (order one ref. is just dxs itself)
ref2 = dxs.^2;  % order two reference line
fs = 50;  % set font size variable

% rgb colors, defined 0-1
badCols = [0.5 0.0 0.5;    % purple
           0.0 0.0 0.9];   % blue

cols = [ 67 179 174;   % teal (marker edge/line 1)
        136 216 192;   % light green (marker face 1)
        198  97 166;   % light magenta (marker face 2)
        202  44 146] / 255;  % magenta (marker edge/line 2)

% define figure, axes, and size
fig = figure( ...
    Theme, light, ...               % plot light even in dark mode
    'Position', [100, 100, 1300, 900]); % match figsize(13,9)

hold on  % ensures all following plot commands go onto fig

% bad color choice example (uses badCols)
% plot(dxs, y1, '-', 'Color', badCols(1,:))
```

```
% plot(dxs, y2, '-', 'Color', badCols(2,:))

% reference lines (excluded from legend, truncated to lower half)
plot(dxs(1:20), dxs(1:20), '-.', 'Color', 'black', ...
    'LineWidth', 5, 'HandleVisibility', 'off')
plot(dxs(1:20), 0.3*ref2(1:20), '--', 'Color', 'black', ...
    'LineWidth', 5, 'HandleVisibility', 'off')

% plots y1 vs dxs every third point as solid line with markers
plot(dxs(1:3:end), y1(1:3:end), '-o', ...
    'Color', cols(1,:), ...             % set line color
    'LineWidth', 2, ...                 % set line width
    'MarkerSize', 24, ...               % set marker size
    'MarkerEdgeColor', cols(1,:), ...   % set marker outline color
    'MarkerFaceColor', cols(2,:), ...   % set marker fill color
    'DisplayName', 'first order')       % set legend entry
plot(dxs(1:3:end), y2(1:3:end), '-p', 'Color', cols(4,:), ...
    'LineWidth', 2, 'MarkerSize', 40, ...
    'MarkerEdgeColor', cols(4,:), 'MarkerFaceColor', cols(3,:), ...
    'DisplayName', 'second order')

% label reference slopes directly on the plot
text(1.3*dxs(6), 0.3*dxs(6), '$$\mathcal{O}(\Delta x)$$', ...
    'Interpreter', 'latex', 'FontSize', 0.75*fs)
text(2.3*dxs(6), 0.4*dxs(6)^2, '$$\mathcal{O}(\Delta x^2)$$', ...
    'Interpreter', 'latex', 'FontSize', 0.75*fs)

ax = gca;  % pull the current axes
set(ax, ...
    'XScale', 'log', ...                % set x-scale to log
    'YScale', 'log', ...                % set y-scale to log
    'LineWidth', 3, ...                 % set axes line width
    'FontSize', fs, ...                 % set tick font size
    'TickLabelInterpreter', 'latex'); % set tick interpreter to latex

grid on  % turn on major grid

xlabel('$$\Delta x$$  (mm)', ...  % label x-axis in latex
    'Interpreter', 'latex', ...   % interpret label with latex
    'FontSize', fs);              % set font size of label
ylabel('Error', 'Interpreter', 'latex', 'FontSize', fs);
legend('FontSize', 0.75*fs, ...   % set legend font size
    'Location', 'southeast', ...  % legend bottom right
    'Interpreter', 'latex');      % interpret with latex
title('Convergence Plot', ...    % set plot title
    'Interpreter', 'latex', ...  % interpret title in latex
    'FontSize', fs);             % set font size of title

hold off

saveas(fig, 'convergencePlot.pdf')  % export figure as a pdf
```

**Fields in MATLAB**

```
% point matlab to any cmap directories you download
% needed for curl and blues
addpath('<path to directory>/cmap/')

% generate data
dx = 0.01;
x = -pi:dx:pi;
y = x;
[X, Y] = meshgrid(x, y);  % creates mesh of x and y (2D fields)

% diverging colormap example (use colormap('curl'))
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
R = sqrt(X.^2 + Y.^2);
f = Y.*sin(2.0*R.^2);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% sequential colormap example (use colormap('Blues'))
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
f = R .* 1.3.^(1.01*(abs(abs(X).*abs(Y) - 2*pi)) + pi/2);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

fs = 35;  % set font size variable
cm = blues(256);  % set colormap to match the example above

fig = figure( ...
    Theme, light, ...                % plot light even in dark mode
    'Position', [100, 100, 1200, 900]); % match figsize(12,9)

hold on  % ensures all following plot commands go onto fig
daspect([1 1 1])  % force square aspect ratio

% pseudocolor plot of f across X and Y
pc = pcolor(X, Y, f);
colormap(cm);
shading interp     % interpolate between cells (matches gouraud)

xlabel('$$x$$', ...              % label x-axis in latex
    'Interpreter', 'latex', ...  % interpret label with latex
    'FontSize', fs);             % set font size of label
ylabel('$$y$$', 'Interpreter', 'latex', 'FontSize', fs);

ax = gca;  % pull the current axes
set(ax, 'LineWidth', 3, 'FontSize', fs)  % set axes width, font size
axesLims = [-pi, pi];  % define axes limits
xlim(axesLims)         % set x axis limits
ylim(axesLims)         % set y axis limits

cb = colorbar('TickLabelInterpreter', 'latex', ...  % add colorbar
    'FontSize', fs);                                % set tick size
ylabel(cb, 'Field Magnitude', ...  % label colorbar
    'FontSize', fs, ...            % set font size
    'Interpreter', 'latex');       % set label interpreter

% tick locations and latex labels
ticks = [-pi, -pi/2, 0, pi/2, pi];
tickLabel = {'$$-\pi$$', '$$-\pi/2$$', '$$0$$', ...
```

```
            '$$\pi/2$$', '$$\pi$$'};
ax.TickLabelInterpreter = 'latex';  % set tick interpreter to latex
ax.XTick = ticks;                   % set x tick locations
ax.YTick = ticks;                   % set y tick locations
ax.XTickLabel = tickLabel;          % set x tick labels to tex strings
ax.YTickLabel = tickLabel;          % set y tick labels to tex strings

hold off

%% note: exports whatever figure is current!
% exportgraphics(fig, 'div.pdf', 'Resolution', 800)

%%---------------------------
%% 3D PLOT
%%---------------------------

fig = figure( ...
    Theme, light, ...               % plot light even in dark mode
    'Position', [100, 100, 1200, 900]); % match figsize(12,9)

% surface plot of f across X and Y
s = surf(X, Y, f, 'EdgeColor', 'none'); % EdgeColor none removes mesh lines
colormap(cm);  % set colormap (cm defined above)

fs = 0.8*fs;  % shrink fonts for the 3d view

% tick locations and latex labels
ticks = [-pi, -pi/2, 0, pi/2, pi];
tickLabel = {'$$-\pi$$', '$$-\pi/2$$', '$$0$$', ...
             '$$\pi/2$$', '$$\pi$$'};

ax = gca;  % pull the current axes
ax.TickLabelInterpreter = 'latex';  % set tick interpreter to latex
ax.XTick = ticks;                   % set x tick locations
ax.YTick = ticks;                   % set y tick locations
ax.XTickLabel = tickLabel;          % set x tick labels to tex strings
ax.YTickLabel = tickLabel;          % set y tick labels to tex strings

xlabel('$$x$$', ...              % label x-axis in latex
    'Interpreter', 'latex', ...  % interpret label with latex
    'FontSize', fs);             % set font size of label
ylabel('$$y$$', 'Interpreter', 'latex', 'FontSize', fs);
zlabel('$$f(x,y)$$', 'Interpreter', 'latex', 'FontSize', fs);

set(ax, 'LineWidth', 3, 'FontSize', fs)  % set axes width, font size
axesLims = [-pi - 0.3, pi + 0.3];  % define axes limits (small margin)
xlim(axesLims)                     % set x axis limits
ylim(axesLims)                     % set y axis limits

%% note: exports whatever figure is current!
% exportgraphics(fig, 'seq3D.pdf', 'Resolution', 800)
```

# 2. Finite Difference Methods


2.1 The Basics 23

2.2 First Derivative Approximations 24

2.3 Second Derivative Approximations 26


Finite difference methods (FDM) are collocation methods based on Taylor series approximation and are used to approximate derivatives and derivative operators. They are an old and simple numerical method which became popular after the development of computers due to the sheer amount of arithmetic for application. In this chapter, we will define numerical schemes to approximate first and second derivatives. These schemes can then be extended to help solve PDEs by approximating derivatives and derivative operators ($\nabla, \nabla^2$).

## 2.1 The Basics

Consider a domain $[a,b] \subset \mathbb{R}$ and a grid of equally spaced points $\{x_i\}_{i=0}^N \subset [a,b]$ such that $x_0 = a$ and $x_N = b$. The spacing between neighboring points can be computed as $\Delta x = \frac{b-a}{N}$ so that $x_i = a + i\,\Delta x$. Given some function $\phi : [a,b] \to \mathbb{R}$, we denote $\phi_i \equiv \phi(x_i)$.

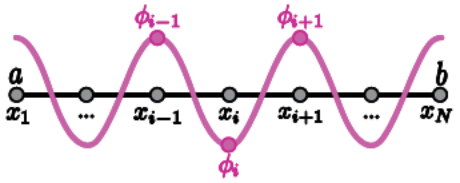



*Figure 2.1: A smooth function $\phi$ sampled on the equally spaced grid $\{x_i\}$ partitioning $[a,b]$. $\phi$ is discretely sampled on the grid, where $\phi_i \equiv \phi(x_i)$.*

Recall that a sufficiently smooth function can be written as a Taylor series about a single point. Assuming $\phi$ is smooth, its Taylor series about some point $x_i$ is

$$\phi(x) = \phi_i + \phi_i' \Delta x + \frac{\phi_i''}{2} \Delta x^2 + \cdots \tag{2.1}$$

for $x \in [a,b]$. The finite difference approach evaluates $\phi$ at respective $x \in \{x_i\}_{i=0}^N$ in terms of a Taylor series about one of its neighbors (i.e., $\phi_i$), where the series can be truncated for sufficiently small $\Delta x$. The remaining series terms are rearranged to produce approximations for various derivatives with well-defined error from the truncated terms.

## 2.2 First Derivative Approximations

We will discuss the forward, backward, and central difference scheme for approximating $\phi_i'$. The former two are first-order schemes and the latter is second-order.

To discuss finite difference schemes, we will adopt the compass notation. For a grid point of interest $P$, its neighbor to the west (decreasing $x$) is $W$ and its neighbor to the east (increasing $x$) is $E$.

### 2.2.1 Forward difference

Following from (2.1), $\phi_{i+1}$ can be approximated as

$$\phi_{i+1} = \phi_i + \phi_i' \Delta x + \frac{\phi_i''}{2}(\Delta x^2) + \dots$$

Solving this approximation for $\phi_i'$ yields

$$\phi_i' = \frac{\phi_{i+1} - \phi_i}{\Delta x} - R, \qquad R = \frac{\phi_i''}{2}\Delta x + \dots,$$

where $R$ is the local truncation error. The magnitude of error $R$ is controlled by the leading retained term $\frac{1}{2}\phi_i''\Delta x$, which is proportional to $\Delta x$. This error is generally recorded using Big $\mathcal{O}$ notation,

$$\phi_i' = \frac{\phi_{i+1} - \phi_i}{\Delta x} + \mathcal{O}(\Delta x), \quad \phi_P' = \frac{\phi_E - \phi_P}{\Delta x} + \mathcal{O}(\Delta x), \tag{2.2}$$

Equation (2.2) is the *first-order forward difference* approximation of $\phi_i'$ and the points of interest are highlighted in Figure 2.2. The word forward refers to the fact that the Taylor series around $x_i$ estimates $\phi$ at the next or forward point $x_{i+1}$. Note how derivative approximation resembles the limit definition of the derivative!




*Figure 2.2: The forward difference stencil, highlighting the used points $P, E$ in cyan.*

### 2.2.2 Backward difference

Estimating the previous point $\phi_{i-1}$ in terms of (2.1) yields

$$\phi_{i-1} = \phi_i - \phi_i' \Delta x + \frac{\phi_i''}{2}\Delta x^2 - \frac{\phi_i'''}{6}\Delta x^3 + \cdots, \tag{2.3}$$

The odd terms of the series are now negative, as $(x_i - \Delta x) - x_i = -\Delta x$. Solving 2.3 for $\phi'(x_i)$ gives the *first-order backward difference*,

$$\phi'(x_i) = \frac{\phi_i - \phi_{i-1}}{\Delta x} + \mathcal{O}(\Delta x), \quad \phi'(x_P) = \frac{\phi_P - \phi_W}{\Delta x} + \mathcal{O}(\Delta x). \tag{2.4}$$




*Figure 2.3: The backward difference stencil, highlighting the used points P, W in cyan.*

The stencil for this scheme is showcased in Figure 2.3. The word backwards refers to the fact that the Taylor series around $x_i$ estimates $\phi$ at the last or backward point $x_{i-1}$.

### 2.2.3 Central difference

Reorganizing the forward and backwards Taylor series approximations yields

$$\phi'_i = \frac{\phi_{i+1} - \phi_i}{\Delta x} - \frac{\phi''_i}{2}\Delta x - \frac{\phi'''_i}{6}\Delta x^2 + \cdots \tag{2.5}$$

$$\phi'_i = \frac{\phi_i - \phi_{i-1}}{\Delta x} + \frac{\phi''_i}{2}\Delta x - \frac{\phi'''_i}{6}\Delta x^2 + \cdots \tag{2.6}$$

Adding (2.5) and (2.6) and dividing by a factor of two obtains a second-order accurate scheme:

$$\phi'(x_i) = \frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} - \frac{\phi'''(x_i)}{6}\Delta x^2 + \cdots = \frac{\phi_{i+1} - \phi_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2). \tag{2.7}$$

Using our compass notation, the central difference scheme is written as

$$\phi'(x_P) = \frac{\phi_E - \phi_W}{2\Delta x} + \mathcal{O}(\Delta x^2).$$

This stencil is highlighted in Figure 2.4.




*Figure 2.4: The central difference stencil, highlighting the used points E, W in cyan.*

## 2.3 Second Derivative Approximations

To approximate $\phi_i''$ we estimate in the forward and backward directions:

$$\phi_{i+1} = \phi_i + \phi_i' \Delta x + \frac{\phi_i''}{2} \Delta x^2 + \frac{\phi_i'''}{6} \Delta x^3 + \frac{\phi_i''''}{24} \Delta x^4 + \cdots, \tag{2.8}$$

$$\phi_{i-1} = \phi_i - \phi_i' \Delta x + \frac{\phi_i''}{2} \Delta x^2 - \frac{\phi_i'''}{6} \Delta x^3 + \frac{\phi_i''''}{24} \Delta x^4 - \cdots. \tag{2.9}$$

Adding (2.8) and (2.9), the odd-order terms cancel and

$$\phi_{i+1} + \phi_{i-1} = 2\phi_i + \phi_i'' \Delta x^2 + \frac{\phi_i''''}{12} \Delta x^4 + \cdots.$$

Solving for $\phi_i''$ gives the second-order central difference scheme for the second derivative:

$$\phi_i'' = \frac{\phi_{i+1} - 2\phi_i + \phi_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2). \tag{2.10}$$

On their own, these approximations only tell us how $\phi$ changes locally; to actually solve a differential equation we must define how $\phi$ behaves at the edges of the domain through *boundary conditions* which rely on *ghost points*. We will use an example problem in the next section to define two styles of spatial discretization and how to use ghost points to define boundary conditions.

# 3. One-Dimensional FDMs in Practice


| | | |
|---|---|---|
| 3.1 | Type I Grids | 28 |
| 3.2 | Type II Grids | 29 |
| 3.3 | Boundary Conditions | 29 |
| 3.4 | Basic Numerical Implementation | 33 |


Consider the second-derivative operator $D_{xx} \equiv \frac{d^2}{dx^2}$ acting on a smooth function $\phi : [a, b] \to \mathbb{R}$. We want to use finite difference methods to solve the following for $\phi$:

$$D_{xx}\phi = -\sin(x), \qquad x \in [a, b] \tag{3.1}$$

Equation (3.1) tells us that the second derivative of $\phi$ equals $f(x) = -\sin(x)$. Integrating $f$ twice reveals that

$$\phi'(x) = \cos(x) + c_1, \tag{3.2}$$
$$\phi(x) = \sin(x) + c_1\, x + c_2, \tag{3.3}$$

where $c_1, c_2 \in \mathbb{R}$ are constants of integration. Notice that (3.3) is not a single solution but an entire two-parameter family: any choice of $c_1$ and $c_2$ satisfies (3.1).

*Remark* 1. To single out a solution we must supply extra pieces of information through boundary conditions.

Boundary conditions are almost always imposed at the ends of the domain, which are $x = a$ and $x = b$ for this problem. For example, on $[a, b] = [0, 2\pi]$ with the conditions $\phi(0) = 0$ and $\phi(2\pi) = 0$, the two constants in (3.3) are forced to $c_1 = c_2 = 0$, leaving the unique exact solution $\phi(x) = \sin(x)$. We will return to this case as a convenient way to check our numerical schemes.

We can think about this need for boundary conditions in terms of our discrete grid as well. Suppose we sample $\phi$ on the equally spaced grid seen in Figure 3.1 and approximate $D_{xx}\phi$ at each grid point $P$ using the second-order central difference

$$D_{xx}\phi\big|_P \approx \frac{\phi_E - 2\phi_P + \phi_W}{\Delta x^2}, \tag{3.4}$$

we would end up with this linear system:

$$\frac{1}{\Delta x^2}\begin{bmatrix} ? & ? & ? & ? & \cdots & ? \\ 1 & -2 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & -2 & 1 \\ ? & \cdots & ? & ? & ? & -? \end{bmatrix}\begin{bmatrix} \phi_0 \\ \phi_1 \\ \vdots \\ \vdots \\ \phi_{N-1} \\ \phi_N \end{bmatrix} = \begin{bmatrix} -\sin(x_0) \\ -\sin(x_1) \\ \vdots \\ \vdots \\ -\sin(x_{N-1}) \\ -\sin(x_N) \end{bmatrix}. \tag{3.5}$$

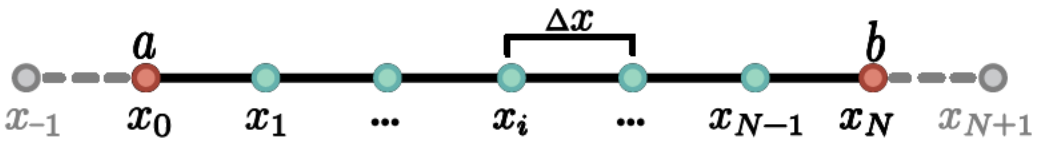



*Figure 3.1: A standard grid discretization of some real-valued domain $[a,b]$. The domain is discretized by $N+1$ equally-spaced points a distance $\Delta x$ apart. Exterior points (orange) at domain boundaries $x=a,b$ highlighted, as well as ghost points $x_{-1}$ and $x_{N+1}$.*

At any *interior* (cyan) point the three values $W,P,E$ are all available and (3.4) is well defined. But at the *exterior* (orange) grid points $x_0$ and $x_N$, the stencil calls for $\phi_{-1}$ and $\phi_{N+1}$... these values lie outside the grid! The linear system is then left with more unknowns than equations, so our linear system is under-determined*. To create a well-determined system and force a unique solution, we must define these values that lie outside the grid, which we call *ghost points*. Note that one could instead use first-order one-sided differences at the boundaries, but this lowers the order of the boundary approximation and can degrade the accuracy of the whole scheme, so it is generally avoided in favor of the ghost-point construction.

Where these ghost point actually sits depends on how we lay the grid down relative to the boundary. The next two sections describe the two standard choices for grids.

## 3.1 Type I Grids

A **Type I** (or *nodal*, *vertex-centered*) grid places discrete points *on* the boundaries $a,b$. We have been already thinking about discretization on a Type I grid, which defines discrete points $x_i \in [a,b]$ as

$$x_i = a + i\Delta x, \quad i = 0,1,\ldots,N, \qquad \Delta x = \frac{b-a}{N}, \tag{3.6}$$

*Note that if we were to not notice this, and set $D_{xx}(0,0)=2, D_{xx}(0,1)=-1$, etc., the system would solve and give a real solution. Consider why this is, and determine what it implies about the boundaries!

so that $x_0 = a$ and $x_N = b$ are themselves grid points. The ghost points are the two nodes obtained by stepping one $\Delta x$ past each end,

$$x_{-1} = a - \Delta x, \qquad x_{N+1} = b + \Delta x, \tag{3.7}$$

shown as the grey or first/last points in Figure 3.2. At the left boundary $x = a$ we can lay our finite difference stencil, seen as the shaded region, where $P = x_0$, $W = x_{-1}$, and $E = x_1$. The same idea can be applied to the right boundary $x = b$, where $P = x_N$, $E = x_{N+1}$, and $W = x_{N-1}$.

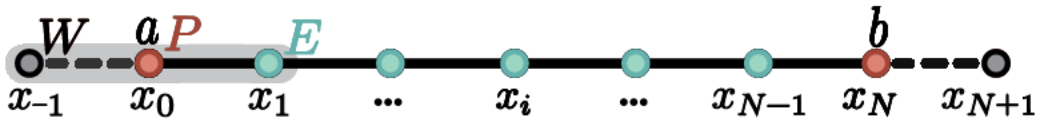



*Figure 3.2: A Type I (nodal) grid on $[a,b]$. Interior points $x_1,\ldots,x_{N-1}$ seen in lighter cyan and exterior points $x_0, x_N$ seen as darker orange. Ghost nodes are first and last points defined in gray and black. Finite different stencil for $P = x_0$ highlight as grey stroke, with contributors P, E, and W labelled.*

A defining feature of a Type I grid is that implementing boundary conditions is direct, since the boundary value is a nodal value.

## 3.2 Type II Grids

A **Type II** (or *cell-centered*) grid instead partitions $[a,b]$ into $N$ **cells** of width $\Delta x = \frac{b-a}{N}$ and places the discrete points of $x$ at the cell *centers*. This results in one less discrete point when compared to Type I grids. The cell *faces* fall on the boundaries, so no discrete point $x$ lies on the domain edge. The cell centers are

$$x_{i+1/2} = a + \left(i + \tfrac{1}{2}\right)\Delta x, \quad i = 0, 1, \ldots, N-1 \tag{3.8}$$

and the faces still sit at $x_i = a + i\,\Delta x$. This cell idea is depicted for the discrete point $x_i$ in Figure 3.3. The two boundaries still sit at $x_0$ and $x_N$, but the nearest grid point is half a spacing away. The ghost points lie half a spacing outside each end of the domain, as seen by the gray and black crosses in Figure 3.3. At the left boundary, we can lay our finite difference stencil, seen as the gray shaded region, where $P = x_{1/2}$, $W = x_{-1/2}$, and $E = x_{3/2}$, and the boundary itself is the west face of $x_{1/2}$ and east face of $x_{-1/2}$.

A defining feature for Type II grids is that the domain boundaries sit *exactly halfway* between the ghost points and the corresponding interior point. A difference across $W$ and $P$ is therefore naturally centered on the boundary. We will discuss later that layout makes first-order methods second-order accurate at the cell faces without any extra work.

## 3.3 Boundary Conditions

We will now define the three boundary conditions used in this text and, for each, give the ghost value it implies on both grid types. Throughout we write the prescribed left-boundary




*Figure 3.3: A Type II (cell-centered) grid on $[a,b]$ shown as stars. Interior points $x_{3/2},\dots,x_{N-3/2}$ seen in lighter cyan and exterior points $x_{1/2},x_{N-1/2}$ seen as darker orange. Ghost nodes are first and last points defined in gray and black. Finite different stencil for $P = x_{1/2}$ highlight as grey stroke, with contributors $P,E$, and $W$ labelled. The cell definition is labelled for $x_{i+1/2}$, with the diameter $\Delta x$ and region referred to as the cell face labelled.*

value as $\alpha \in \mathbb{R}$ and the right-boundary value as $\beta \in \mathbb{R}$, and derive the left boundary in detail. The right boundary follows by symmetry and is left as an exercise.

### 3.3.1 Periodic Boundaries

A **periodic** boundary condition treats the domain as a loop: the solution $\phi$ and its derivatives at $x = a$ match those at $x = b$. This is implemented by setting ghost points as the opposite end of the domain.

**Type I.** Because the nodes $x_0 = a$ and $x_N = b$ represent the same physical location, we carry the $N$ distinct unknowns $\phi_0,\dots,\phi_{N-1}$ and remove $\phi_N \equiv \phi_0$ from our system as a known value. Stepping west past the first node wraps around to the last interior node, so the ghost is

$$\phi_W = \phi_{-1} \equiv \phi_{N-1}. \tag{3.9}$$

For the first row of the system 3.5, this would look like

$$\frac{1}{\Delta x^2}\begin{bmatrix}-2 & 1 & 0 & 0 & \cdots & 0 & 1\end{bmatrix}\begin{bmatrix}\phi_0 & \phi_1 & \phi_2 & \phi_3 & \cdots & \phi_{N-1}\end{bmatrix}^T = \begin{bmatrix}-\sin(x_0)\end{bmatrix}. \tag{3.10}$$

**Type II.** The cells wrap directly: the left ghost cell is a copy of the last interior cell and the right ghost cell is a copy of the first,

$$\phi_W = \phi_{-1/2} \equiv \phi_{N-1/2} \tag{3.11}$$

This would be imposed just the same as (3.10).

### 3.3.2 Dirichlet

A **Dirichlet** boundary condition prescribes a *value* $\alpha$ of $\phi$ at the boundary, such that $\phi(a) = \alpha$. Consider how this might be described in two different ways within our linear system for the Type I grid, and why there is only one method for a Type II grid.

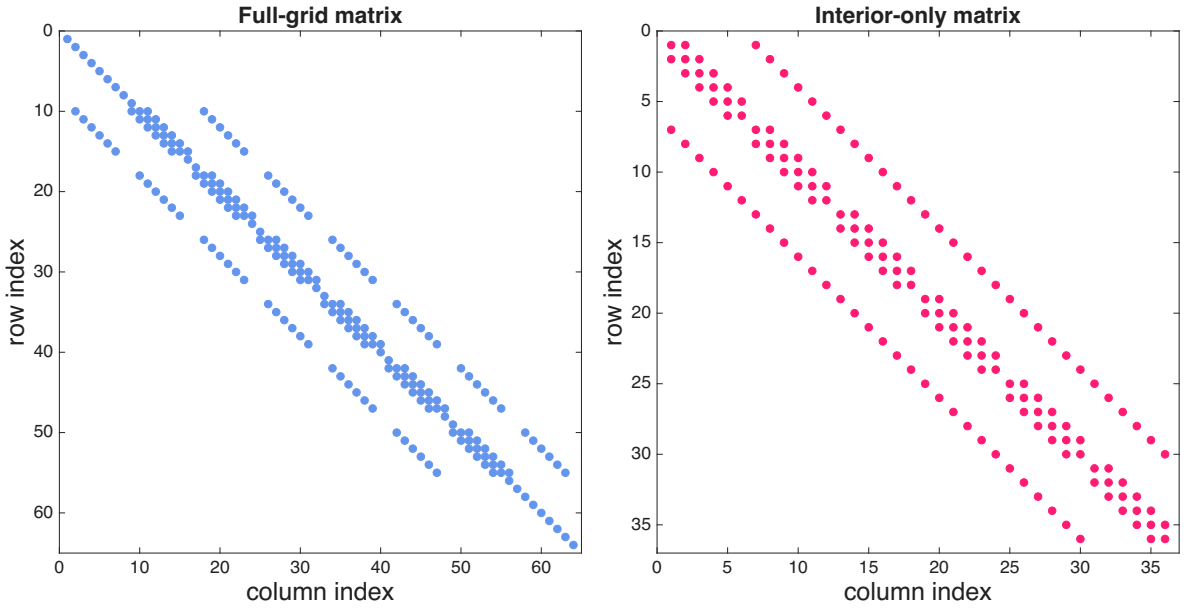



*Figure 3.4: Sparsity pattern produced by* `spy(L)` *for equivalent Laplacian operators L, either including exterior points (left) or excluding exterior points (right).*

**Type I.** The boundary value lives at the node $P = x_0$, so it is imposed directly,

$$\phi_P = \alpha. \tag{3.12}$$

One method for implementing this condition is by writing the first row of the system (3.5) as

$$\frac{1}{\Delta x^2}\begin{bmatrix}1 & 0 & \cdots & 0\end{bmatrix}\begin{bmatrix}\phi_0 & \phi_1 & \cdots & \phi_N\end{bmatrix}^T = \begin{bmatrix}\alpha\end{bmatrix}. \tag{3.13}$$

Alternatively, we can treat $\phi_0 = \alpha$ and $\phi_N = \beta$ as known and remove them from the unknowns, leaving a linear system for $\phi_1, \ldots, \phi_{N-1}$ and removing the first and last rows of the system (3.5). Moving the known value across the linear system, the (now first) row for $\phi_1$ becomes

$$\frac{1}{\Delta x^2}\begin{bmatrix}-2 & 1 & 0 & \cdots & 0\end{bmatrix}\begin{bmatrix}\phi_1 & \phi_2 & \phi_3 & \cdots & \phi_{N-1}\end{bmatrix}^T = \begin{bmatrix}-\sin(x_1) - \frac{\alpha}{\Delta x^2}\end{bmatrix}. \tag{3.14}$$

Both methods are equivalent, but produce different Laplacian matricies, as seen in 3.4

**Type II.** On a Type II grid there is no discrete point at the boundary, so we can only implement the condition on the right-hand side of our linear system. We approximate the boundary as the midpoint between $x_{-1/2}$ and $x_{1/2}$:

$$\alpha = \frac{\phi_{-1/2} + \phi_{1/2}}{2} \implies \phi_{-1/2} = 2\alpha - \phi_{1/2}$$

Plugging this into (3.4) yields

$$\frac{-3\phi_{1/2}+\phi_{3/2}}{\Delta x^2} = -\sin(x_{1/2}) - \frac{2\alpha}{\Delta x^2}. \tag{3.15}$$

The first row of our system (3.5) then becomes

$$\frac{1}{\Delta x^2}\begin{bmatrix}-3 & 1 & 0 & \cdots & 0\end{bmatrix}\begin{bmatrix}\phi_{1/2} & \phi_{3/2} & \phi_{5/2} & \cdots & \phi_{N-1/2}\end{bmatrix}^T = \begin{bmatrix}-\sin(x_{1/2}) - \frac{2\alpha}{\Delta x^2}\end{bmatrix}. \tag{3.16}$$

### 3.3.3 Neumann

A **Neumann** condition prescribes the first derivative at the boundary, $\phi'(a) = \alpha$.

**Type I.** Since we have the boundary as a discrete point $x_0 = a$, we can implement the central difference scheme for the first derivate (2.7) around $a$ :

$$\phi'(a) \approx \frac{\phi_E - \phi_W}{2\Delta x} = \alpha \quad \Longrightarrow \quad \phi_W = \phi_E - 2\Delta x\,\alpha. \tag{3.17}$$

Substituting (3.17) into (3.4) at $P = x_0$ gives

$$D_{xx}\phi\big|_P \approx \frac{2\phi_E - 2\phi_P}{\Delta x^2} - \frac{2\alpha}{\Delta x}. \tag{3.18}$$

The first row of our system (3.5) then becomes

$$\frac{1}{\Delta x^2}\begin{bmatrix}-2 & 2 & 0 & \cdots & 0\end{bmatrix}\begin{bmatrix}\phi_0 & \phi_1 & \phi_2 & \cdots & \phi_N\end{bmatrix}^T = \begin{bmatrix}-\sin(x_0) + \frac{2\alpha}{\Delta x}\end{bmatrix}. \tag{3.19}$$

**Type II.** The derivative is specified at the shared face of cells $P = x_{1/2}$ and $W = x_{-1/2}$, which lies exactly halfway between them at $x = a$. Instead of relying on the central difference, the backwards difference scheme can be implemented and is second-order accurate at its center $x = a$ as written. Therefore,

$$\phi'(a) \approx \frac{\phi_P - \phi_W}{\Delta x} = \alpha \quad \Longrightarrow \quad \phi_W = \phi_P - \Delta x\,\alpha. \tag{3.20}$$

Substituting (3.20) into (3.4) at $P = x_{1/2}$ gives the boundary row

$$D_{xx}\phi\big|_P \approx \frac{\phi_E - \phi_P}{\Delta x^2} - \frac{\alpha}{\Delta x}. \tag{3.21}$$

The first row of our system (3.5) then becomes

$$\frac{1}{\Delta x^2}\begin{bmatrix}-1 & 1 & 0 & \cdots & 0\end{bmatrix}\begin{bmatrix}\phi_{1/2} & \phi_{3/2} & \phi_{5/2} & \cdots & \phi_{N-1/2}\end{bmatrix}^T = \begin{bmatrix}-\sin(x_{1/2}) + \frac{\alpha}{\Delta x}\end{bmatrix}. \tag{3.22}$$

### 3.3.4 Assembling the Discrete System

Now you are prepared to implement finite difference methods for a variety of cases. Let's build two linear systems to solve (3.1).

**Neumann Conditions on a Type II Grid**

$$\frac{1}{\Delta x^2}\begin{bmatrix}-1 & 1 & 0 & 0 & \cdots & 0\\ 1 & -2 & 1 & 0 & \cdots & 0\\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots\\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots\\ 0 & \cdots & 0 & 1 & -2 & 1\\ 0 & \cdots & 0 & 0 & 1 & -1\end{bmatrix}\begin{bmatrix}\phi_{1/2}\\ \phi_{3/2}\\ \vdots\\ \vdots\\ \phi_{N-3/2}\\ \phi_{N-1/2}\end{bmatrix} = \begin{bmatrix}-\sin(x_{1/2}) + \frac{\alpha}{\Delta x}\\ -\sin(x_{3/2})\\ \vdots\\ \vdots\\ -\sin(x_{N-3/2})\\ -\sin(x_{N-1/2}) - \frac{\beta}{\Delta x}\end{bmatrix}. \tag{3.23}$$

**Periodic on a Type I Grid**

$$\frac{1}{\Delta x^2}\begin{bmatrix}-2 & 1 & 0 & 0 & \cdots & 0 & 1\\ 1 & -2 & 1 & 0 & \cdots & 0 & 0\\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots\\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots\\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots\\ 0 & 0 & \cdots & 0 & 1 & -2 & 1\\ 1 & 0 & \cdots & 0 & 0 & 1 & -2\end{bmatrix}\begin{bmatrix}\phi_0\\ \phi_1\\ \vdots\\ \vdots\\ \vdots\\ \phi_{N-2}\\ \phi_{N-1}\end{bmatrix} = \begin{bmatrix}-\sin(x_0)\\ -\sin(x_1)\\ \vdots\\ \vdots\\ \vdots\\ -\sin(x_{N-2})\\ -\sin(x_{N-1})\end{bmatrix}. \tag{3.24}$$

## 3.4 Basic Numerical Implementation

Below are definitions of the linear systems discussed thus far. We have left the implementation of boundary conditions as an exercise. Note that dense matrices in the most basic form are defined in MATLAB and python. Neither of these should be used and are here for educational purposes only.

### 3.4.1 MATLAB

```
% make grid
a = 0;                  % left endpoint
```

```
b = 2*pi;                % right endpoint
N = 20;                  % grid discretization count
dx = (b - a) / N;        % grid spacing
x = a:dx:b;              % grid points x_0 ... x_N  (row vector, length N+1)
x = x';                % make x a column vector

n = length(x);           % number of points (N+1)

% build basic Dxx WITH NO boundary conditions
P = -2 * ones(n, 1);      % main diagonal
E =      ones(n-1, 1);    % super-diagonal
W =      ones(n-1, 1);    % sub-diagonal

%%%%% Basic Matrix Type
%% DONT DO THIS - here for your reference
Dxx = diag(P) + diag(E, 1) + diag(W, -1);  % Places P on main diag, E on lower,
    and W on upper
Dxx = Dxx / dx^2;             % divide by dx^2

%%%%% OPTION 1: Sparse Matrix Type
% this is ideal for memory -- especially as N gets large
e = ones(n, 1);
Dxx = spdiags([e, -2*e, e], [-1, 0, 1], n, n) / dx^2;

% build right hand side
f = -sin(x);

%%%
%IMPLEMENT BC HERE ON Dxx AND f%
%%%

% solve for phi
phi = Dxx\f
```

### 3.4.2 python

```
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

a = 0.0                           # left endpoint
b = 2 * np.pi                     # right endpoint
N = 20                            # grid discretization count
dx = (b - a) / N                  # grid spacing
x = np.linspace(a, b, N + 1)    # grid points x_0 ... x_N

n = len(x)                        # number of points (N+1)

# build basic Dxx WITH NO boundary conditions
P = -2 * np.ones(n)               # main diagonal
E = np.ones(n - 1)                # super-diagonal
W = np.ones(n - 1)                # sub-diagonal

# build right hand side
f = -np.sin(x)
```

```
##### Basic Matrix Type
# this matrix is dense, and inefficient for large N
## DONT DO THIS - here for your reference

# P on main diag, E on super, W on sub
Dxx1 = np.diag(P) + np.diag(E, 1) + np.diag(W, -1)

# divide by dx^2
Dxx1 = Dxx1 / dx**2
# copying around f for each option
f1 = f.copy()

###
# IMPLEMENT BC HERE ON Dxx1 AND f1 #
###

# solve lienar system
phi1 = np.linalg.solve(Dxx1, f1)


##### OPTION 1: Dense, then converted to sparse (COO)
# easy, but you still allocate the full n x n dense matrix first

# P on main diag, E on super, W on sub
Dxx2 = np.diag(P) + np.diag(E, 1) + np.diag(W, -1)

# divide by dx^2
Dxx2 = Dxx2 / dx**2

# copying around f for each option
f2 = f.copy()

###
# IMPLEMENT BC HERE ON Dxx2 AND f2 while it is still dense #
###

# convert to sparse AFTER the BCs are set
# this type is difficult to manipulate when sparse
Dxx2 = sp.coo_matrix(Dxx2)
phi2 = spla.spsolve(Dxx2.tocsr(), f2)


##### OPTION 2: Build rows, cols, vals then make sparse (COO)
# nothing dense is ever allocated -- this is the one that scales well!
# this build can be thought of as saving (ROW, COL, VAL) for each nonzero point
# in the matrix. every undefined point is considered zero.

# build vector of row indicies
rows = np.concatenate([np.arange(1, n),  np.arange(n), np.arange(n - 1)])

# build vector of column indicies
cols = np.concatenate([np.arange(n - 1), np.arange(n), np.arange(1, n)])

# build vector of values
vals = np.concatenate([W,  P,   E]) / dx**2
f3 = f.copy()

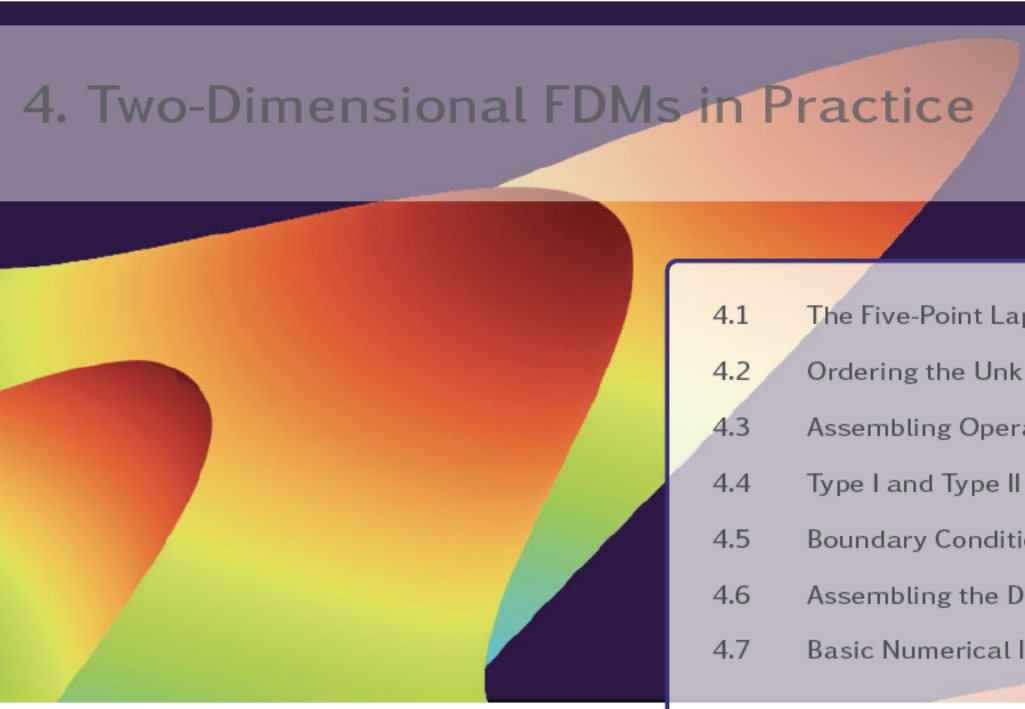
###
```

```
# IMPLEMENT BC HERE ON rows, cols, vals AND f3 #
# make sure you do this BEFORE converting to sparse
###

#convert to coo matrix
Dxx3 = sp.coo_matrix((vals, (rows, cols)), shape=(n, n))

#solve with spsolve
phi3 = spla.spsolve(Dxx3.tocsr(), f3)
```

# 4. Two-Dimensional FDMs in Practice




| | | |
|---|---|---|
| 4.1 | The Five-Point Laplacian | 38 |
| 4.2 | Ordering the Unknowns | 39 |
| 4.3 | Assembling Operators with Kronecker Products | 40 |
| 4.4 | Type I and Type II Grids in Two Dimensions | 41 |
| 4.5 | Boundary Conditions in Two Dimensions | 42 |
| 4.6 | Assembling the Discrete System | 45 |
| 4.7 | Basic Numerical Implementation | 45 |


Chapter 2 derived finite difference approximations of first and second derivatives in one dimension and used ghost points to impose boundary conditions on Type I and Type II grids. Almost everything we need for two dimensions is already in hand. This chapter reuses the same Taylor expansions, compass notation, and ghost-point boundary-condition system to implement finite difference methods on two-dimensional domains. We discuss how to arrange a two-dimensional grid of unknowns into a single vector, assemble the resulting matrix, and manage boundary conditions, especially at the corners.

We will consider the two-dimensional Poisson problem and use it to define Dirichlet, Neumann, and periodic boundary conditions on both Type I and II grids.

Let $\Omega = [a,b]\times[c,d] \subset \mathbb{R}^2$ be a two-dimensional domain. Given a function $f : \Omega \to \mathbb{R}$, we aim to find a function $\phi : \Omega \to \mathbb{R}$ satisfying

$$\nabla^2\phi = \frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = f(x,y), \qquad (x,y) \in \Omega, \tag{4.1}$$

and user-defined boundary condition on the domain's surface $\partial\Omega$. Equation (4.1) is the natural two-dimensional cousin of the model problem $D_{xx}\phi = -\sin(x)$ from Section 3 and the boundary condition is what singles out a unique solution, just as in one dimension.

## A verification case: manufactured solutions

Before solving this system for an unknown $\phi$, we would like a case whose exact answer we already know to measure the error of the numerical method. This approach is called the *method of manufactured solutions*: we choose a smooth $\phi$, substitute it into (4.1) to define the forcing function $f$, and read the boundary data off of $\phi$ directly. THis method allows us to find bugs in our code, verify error, and give confidence in the method. Throughout this chapter we will choose

$$\phi(x,y) = \sin(x)\sin(y) \qquad \text{on} \qquad \Omega = [0,2\pi]\times[0,2\pi]. \tag{4.2}$$

To obtain the forcing function $f$ that satisfies our chosen $\phi$, we analytically evaluate the left-hand side of (4.1) to obtain

$$f(x,y) = -2\sin(x)\sin(y). \tag{4.3}$$

The function $\phi$ provides information on how to test different boundary conditions. Homogeneous Dirichlet conditions (i.e., Dirichlet on all sides of the domain) can be conveniently tested, as $\sin(0) = \sin(2\pi) = 0$. The normal derivative of $\phi$, $\cos(x)\sin(y)$ or $\sin(x)\cos(y)$ depending on the edge, gives us ready-made Neumann conditions, and (4.2) is $2\pi$-periodic in both directions, so the same field also serves as a periodic test.

*Remark* 2. A manufactured solution tests your discretization by showing whether the scheme converges at the expected order.

## 4.1 The Five-Point Laplacian

We discretize $\Omega$ with a grid that is equally spaced in $x$ and $y$. Reconsidering the Type I grid (nodal), we define

$$x_i = a + i\,\Delta x, \quad i = 0,\dots,N_x, \qquad y_j = c + j\,\Delta y, \quad j = 0,\dots,N_y, \tag{4.4}$$

with $\Delta x = (b-a)/N_x$ and $\Delta y = (d-c)/N_y$. As in one dimension we write $\phi_{i,j} \equiv \phi(x_i, y_j)$ for the unknown grid values.

The Laplacian ($\nabla^2$) operation in (4.1) is a sum of two second derivatives of $\phi$, one with respect to $x$ and one with respect to $y$. These derivatives are by themselves the one-dimensional operator we already know how to approximate ($D_{xx}$). To extend this to 2D, we can simply apply the second-order central difference (2.10) in the $x$ direction (holding $j$ fixed) and again in the $y$ direction (holding $i$ fixed), such that

$$\nabla^2\phi\big|_{i,j} \approx \frac{\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}}{\Delta x^2} + \frac{\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}}{\Delta y^2}. \tag{4.5}$$

This is the *five-point Laplacian* stencil. It couples each point to its four nearest neighbors and carries a local truncation error of $\mathcal{O}(\Delta x^2) + \mathcal{O}(\Delta y^2)$, inherited directly from the 1D scheme.

Our compass notation extends naturally as well. For a point of interest $P$, keep the east ($E$, increasing $x$) and west ($W$, decreasing $x$) neighbors of Chapter 2, and add the *north* ($N$, increasing $y$) and *south* ($S$, decreasing $y$) neighbors. In this notation,

$$\nabla^2\phi\big|_{P} \approx \frac{\phi_E - 2\phi_P + \phi_W}{\Delta x^2} + \frac{\phi_N - 2\phi_P + \phi_S}{\Delta y^2}. \tag{4.6}$$

The five-point stencil in terms of compass notation is depicted in Figure 4.1. When $\Delta x = \Delta y = h$, (4.6) collapses to the form

$$\nabla^2\phi\big|_{P} \approx \frac{\phi_E + \phi_W + \phi_N + \phi_S - 4\phi_P}{h^2}. \tag{4.7}$$

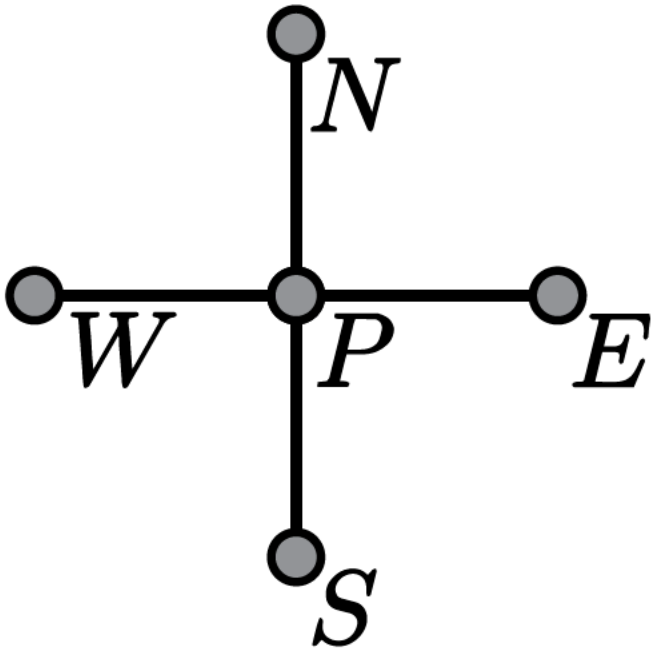



*Figure 4.1: The five-point stencil (4.6) at an interior point P, coupling P to its north (y), south (y), east (x), and west (x) neighbors.*

## 4.2 Ordering the Unknowns

A linear solver in the form $Ax = b$ wants a vector of unknowns $x$, not the matrix $\Phi \in \mathbb{R}^{(N_x+1)\times(N_y+1)}$ of grid values we would obtain from evaluating $\phi$ over $\Omega$. We therefore *flatten* $\Phi$ into a vector of size $\boldsymbol{\phi} \in \mathbb{R}^{M\times 1}$, where $M$ is the product of the number of unknowns in the $x$- and $y$-direction. The standard choice of flattening is row-wise, which sweeps a full row of $\Phi$ before moving to the next. Indexing this,

$$\boldsymbol{\phi}_k = \Phi_{i,j}, \text{ where } k(i,j) = i + j\,(N_x + 1) \tag{4.8}$$

so consecutive entries of $\boldsymbol{\phi}$ sweep a row of $\Phi$ (fixed $y_j$, increasing $x_i$), and after $N_x+1$ entries we advance to the next row (the next $y_j$). The total number of unknowns is $M = (N_x+1)(N_y+1)$ for a Type I grid and $M = (N_x)(N_y)$ for a Type II grid.

Using row-wise flattening, the five-point Laplacian has a *block tridiagonal* structure: tridiagonal blocks (the $x$ coupling to $E/W$) along the main block diagonal, and identity-like blocks (the $y$ coupling to $N/S$) one block above and below. Let's consider what this looks like discretely - let $\Delta x = \Delta y = h$ on a $3 \times 3$ Type I grid ($N_x = N_y = 2$). Ordering the nine

unknowns with (4.8) and applying the five-point stencil (4.7) at each node produces

$$\frac{1}{h^2}\begin{bmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{bmatrix}\begin{bmatrix} \phi_{0,0} \\ \phi_{0,1} \\ \phi_{0,2} \\ \phi_{1,0} \\ \phi_{1,1} \\ \phi_{1,2} \\ \phi_{2,0} \\ \phi_{2,1} \\ \phi_{2,2} \end{bmatrix} = \begin{bmatrix} -2\sin(x_0)\sin(y_0) \\ -2\sin(x_0)\sin(y_1) \\ -2\sin(x_0)\sin(y_2) \\ -2\sin(x_1)\sin(y_0) \\ -2\sin(x_1)\sin(y_1) \\ -2\sin(x_1)\sin(y_2) \\ -2\sin(x_2)\sin(y_0) \\ -2\sin(x_2)\sin(y_1) \\ -2\sin(x_2)\sin(y_2) \end{bmatrix}. \tag{4.9}$$

We have highlighted the five-point stencil application at $P = x_{1,1}$ in purple and the tridiagonal block in orange. Along each row, the $x$ coupling contributes the super- and subdiagonal $+1$ bands, but those bands break at the boundaries of the domain. For example, node $(0,2)$ and node $(1,0)$ are consecutive in the flattened ordering yet are not spatial neighbors, so the $x$-coupling between them is zero and the band structure breaks. The $y$ coupling appears as $\pm 1$ bands set $N_x + 1$ columns out from the diagonal. Grouping the unknowns row by row exposes this as a *block tridiagonal* matrix,

$$L = \frac{1}{h^2}\begin{bmatrix} T & I & 0 & \cdots & 0 \\ I & T & I & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & I & T & I \\ 0 & \cdots & 0 & I & T \end{bmatrix}, \qquad T = \begin{bmatrix} -4 & 1 & 0 & \cdots & 0 \\ 1 & -4 & 1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 1 & -4 & 1 \\ 0 & \cdots & 0 & 1 & -4 \end{bmatrix}, \tag{4.10}$$

where $I$ is the $(N_y+1)\times(N_y+1)$ identity and $T$ is the tridiagonal block highlighted in orange in (4.9), shown for the final block row $i = N_x = 2$. We will define this further in the next section. Note the boundary rows here still show the raw stencil; their treatment is deferred to Section 4.5.

## 4.3 Assembling Operators with Kronecker Products

The block structure just described is the signature of a *Kronecker product*, which gives us a straightfoward way to build the 2D operator directly from the 1D matrices of Chapter 2, without ever looping over individual stencils. Note however that both approaches are equivalent. We will discuss the Kronecker implementation and leave the stencil looping as an exercise.

**Definition 4.1 (Kronecker product)** For $A \in \mathbb{R}^{m\times m}$ and $B \in \mathbb{R}^{n\times n}$, the Kronecker product $A \otimes B \in \mathbb{R}^{mn\times mn}$ is the block matrix obtained by multiplying each entry $a_{ij}$ of $A$ by the whole matrix $B$, such that

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1m}B \\ a_{21}B & a_{22}B & \cdots & a_{2m}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mm}B \end{bmatrix}.$$

Let $D_x \in \mathbb{R}^{(N_x+1)\times(N_x+1)}$ and $D_y \in \mathbb{R}^{(N_y+1)\times(N_y+1)}$ be the 1D second-difference matrices in the $x$ and $y$ directions, i.e. the tridiagonal $[1, -2, 1]$ operators scaled by $1/\Delta x^2$ and $1/\Delta y^2$ respectively. Writing the grid values as a matrix $\Phi$ with entries $\Phi_{ij} = \phi_{i,j}$, the five-point Laplacian (4.5) is exactly

$$\nabla^2\phi \;\approx\; D_x\Phi + \Phi D_y^{\mathsf{T}}.$$

Flattening $\Phi$ into $\boldsymbol{\phi}$ row-wise as shown in (4.8), we can evaluate the Laplacian discretely through the matrix

$$L \;=\; I_y \otimes D_x \;+\; D_y \otimes I_x \tag{4.11}$$

where $I_x$ and $I_y$ are identity matrices of size $N_x+1$ and $N_y+1$, respectively, and $L$ follows the same structure shown in (4.9). The discrete Poisson problem is then simply

$$L\boldsymbol{\phi} = \boldsymbol{f}, \tag{4.12}$$

with $\boldsymbol{f}$ the forcing (4.3) flattened in the same order.

## 4.4 Type I and Type II Grids in Two Dimensions

The two grid layouts of Chapter 2 extend to two dimensions by taking the corresponding 1D layout in *each* direction.

Recall a **Type I** (nodal) grid is discretized as

$$x_i = a + i\,\Delta x, \quad i = 0,\ldots,N_x, \qquad y_j = c + j\,\Delta y, \quad j = 0,\ldots,N_y, \tag{4.13}$$

which places points on the boundary in both directions, as depicted in Figure 4.2. The four edges $x = a, b$ and $y = c, d$ are exterior points of the grid, highlighted in orange, and the four corners are themselves nodes of our grid. The ghost points shown in grey are the rings of nodes one spacing outside the domain, e.g. $x_{-1} = a - \Delta x$ along the west edge and $y_{-1} = c - \Delta y$ along the south edge. All interior points are shown in cyan.

Recall a **Type II** (cell-centered) grid instead discretizes $\Omega$ with $N_x \times N_y$ rectangular cells of size $\Delta x \times \Delta y$ and places the unknowns at the cell centers. Extending this idea to two-

dimension gives

$$x_{i+1/2} = a + \left(i + \tfrac{1}{2}\right)\Delta x, \qquad y_{j+1/2} = c + \left(j + \tfrac{1}{2}\right)\Delta y, \tag{4.14}$$

for $i = 0, \ldots, N_x - 1$ and $j = 0, \ldots, N_y - 1$. No unknown lies on the domain boundary and each edge is a line of cell faces. The ghost values live half a spacing outside the domain. Figure 4.2 depicts a Type II grid, with exterior points in orange, interior points in cyan, and ghost points in grey.

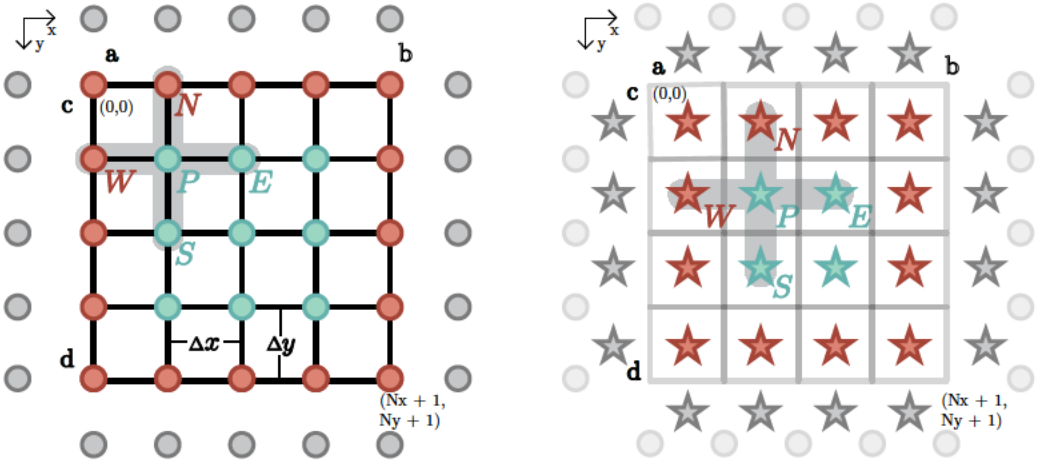



*Figure 4.2: On the left, a Type I grid for $\Omega$, with interior points highlighted in cyan, exterior points in orange, and ghost values in grey. On the right, a Type II grid for $\Omega$ with the same shading convention.*

## 4.5 Boundary Conditions in Two Dimensions

Everything from the 1D discussion carries over to 2D for the three boundary conditions of interest, which we will now impose on (4.1). In each case we treat the west edge $x = a$ in detail. The remaining edges follow by relabelling and are left as exercises. A new focus for 2D is defining boundary conditions on the corners where two edges meet, which we will treat separately in Section 4.5.4.

### 4.5.1 Dirichlet Conditions

Recall that a Dirichlet condition prescribes the *value* of $\phi$ on the boundary. Along the west edge, $\phi(a, y_j) \equiv \alpha_j$, such that $\phi$ evaluates as the constant $\alpha_j$ for all $j$ along the west edge ($x = a$).

**Type I.** The west-edge nodes $(0, j)$ lie exactly on the boundary, so the condition is simply $\phi_{0,j} = \alpha_j$. As in one dimension there are two equivalent implementations. We may overwrite the boundary row so that it reads $\phi_{0,j} = \alpha_j$ (a 1 on the diagonal, right-hand side $\alpha_j$), or we may treat $\phi_{0,j}$ as known and eliminate it for our left-hand side, moving its contributions to its neighbors to the right-hand side. For the interior node $(1, j)$ adjacent to the west edge, the

elimination gives (with $\Delta x = \Delta y = h$)

$$\nabla^2\phi\big|_{1,j} \approx \frac{\phi_{2,j} - 2\phi_{1,j}}{h^2} + \frac{\phi_{1,j+1} - 2\phi_{1,j} + \phi_{1,j-1}}{h^2} = f_{1,j} - \frac{\alpha_j}{h^2}. \tag{4.15}$$

If we were to impose Dirichlet conditions for all edges, consider what happens at the node $(1,1)$. (See Section 4.5.4.)

**Type II.** There is no node on the boundary, so the boundary condition is enforced through a midpoint formula. The west face at $x = a$ is the midpoint of the ghost point $x_{-1/2}$ and the exterior point $x_{1/2}$, so

$$\alpha_j = \frac{\phi_{-1/2,j} + \phi_{1/2,j}}{2} \quad \Longrightarrow \quad \phi_{-1/2,j} = 2\alpha_j - \phi_{1/2,j}. \tag{4.16}$$

Substituting (4.16) into the five-point stencil at $(1/2, j)$ turns the west coupling into

$$\frac{-3\,\phi_{1/2,j} + \phi_{3/2,j}}{\Delta x^2} + (y \text{ terms}) = f_{1/2,j} - \frac{2\alpha_j}{\Delta x^2}, \tag{4.17}$$

exactly the $-3$ boundary coefficient found in (3.16). To apply this along the whole west edge, we simply implement for every $j$.

### 4.5.2 Neumann Conditions

Recall a Neumann condition prescribes the outward normal derivative. On the west edge $\hat{n} = -\hat{x}$, so the condition $\partial\phi/\partial n\,(a, y_j) = \alpha_j$ implies that $\partial_x\phi(a, y_j) = -\alpha_j$.

> *Remark* 3. The normal direction for each boundary differs and should be considered when defining any boundary condition.

**Type I.** The boundary is the line of nodes $x_0 = a$, so we apply the central difference first derivative (2.7) in $x$:

$$\frac{\phi_{1,j} - \phi_{-1,j}}{2\Delta x} = -\alpha_j \quad \Longrightarrow \quad \phi_{-1,j} = \phi_{1,j} + 2\Delta x\,\alpha_j. \tag{4.18}$$

Substituting into the five-point stencil at $(0, j)$ and moving the constant to the right-hand side,

$$\frac{2\phi_{1,j} - 2\phi_{0,j}}{\Delta x^2} + (y \text{ terms}) = f_{0,j} - \frac{2\alpha_j}{\Delta x}. \tag{4.19}$$

Compare the $x$ part, $[-2,\ 2]/\Delta x^2$, with the 1D result (3.19). The discretizations for $x$ are identical, but in 2D this is now enforced across the entire west edge.

**Type II.** The derivative is specified at the west face, which lies exactly halfway between the ghost center $x_{-1/2}$ and the interior center $x_{1/2}$. As in Section 3.3.3, a one-sided difference across that face is second-order accurate at the domain boundary, such that

$$\frac{\phi_{1/2,j} - \phi_{-1/2,j}}{\Delta x} = -\alpha_j \quad \Longrightarrow \quad \phi_{-1/2,j} = \phi_{1/2,j} + \Delta x\, \alpha_j. \tag{4.20}$$

Substituting into the five-point stencil at $(1/2, j)$ gives the west-edge row

$$\frac{\phi_{3/2,j} - \phi_{1/2,j}}{\Delta x^2} + (y \text{ terms}) = f_{1/2,j} - \frac{\alpha_j}{\Delta x}, \tag{4.21}$$

matching the $[-1,\ 1\,]/\Delta x^2$ boundary row of (3.22). Again, this can be applied across the entire edge.

### 4.5.3 Periodic Conditions

Recall a periodic condition in $x$ states that $\phi$ and its $x$-derivatives at $x = a$ equal those at $x = b$. As in Section 3.3.1, this is implemented by wrapping the ghost value around to the opposite end of the domain and cutting off the last point of the domain.

**Type I.** The nodes $x_0 = a$ and $x_{N_x} = b$ represent the same physical location, so we carry the $N_x$ distinct unknowns $\phi_{0,j}, \ldots, \phi_{N_x-1,j}$ in each row and drop $\phi_{N_x,j} \equiv \phi_{0,j}$. Stepping west past the first node wraps to the last interior node,

$$\phi_{-1,j} \equiv \phi_{N_x-1,j}. \tag{4.22}$$

**Type II.** The cells wrap just the same as a Type I grid, such that the west ghost cell is a copy of the last interior cell in the row,

$$\phi_{-1/2,j} \equiv \phi_{N_x-1/2,j}. \tag{4.23}$$

### 4.5.4 Managing Corners

Consider the top-left corner on a Type I grid, node $(0, 0)$. Its five-point stencil calls for a west ghost $\phi_{-1,0}$ *and* a north ghost $\phi_{0,-1}$.

This highlights the beauty of the Kronecker build. The west/east conditions are enforced once in the boundary rows of $D_x$, and the term $D_x \otimes I_y$ places it into every $x$-block, including the block containing the corner. The south/north conditions are enforced in $D_y$, and the term $I_x \otimes D_y$ places it within every block, including the corner's. The corner node therefore receives both modifications automatically, with no special code.

To see that this produces the right coefficients, take homogeneous Neumann on the west and north edges with $\Delta x = \Delta y = h$. The 1D factors have boundary rows $[-2,\ 2\,]/h^2$ (from (4.19)), and (4.11) gives the corner row

$$\nabla^2\phi\big|_{0,0} \approx \frac{-4\phi_{0,0} + 2\phi_{1,0} + 2\phi_{0,1}}{h^2}, \tag{4.24}$$

i.e. diagonal $-4/h^2$ with weight $2/h^2$ to the east and to the north. It is a worthwhile check to rederive (4.24) by hand, substituting the two ghost relations directly into the stencil, and confirm that the Kronecker construction agrees. The right-hand side is where care is needed: at a corner the constants output by the respective difference methods from *both* edges land on the same equation.

## 4.6 Assembling the Discrete System

You are now equipped to assemble (4.12) for any combination of the three boundary conditions on either grid type. The recipe is:

1. Build the 1D factors $D_x$ and $D_y$ as in Chapter 2.
2. Modify the *boundary rows* of each factor to encode that direction's boundary condition (identity/$-3$ for Dirichlet, $[-2, 2]$ or $[-1, 1]$ for Neumann, circulant for periodic), matching the grid type.
3. Form $L$ from $D_x, D_y, I_y$, and $I_x$.
4. Flatten the forcing $f$ with the same ordering (4.8), and add the boundary-data contributions (including the doubled corner terms) to the appropriate entries of $\boldsymbol{f}$.
5. Solve $L\boldsymbol{\phi} = \boldsymbol{f}$.
6. Reshape $\boldsymbol{\phi}$ back to the grid.

## 4.7 Basic Numerical Implementation

Below we assemble the five-point Laplacian by Kronecker product in MATLAB and python. As in Chapter 2, *the matrices are shown without boundary conditions* — imposing them is left as an exercise. Note that both listings use the identical formula `kron(Ix, Dy) + kron(Dx, Iy);` only the array-flattening convention differs between the two languages, and each listing comments its own convention.

### 4.7.1 MATLAB

```
%% USER INPUTS
Nx = 8;                  % grid points per side (Type I grid, boundaries included
    )
L  = 1.0;                % domain is [0, L] x [0, L]
h  = L / (Nx - 1);       % grid spacing

% Dirichlet boundary values
BC = @(x, y) sin(pi*x) .* sinh(pi*y);

% unknowns are stacked row-wise: k = i + (j-1)*n, as in the text
% (MATLAB indices start at 1, so i = 1 is x = 0 and j = 1 is y = 0)

% 5-point stencil coefficients
P = -4 / h^2;            % this point
W =  1 / h^2;            % west neighbor
E =  1 / h^2;            % east neighbor
```

```
S =  1 / h^2;             % south neighbor
N =  1 / h^2;             % north neighbor


%% OPTION 1: every grid point is an unknown
% boundary rows just say phi = BC, interior rows get the 5-point stencil
rows = []; cols = []; vals = [];
f_full = zeros(Nx*Nx, 1);

for j = 1:Nx
    for i = 1:Nx
        k = i + (j - 1)*Nx;
        x = (i - 1)*h;
        y = (j - 1)*h;

        if i == 1 || i == Nx || j == 1 || j == Nx      % boundary point
            rows(end+1) = k; cols(end+1) = k; vals(end+1) = 1;
            f_full(k) = BC(x, y);
        else                                           % interior point
            rows(end+1) = k; cols(end+1) = k;      vals(end+1) = P;
            rows(end+1) = k; cols(end+1) = k - 1;  vals(end+1) = W;
            rows(end+1) = k; cols(end+1) = k + 1;  vals(end+1) = E;
            rows(end+1) = k; cols(end+1) = k - Nx; vals(end+1) = S;
            rows(end+1) = k; cols(end+1) = k + Nx; vals(end+1) = N;
        end
    end
end

A_full = sparse(rows, cols, vals, Nx*Nx, Nx*Nx);


%% OPTION 2: only interior points are unknowns
% boundary values are known, so they move to the right hand side:
%   (stencil on interior points) = f_P - alpha / h^2
% where alpha is the known value at a boundary neighbor (see Chapter 2)
n = Nx - 2;               % interior points per side
rows = []; cols = []; vals = [];
f_int = zeros(n*n, 1);    % f = 0 for Laplace's equation

for j = 2:Nx-1
    for i = 2:Nx-1
        k = (i - 1) + (j - 2)*n;
        x = (i - 1)*h;
        y = (j - 1)*h;

        rows(end+1) = k; cols(end+1) = k; vals(end+1) = P;

        if i == 2                      % W is on the boundary x = 0
            alpha = BC(0, y);
            f_int(k) = f_int(k) - alpha / h^2;
        else
            rows(end+1) = k; cols(end+1) = k - 1; vals(end+1) = W;
        end

        if i == Nx - 1                 % E is on the boundary x = L
            alpha = BC(L, y);
            f_int(k) = f_int(k) - alpha / h^2;
        else
            rows(end+1) = k; cols(end+1) = k + 1; vals(end+1) = E;
```

```
        end

        if j == 2                       % S is on the boundary y = 0
            alpha = BC(x, 0);
            f_int(k) = f_int(k) - alpha / h^2;
        else
            rows(end+1) = k; cols(end+1) = k - n; vals(end+1) = S;
        end

        if j == Nx - 1                  % N is on the boundary y = L
            alpha = BC(x, L);
            f_int(k) = f_int(k) - alpha / h^2;
        else
            rows(end+1) = k; cols(end+1) = k + n; vals(end+1) = N;
        end
    end
end

A_int = sparse(rows, cols, vals, n*n, n*n);


%% OPTION 3: same matrix as option 2, built with Kronecker products
% 1D second derivative on the interior points (boundaries removed)
e = ones(n, 1);
Dxx = spdiags([e, -2*e, e], [-1, 0, 1], n, n) / h^2;
Dyy = Dxx;                % same here since the grid is square
I = speye(n);

% unknowns are stacked row-wise, so x-derivatives act within each block of n rows
A_kron = kron(I, Dxx) + kron(Dyy, I);

%% SPARSITY PLOTS
figure('Position', [100, 100, 2100, 700]);

mats   = {A_full, A_int, A_kron};
titles = {'Full-grid matrix', 'Interior-only matrix', 'Kronecker matrix'};
colors = {[0.39, 0.58, 0.93], [1.0, 0.1, 0.45], [1.0, 0.55, 0.0]};

for m = 1:3
    ax = subplot(1, 3, m);
    spy(mats{m}, 'o', 6)
    set(get(ax, 'Children'), 'Color', colors{m})   % marker color
    set(ax, 'FontSize', 16)                         % tick labels (set first!)
    title(titles{m}, 'FontSize', 20)
    xlabel('column index', 'FontSize', 20)
    ylabel('row index', 'FontSize', 20)
end
```

### 4.7.2 python

```
import numpy as np
import scipy.sparse as sp
import matplotlib.pyplot as plt

## USER INPUTS
Nx = 8                      # grid points per side (Type I grid, boundaries included
    )
L = 1.0                     # domain is [0, L] x [0, L]
h = L / (Nx - 1)            # grid spacing
```

```
# Dirichlet boundary values
BC = lambda x, y: np.sin(np.pi * x) * np.sinh(np.pi * y)

# unknowns are stacked row-wise: k = i + j*n, as in the text

# 5-point stencil coefficients
P = -4 / h**2              # this point
W = 1 / h**2               # west neighbor
E = 1 / h**2               # east neighbor
S = 1 / h**2               # south neighbor
N = 1 / h**2               # north neighbor


## OPTION 1: every grid point is an unknown
# boundary rows just say phi = BC, interior rows get the 5-point stencil
rows, cols, vals = [], [], []
f_full = np.zeros(Nx * Nx)

for j in range(Nx):
    for i in range(Nx):
        k = i + j * Nx
        x = i * h
        y = j * h

        if i == 0 or i == Nx - 1 or j == 0 or j == Nx - 1:    # boundary point
            rows.append(k); cols.append(k); vals.append(1.0)
            f_full[k] = BC(x, y)
        else:                                                 # interior point
            rows.append(k); cols.append(k);       vals.append(P)
            rows.append(k); cols.append(k - 1);   vals.append(W)
            rows.append(k); cols.append(k + 1);   vals.append(E)
            rows.append(k); cols.append(k - Nx); vals.append(S)
            rows.append(k); cols.append(k + Nx); vals.append(N)

A_full = sp.coo_matrix((vals, (rows, cols)), shape=(Nx * Nx, Nx * Nx)).tocsr()


## OPTION 2: only interior points are unknowns
# boundary values are known, so they move to the right hand side:
#   (stencil on interior points) = f_P - alpha / h^2
# where alpha is the known value at a boundary neighbor (see Chapter 3)
n = Nx - 2                 # interior points per side
rows, cols, vals = [], [], []
f_int = np.zeros(n * n)    # f = 0 for Laplace's equation

for j in range(1, Nx - 1):
    for i in range(1, Nx - 1):
        k = (i - 1) + (j - 1) * n
        x = i * h
        y = j * h

        rows.append(k); cols.append(k); vals.append(P)

        if i == 1:                        # W is on the boundary x = 0
            alpha = BC(0, y)
            f_int[k] = f_int[k] - alpha / h**2
        else:
            rows.append(k); cols.append(k - 1); vals.append(W)
```

```
        if i == Nx - 2:                # E is on the boundary x = L
            alpha = BC(L, y)
            f_int[k] = f_int[k] - alpha / h**2
        else:
            rows.append(k); cols.append(k + 1); vals.append(E)

        if j == 1:                     # S is on the boundary y = 0
            alpha = BC(x, 0)
            f_int[k] = f_int[k] - alpha / h**2
        else:
            rows.append(k); cols.append(k - n); vals.append(S)

        if j == Nx - 2:                # N is on the boundary y = L
            alpha = BC(x, L)
            f_int[k] = f_int[k] - alpha / h**2
        else:
            rows.append(k); cols.append(k + n); vals.append(N)

A_int = sp.coo_matrix((vals, (rows, cols)), shape=(n * n, n * n)).tocsr()


## OPTION 3: same matrix as option 2, built with Kronecker products
# 1D second derivative on the interior points (boundaries removed)
e = np.ones(n)
Dxx = sp.diags([e[:-1], -2 * e, e[:-1]], [-1, 0, 1]) / h**2
Dyy = Dxx.copy()          # same here since the grid is square
I = sp.identity(n)

# unknowns are stacked row-wise, so x-derivatives act within each block of n rows
A_kron = (sp.kron(I, Dxx) + sp.kron(Dyy, I)).tocsr()


## SPARSITY PLOTS
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(21, 7))

ax1.spy(A_full, marker='o', markersize=6, color='cornflowerblue')
ax1.set_title('Full-grid matrix', fontsize=20)

ax2.spy(A_int, marker='o', markersize=6, color=[1.0, 0.1, 0.45])
ax2.set_title('Interior-only matrix', fontsize=20)

ax3.spy(A_kron, marker='o', markersize=6, color='darkorange')
ax3.set_title('Kronecker matrix', fontsize=20)

for ax in (ax1, ax2, ax3):
    ax.set_xlabel('column index', fontsize=20)
    ax.set_ylabel('row index', fontsize=20)
    ax.tick_params(labelsize=16)

fig.tight_layout()
plt.show()
```

*Remark* 4. As assembled above, the top and bottom rows of $D_x$ and $D_y$ are *invalid* stencils — they reference neighbors that do not exist — so $L$ is not yet solvable in a meaningful way. This is the two-dimensional echo of the "more unknowns than equations" situa-

tion from Section 3: the boundary rows are placeholders waiting for you to impose a boundary condition.

# 5. Staggered Grids




5.1 Coupled Unknowns on a Single Grid 51

5.2 Staggering in One Dimension 52

5.3 The Marker-and-Cell Grid 53

5.4 The Stokes Equations 56


In Chapter 2 every problem had a single unknown $\phi$ and we were free to choose the grid it lived on. Many PDEs of interest instead couple *several* unknowns together, such as the velocity and pressure of a fluid. This chapter introduces staggered grids, which place each unknown on its own grid. The choice matters: a placement of pressure on a grid that seems natural can produce a linear system with spurious, non-physical solutions! We will see that a staggered grid is nothing more than the Type I and Type II grids of Sections 3.1 and 3.2 used in combination, so the boundary condition tools from Chapter 2 and 3 carry over directly. We close the chapter by setting up a two-dimensional flow problem on a staggered grid.

## 5.1 Coupled Unknowns on a Single Grid

Consider a one-dimensional problem with unknowns velocity $u(x)$ and pressure $p(x)$ on the periodic domain $[a, b]$. Suppose we place *both* unknowns on the same Type I grid $\{x_i\}$ (collocated grid layout) and approximate the pressure gradient with the second-order central difference (2.7):

$$p'_P \approx \frac{p_E - p_W}{2\,\Delta x}. \tag{5.1}$$

Notice that $p_P$ itself never appears in (5.1)! The gradient at even-indexed points depends only on odd-indexed pressures, and vice versa. The pressure is effectively split across two independent sub-grids that never communicate—this is known as the checkerboard problem. Because the two sub-grids are decoupled, a constant pressure that alternates in sign from point to point has a discrete gradient of zero. Such a mode lies in the null space of the linear system so it can be added to any computed pressure without changing anything else. Therefore, computed pressures often show non-physical oscillations that resemble a checkerboard!

One could avoid the checkerboard issue by using one-sided differences, but as discussed in Chapter 2, these are only first-order accurate. We would like a difference that is *compact* (it uses neighboring values), centered, and second-order accurate.

## 5.2 Staggering in One Dimension

Recall from Section 3.2 that a difference across two neighboring points (i.e., $P$ and $E$, or $W$ and $P$) is naturally centered and second-order accurate at the point halfway between them. On a **staggered grid**, different unknowns live at different, offset locations, and derivative calculations take advantage of this naturally second-order difference between two neighbors. In one dimension we partition $[a, b]$ into $N$ cells of width $\Delta x$ and place

- the pressure $p$ at the cell centers $x_{i+1/2}$, labelled with stars (a Type II grid), and
- the velocity $u$ at the cell faces $x_i$, labelled with circles (a Type I grid).

This layout is shown in Figure 5.1. We extend the compass notation of Chapter 2: for a cell centered at $P$, the neighboring cell centers are still $W$ and $E$, while the faces of cell $P$ are denoted by lowercase letters $w$ and $e$. Using neighbors to compute derivatives produces the pressure derivative at the faces (where the velocity $u$ is defined) and the velocity derivative at the centers (where the pressure is defined), so

$$p'_e \approx \frac{p_E - p_P}{\Delta x}, \tag{5.2}$$

$$u'_p \approx \frac{u_e - u_w}{\Delta x}. \tag{5.3}$$

Each of these is a difference between immediate neighbors, centered exactly on the point where it is needed. Because every pressure value now appears in the derivative at the faces on either side of it, the sub-grids are coupled and the checkerboard problem is removed.

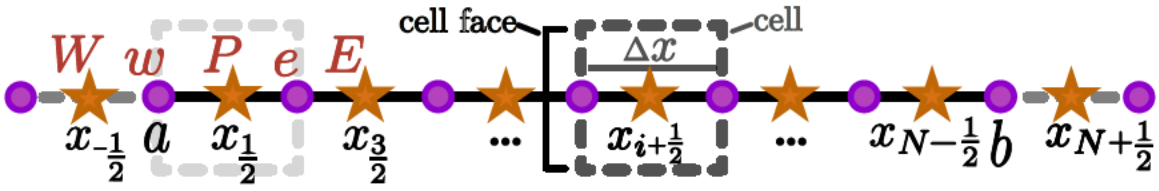



*Figure 5.1: A one-dimensional staggered grid. Pressure p lives at cell centers (Type II, stars) and velocity u lives at cell faces (Type I, circles). Cell centers are labelled with uppercase compass letters and faces with lowercase letters.*

On a domain with $N$ cells there are $N$ cell centers (where pressure lives) and $N + 1$ faces (where $u$ lives). On a periodic domain the first and last faces are the same physical point, leaving $N$ distinct faces. Writing (5.2) for every face defines a gradient operator $G$ that maps cell values to face values, and writing (5.3) for every cell defines a divergence operator $D$ that maps face values to cell values.

**Exercise 5.1** For a periodic domain with $N$ cells:

(a) Write out $G$ and $D$ for $N = 5$. Be careful about which face is indexed alongside which cell.

(b) Show that $D = -G^T$. What property of the continuous operators does this mimic? *Hint: integrate $u\,p'$ by parts on a periodic domain.*

*Remark* 5. Staggering is not free. Since $u$ and $p$ no longer share locations, any quantity that combines them, or any plot of them together, requires **interpolation** between grids, which is discussed further in Section 5.4.3.

## 5.3 The Marker-and-Cell Grid

The two-dimensional staggered grid most commonly used is the **Marker-and-Cell** (MAC) grid. Consider the domain $[a,b]\times[c,d]$ partitioned into $N_x \times N_y$ rectangular cells with

$$\Delta x = \frac{b-a}{N_x}, \qquad \Delta y = \frac{d-c}{N_y}, \qquad x_i = a + i\,\Delta x, \qquad y_j = c + j\,\Delta y.$$

The velocity $\mathbf{u} = [u(x,y), v(x,y)]$ has two components and each is given its own grid, where

- the horizontal component $u_{i,\,j+1/2}$ lives on the east and west faces $(x_i,\, y_{j+1/2})$ (squares), and
- the vertical component $v_{i+1/2,\,j}$ lives on the north and south faces $(x_{i+1/2},\, y_j)$ (triangles).

We also define the pressure $p(x,y)$, which lives at the cell centers $(x_{i+1/2},\, y_{j+1/2})$ (circles). The compass notation extends to two dimensions with $N$ and $S$ for the neighboring cell centers, and $n$ and $s$ for the faces of cell $P$, where east is the direction of increasing $x$ and north is the direction of decreasing $y$. Table 5.1 summarizes the placement in terms of the grid types of Chapter 2, and Figure 5.2 depicts the grid with a stencil for one cell.

| *Unknown* | *Location* | *Grid in x* | *Grid in y* |
|---|---|---|---|
| $p$ | cell centers | Type II | Type II |
| $u$ | east/west faces | Type I | Type II |
| $v$ | north/south faces | Type II | Type I |

*Table 5.1: Placement of unknowns on the MAC grid, described by the grid type each unknown sees in each coordinate direction.*

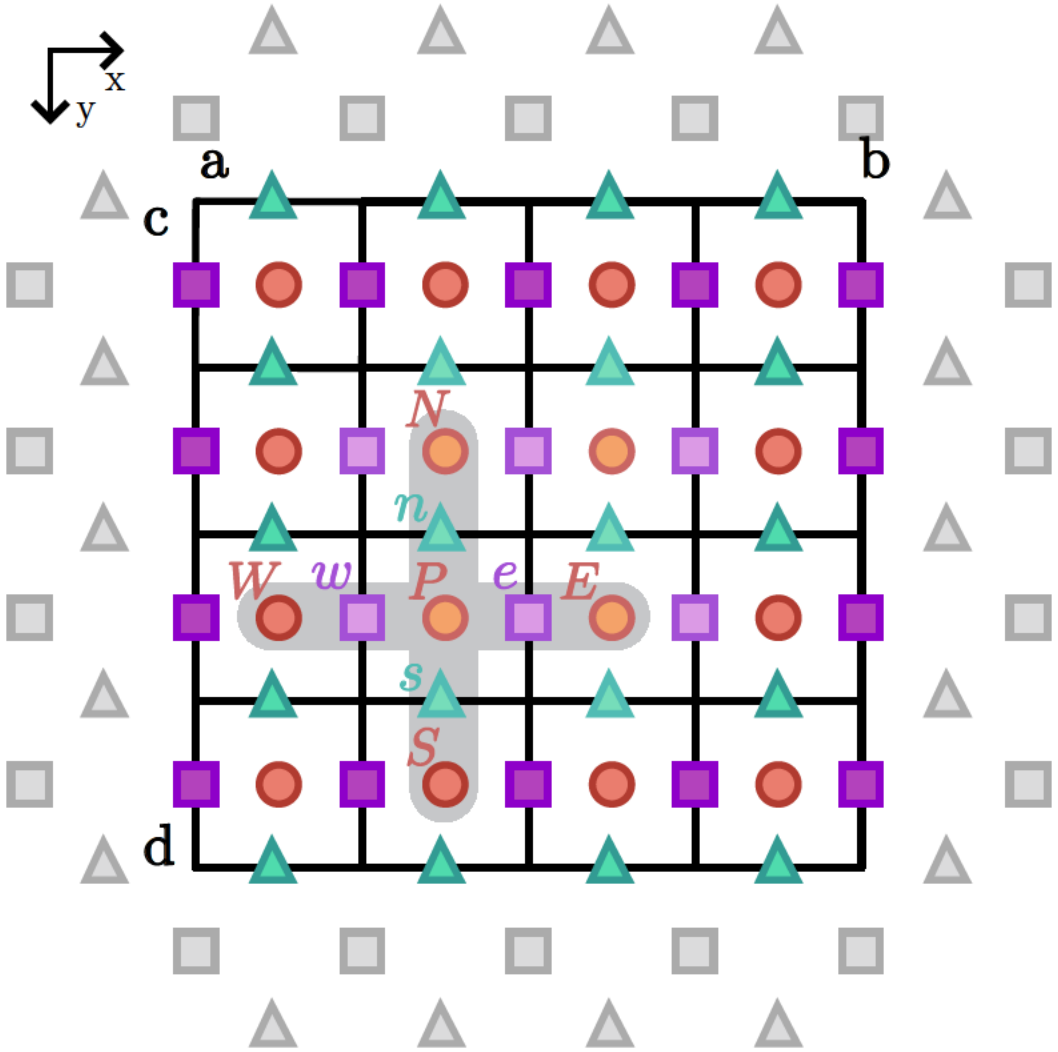



*Figure 5.2: Depiction of MAC staggered grid in two-dimensions for $N_x = N_y = 4$. Pressure lives at the cell center (circles), horizontal velocity $u$ on the east and west faces (squares), and vertical velocity $v$ on the north and south faces (triangles). Ghost cells are shown for the velocity components only.*

Note that each component lives on the faces it carries fluid *through*: $u$ measures flow across east and west faces, $v$ across north and south faces. The divergence evaluated at the cell center,

$$(\nabla \cdot \mathbf{u})_P \approx \frac{u_e - u_w}{\Delta x} + \frac{v_s - v_n}{\Delta y}, \tag{5.4}$$

is then the net flow out of cell $P$ per unit area. The discrete statement $\nabla \cdot \mathbf{u} = 0$ becomes an exact balance of what flows into and out of each cell, which is a strong physical reason to prefer this layout.

**Exercise 5.2** Write the MAC approximations of $\partial p/\partial x$ at the location of $u_e$ and of $\partial p/\partial y$ at the location of $v_n$ in compass notation.

Because each unknown lives on its own grid, each has its own set of ghost points, and boundary conditions must be imposed separately for each grid, which we discuss in the following sections.

### 5.3.1 Ordering Unknowns

In two dimensions the unknowns form a 2D array, but a linear system needs a single vector. We must choose an ordering that maps each pair $(i, j)$ to a single index $k$. In this text we will use row-wise stacking with the $x$-index varying fastest:

$$k = i + j\, n_x, \qquad i = 0, \ldots, n_x - 1, \qquad j = 0, \ldots, n_y - 1, \tag{5.5}$$

where $n_x$ and $n_y$ are the number of unknowns of that variable in each direction and $j = 0$ is the row nearest $y = c$. Note that $n_x$ and $n_y$ depend on the boundary conditions, as discussed in Section 5.4. We end with a vector of the form

$$\begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{p} \end{bmatrix}^T, \tag{5.6}$$

where $u(x, y)$, $v(x, y)$, and $p(x, y)$ have been flattened to the vectors $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{p}$, respectively.

### 5.3.2 Building Two-Dimensional Operators

Recall that two-dimensional operators can be assembled either entry-by-entry or with the Kronecker product—both of these methods were discussed in Section 3. Whichever method you choose, *look at your matrices*! A sparsity plot (`plt.spy` in python, `spy` in MATLAB) shows the nonzero entries of a matrix and reveals misplaced periodic entries, missing diagonals, and wrong block sizes. It is a great debugging tool!

## 5.4 The Stokes Equations

When a fluid moves slowly enough, or is viscous enough, its inertia is negligible and its motion is governed by the **Stokes** equations:

$$\mu \nabla^2 \mathbf{u} - \nabla p = \mathbf{0}, \tag{5.7}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{5.8}$$

On a domain $(x, y) \in \Omega = [0,1] \times [0,1]$, $\mathbf{u} = [u(x,y), v(x,y)]$ is the fluid velocity, $p(x,y)$ is the pressure, and $\mu > 0$ is the viscosity. Equation (5.7) is a balance between viscous forces and pressure forces, and (5.8) states that the fluid is incompressible. The Stokes equations can model the swimming of bacteria and other microorganisms, the settling of fine particles, and the slow flow of very viscous fluids such as magma, glass, and polymer melts. Written out in components, the equations are

$$\mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) - \frac{\partial p}{\partial x} = 0, \tag{5.9}$$

$$\mu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) - \frac{\partial p}{\partial y} = 0, \tag{5.10}$$

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{5.11}$$

Notice that (5.7)–(5.8) contain no equation for the pressure on its own. The pressure appears only through $\nabla p$ and takes whatever values are needed to make the velocity divergence-free, which is the motivation for a MAC grid. As a consequence, when the velocity is prescribed or periodic on every boundary, the pressure is only determined up to a constant: if $(u, v, p)$ is a solution, then so is $(u, v, p + C)$ for any constant $C \in \mathbb{R}$. The discrete system inherits this property, and its null space consists of $\mathbf{u} = \mathbf{v} = \mathbf{0}$ with every entry of $\mathbf{p}$ equal to the same constant. The matrix is therefore singular, and we must decide how to pick out a single pressure. There are a few choices, and none of them is more correct than the others. We can fix the pressure in a single cell by replacing one equation of the system to make the matrix nonsingular. Alternatively, we can solve the singular system as-is and subtract the mean of $\mathbf{p}$ during post-processing. A sparse direct solver will usually return a solution in this case, with the correct velocity and a pressure shifted by an arbitrary constant, which the mean subtraction removes. Be aware that this only works when a solution exists and that the solver may give no warning when it is not!

### 5.4.1 Where Each Equation Lives

On the MAC grid, each equation is enforced at the location of the unknown it is "responsible" for: the $x$-momentum equation (5.9) at every $u$ point, the $y$-momentum equation (5.10) at every $v$ point, and continuity (5.11) at every $p$ point. This gives one equation per unknown. Table 5.2 lists the number of grid locations for each variable. The boundary conditions then determine how many of these locations are unknowns: a location whose value is prescribed by a boundary condition, or that coincides with another location under periodicity, is removed from the system, as in Section 3.3.

| *Unknown* | *Grid locations* | *Equation enforced there* |
|---|---|---|
| $u$ | $(N_x+1)N_y$ | $x$-momentum (5.9) |
| $v$ | $N_x(N_y+1)$ | $y$-momentum (5.10) |
| $p$ | $N_xN_y$ | continuity (5.11) |

*Table 5.2: Number of grid locations for each unknown on an $N_x \times N_y$ MAC grid, before boundary conditions are applied.*

Consider for a moment what grid the derivatives of our equations lie on. The derivatives of (5.9) land on the $u$ grid, (5.10) on the $v$ grid, and (5.11) on the $p$ grid. This allows us to define a larger operator to apply to the flattened vector (5.6). This discrete operator takes the form

$$\begin{bmatrix} \mu L_u & 0 & -G_x \\ 0 & \mu L_v & -G_y \\ D_x & D_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{b}_u \\ \mathbf{b}_v \\ \mathbf{b}_p \end{bmatrix}, \tag{5.12}$$

where $L_u$ and $L_v$ are the discrete Laplacians on the $u$ and $v$ grids, $G_x$ and $G_y$ are discrete pressure gradients, $D_x$ and $D_y$ are the two parts of the discrete divergence, and the right-hand side vectors $\mathbf{b}_u$, $\mathbf{b}_v$, and $\mathbf{b}_p$ are where the boundary conditions are implemented.

**Exercise 5.3** Determine the dimensions of every block in (5.12), including the zero blocks, and the dimension of the full system, when $u$ and $v$ satisfy Dirichlet boundary conditions in the $y$-direction and all unknowns are periodic in the $x$-direction.

### 5.4.2 Building Each Operator

Each block of (5.12) is a finite difference operator from Chapter 2, applied on the grid of one unknown. Use Table 5.1 to identify the grid type each unknown sees in each direction, and then use the matching boundary treatment from Section 3.3. It is easiest to build each operator separately and, as you go, collect the contributions of known boundary values into that operator's own piece of the right-hand side. For example, $\mathbf{b}_u$ is the sum of the boundary contributions from $\mu L_u$ and $-G_x$. $\mathbf{b}_p$ is the sum of the contributions from $D_x$ and $D_y$.

Note that the right-hand side $\mathbf{b}_p$ can contain nonzero entries due to the implementation of boundary conditions, even though $\nabla \cdot \mathbf{u} = 0$.

### 5.4.3 Post-Processing

The velocity components live on different faces, so to plot the flow you must first bring $u$ and $v$ to a common location. The cell centers are a natural choice: average the two face values on either side of each center. Some of the faces you need may be boundary faces, where the value is not an unknown but is known from the boundary conditions.

### 5.4.4 Numerical Implementation in python

```
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt

## USER INPUTS
x_start, x_end = 0.0, 1.0       # domain in x (periodic)
y_start, y_end = 0.0, 1.0       # domain in y (walls)
mu    = 1.0                     # viscosity
Nx    = 50                      # number of cells in x
Ny    = Nx                      # number of cells in y

#any other constants go here :)

dx = (x_end - x_start) / Nx     # grid spacing in x
dy = (y_end - y_start) / Ny     # grid spacing in y

## GRIDS
# unknowns are ordered k = i + j*Nx (x varies fastest)

# p at cell centers
x_P = x_start + (np.arange(Nx) + 0.5) * dx
y_P = y_start + (np.arange(Ny) + 0.5) * dy

# u on east/west faces (x = x_end is the same face as x = x_start)
x_U = x_start + np.arange(Nx) * dx
y_U = y_P

# v on north/south faces (wall values are known, so they are removed)
x_V = x_P
y_V = y_start + np.arange(1, Ny) * dy

# Note the number of unknowns depends on boundary conditions!
nU = #####                      # number of u unknowns
nV = #####                       # number of v unknowns
nP = #####                      # number of p unknowns

###
# BUILD THE OPERATORS HERE AS SPARSE MATRICES
#   Lu : (nU x nU)   Laplacian of u
#   Lv : (nV x nV)   Laplacian of v
#   Gx : (nU x nP)   x-derivative of p at the u points
#   Gy : (nV x nP)   y-derivative of p at the v points
#   Dx : (nP x nU)   x-derivative of u at the p points
#   Dy : (nP x nV)   y-derivative of v at the p points
###

###
# BUILD THE RIGHT HAND SIDE HERE
###
bu = np.zeros(nU)
bv = np.zeros(nV)
bp = np.zeros(nP)

# block system (None is a zero block)
A = sp.bmat([[mu*Lu, None,  -Gx ],
             [None,  mu*Lv, -Gy ],
```

```
            [Dx,    Dy,    None]], format='csr')
b = np.concatenate([bu, bv, bp])


# solve
sol = spla.spsolve(A, b)

U = sol[:nU].reshape(Ny, Nx)                  # u on its grid
V = sol[nU:nU + nV].reshape(Ny - 1, Nx)       # v on its grid
P = sol[nU + nV:nU + nV + nP].reshape(Ny, Nx) # p on its grid
P = P - np.mean(P)                            # zero-mean pressure

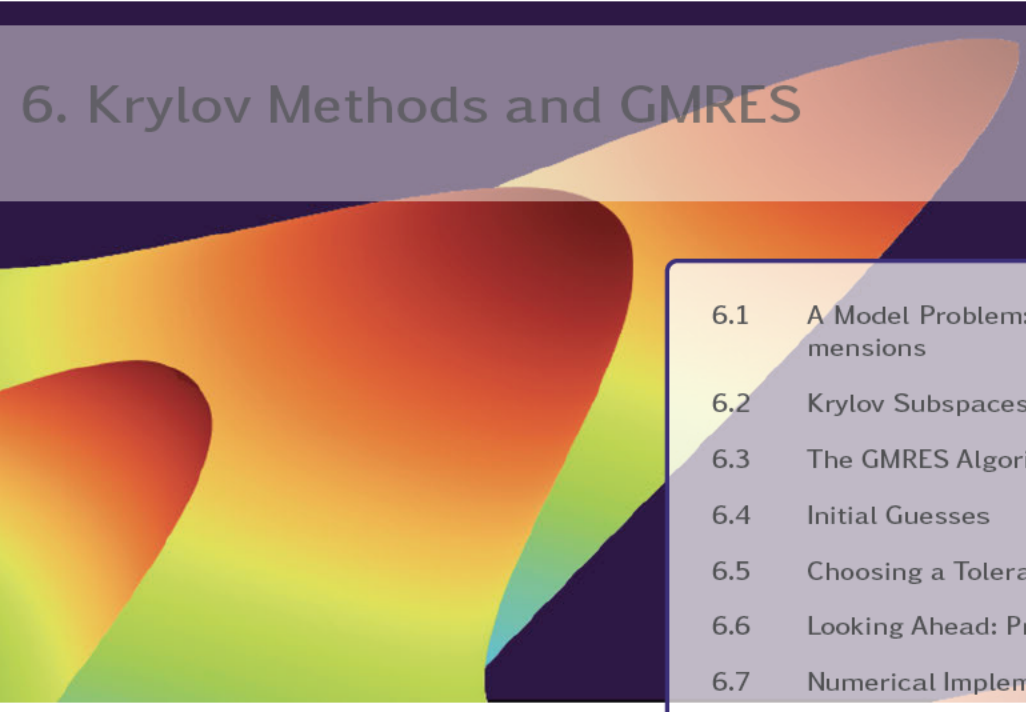
###
# INTERPOLATE U AND V TO THE CELL CENTERS HERE
###

## PLOTS (see Chapter 1 for how to make these better!)
plt.figure()
plt.pcolormesh(x_P, y_P, np.sqrt(Uc**2 + Vc**2), shading='gouraud')
plt.colorbar(label='speed')
plt.xlabel('x')
plt.ylabel('y')

plt.figure()
plt.pcolormesh(x_P, y_P, P, shading='gouraud')
plt.colorbar(label='pressure')
plt.xlabel('x')
plt.ylabel('y')

plt.show()
```

# 6. Krylov Methods and GMRES




6.1 A Model Problem: Poisson's Equation in Three Dimensions 60
6.2 Krylov Subspaces 62
6.3 The GMRES Algorithm 63
6.4 Initial Guesses 64
6.5 Choosing a Tolerance 65
6.6 Looking Ahead: Preconditioning 65
6.7 Numerical Implementation 66


In Chapter 2, every problem ended with a linear system of the form $A\mathbf{u} = \mathbf{f}$, which we solved with a single call to `A\f` or `spsolve`. In one dimension this is perfectly reasonable: the matrix is tridiagonal and a direct solver handles it in $\mathcal{O}(N)$ work. In three dimensions, a modest grid of $N = 128$ points per direction carries $N^3 \approx 2 \times 10^6$ unknowns making direct linear solvers unreasonable. This chapter introduces a different strategy: rather than solve the system exactly, we build an increasingly good approximation to the solution from a small set of carefully chosen vectors. The method we focus on, the *Generalized Minimal RESidual* method (GMRES), has three features:

1. it never needs the matrix itself, only a way to *apply* the matrix to a vector;
2. it is iterative, so it needs an initial guess;
3. it is approximate, so it needs a stopping tolerance.

## 6.1 A Model Problem: Poisson's Equation in Three Dimensions

Consider Poisson's equation on the cube $\Omega = [0, 2\pi]^3$ with homogeneous Dirichlet boundary conditions,

$$\nabla^2 u = f, \qquad u\Big|_{\partial\Omega} = 0, \tag{6.1}$$

where $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$. To check our numerical solutions we choose the manufactured solution

$$u(x, y, z) = e^{\sin x \sin y \sin z} - 1, \tag{6.2}$$

which vanishes on every face of the cube, and compute $f = \nabla^2 u$ by hand.

**Exercise 6.1** Verify that (6.2) satisfies the boundary condition in (6.1) and compute $f = \nabla^2 u$.

### 6.1.1 Discretization

We use the Type I grid of Section 3.1 in each direction and treat the Dirichlet values as known, removing them from the system exactly as in (3.14). Because the boundary values are zero, no correction to the right-hand side is needed. To keep the code compact we now let $N$ denote the number of *interior* points in each direction, so

$$x_i = i\,\Delta x, \qquad i = 1,\dots,N, \qquad \Delta x = \frac{2\pi}{N+1}, \tag{6.3}$$

and likewise for $y_j$ and $z_k$. We write $u_{i,j,k} \approx u(x_i, y_j, z_k)$.

Applying the second-order central difference (2.10) in each direction gives the *seven-point Laplacian*. We extend our compass notation to three dimensions: about the point $P = (i,j,k)$ the neighbors are east/west $(E,W)$ in $x$, north/south $(N,S)$ in $y$, and top/bottom $(T,B)$ in $z$. Then

$$\nabla^2 u\big|_P \approx \frac{u_E + u_W + u_N + u_S + u_T + u_B - 6\,u_P}{\Delta x^2}. \tag{6.4}$$

The notation $u_E$ denotes $u$ as evaluated at $u(i+1,j,k)$ and $u_T = u(i,j,k+1)$.

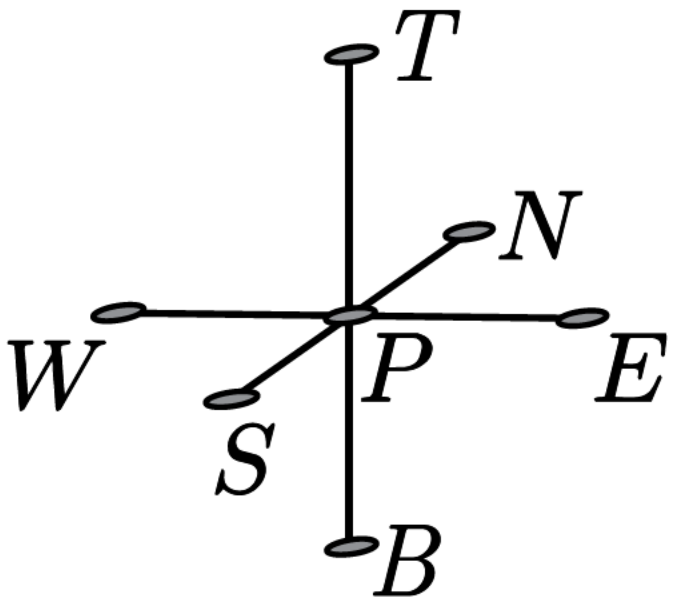



*Figure 6.1: The seven-point stencil for the three-dimensional Laplacian. The center point $P$ should be highlighted, with its six neighbors $E, W$ (along $x$), $N, S$ (along $y$), and $T, B$ (along $z$) labelled in the compass notation and connected to $P$.*

**Exercise 6.2** What is the local truncation error of (6.4)? Use the one-dimensional result of Section 2.3 rather than redoing a three-dimensional Taylor expansion. What order of convergence should we expect for the solution $u$?

### 6.1.2 Assembling the matrix

Stacking the $N^3$ unknowns row-wise into a single vector $\mathbf{u}$ turns (6.4) into a linear system $L\mathbf{u} = \mathbf{f}$. The matrix $L$ can be built from the one-dimensional matrix $D_2 \in \mathbb{R}^{N\times N}$ using Kronecker products, as we did for 2D, such that

$$L = I \otimes I \otimes D_2 \; + \; I \otimes D_2 \otimes I \; + \; D_2 \otimes I \otimes I, \tag{6.5}$$

where $I$ is the $N \times N$ identity. Recall that for $A \in \mathbb{R}^{m\times n}$ and $B$ of any size,

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix}.$$

The matrix $L$ is sparse, but its bandwidth is $N^2$. A direct solver such as Gaussian elimination fills in much of that band as it factors $L$, so both the memory and the work grow far faster than the $\mathcal{O}(N^3)$ nonzeros of $L$ itself.

> *Remark* 6. For large problems in two and especially three dimensions, *how* we solve $L\mathbf{u} = \mathbf{f}$ matters as much as how we discretized the PDE. A good discretization paired with a solver that cannot finish is not useful.

## 6.2 Krylov Subspaces

Let $A \in \mathbb{R}^{n\times n}$ be nonsingular and consider $A\mathbf{x} = \mathbf{b}$. The goal of Krylov methods is to build a vector space $\mathcal{K}_k$ using $\mathbf{b}$ and repeated multiplication by $A$ such that $\mathbf{x} \in \mathcal{K}_k$.

**Definition 6.1 (Krylov subspace)** The $k$-th Krylov subspace generated by $A$ and $\mathbf{b}$ is

$$\mathcal{K}_k(A, \mathbf{b}) = \operatorname{span}\left\{\mathbf{b}, A\mathbf{b}, A^2\mathbf{b}, \ldots, A^{k-1}\mathbf{b}\right\}.$$

Every vector in $\mathcal{K}_k(A, \mathbf{b})$ has the form $p(A)\mathbf{b}$ for some polynomial $p$ of degree at most $k-1$. $A^{-1}$ can be written in the form $p(A)$ for some polynomial $p$ of degree at most $n-1$, as $A$ satisfies its own characteristic polynomial following from the Cayley–Hamilton theorem. Therefore $\mathbf{x} = A^{-1}\mathbf{b} \in \mathcal{K}_n(A, \mathbf{b})$. The hope of every Krylov method is that $\mathcal{K}_k$ contains a good approximation of $A^{-1}\mathbf{b}$ for $k \ll n$.

### 6.2.1 Only the action of $A$ is needed

Consider the basis $\{\mathbf{b}, A\mathbf{b}, A^2\mathbf{b}, \ldots\}$. To build it we need exactly two things:

1. the vector $\mathbf{b}$, and
2. a procedure that takes a vector $\mathbf{v}$ and returns $A\mathbf{v}$.

We never need to look at an individual entry $a_{ij}$ or store $A$. Any function that computes $\mathbf{v} \mapsto A\mathbf{v}$ will do. Operators that exploit this are called *matrix-free* methods. Utilizing these matrix-free methods allow us to recover $L\mathbf{u}$ without ever building or operating with $L$. Compare this with getting the Kronecker structure of (6.5) right or with assembling a matrix for a more complicated operator. The function `Apply_Lap` in Section 6.7 is a matrix-free implementation of $L$.

**Exercise 6.3** In `Apply_Lap`, the neighbor values `u_E`, `u_W`, ... are initialized to zero and only overwritten when the neighbor is inside the grid. Explain why this implements the boundary condition in (6.1). How would you modify the function if the Dirichlet value on the face $x = 0$ were a nonzero constant $\alpha$? (Hint: think about what is linear and what is not, and look back at (3.14).) How would you implement a Neumann condition on that face using the ghost points of Section 3.3.3?

### 6.2.2 Building a good basis: the Arnoldi process

In floating point arithmetic the vectors $A^k\mathbf{b}$ are a poor basis as they share common eigenvectors. Instead we build an *orthonormal* basis $\mathbf{q}_1, \dots, \mathbf{q}_k$ for the same space one vector at a time by multiplying the newest basis vector by $A$, then orthogonalizing the result against all previous vectors with Gram–Schmidt. This is called the Arnoldi process and laid out in Algorithm 1.

**Algorithm 1** Arnoldi process (modified Gram–Schmidt)

$\mathbf{q}_1 \leftarrow \mathbf{r}_0 / \|\mathbf{r}_0\|_2$
**for** $j = 1, 2, \dots, k$ **do**
  $\mathbf{w} \leftarrow A\,\mathbf{q}_j$ ▷ the only use of $A$
  **for** $i = 1, \dots, j$ **do**
    $h_{ij} \leftarrow \mathbf{q}_i^T \mathbf{w}$
    $\mathbf{w} \leftarrow \mathbf{w} - h_{ij}\,\mathbf{q}_i$
  **end for**
  $h_{j+1,j} \leftarrow \|\mathbf{w}\|_2$
  $\mathbf{q}_{j+1} \leftarrow \mathbf{w} / h_{j+1,j}$
**end for**

Collecting the basis vectors into $Q_k = [\mathbf{q}_1 \cdots \mathbf{q}_k]$ and the coefficients $h_{ij}$ into the $(k+1) \times k$ upper Hessenberg matrix $\widetilde{H}_k$, the Arnoldi process produces the relation

$$A\,Q_k = Q_{k+1}\,\widetilde{H}_k. \tag{6.6}$$

## 6.3 The GMRES Algorithm

Given an initial guess $\mathbf{x}_0$ with residual $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$, GMRES chooses the $k$-th iterate as the vector in the shifted Krylov space that makes the residual as small as possible, such that

$$\mathbf{x}_k = \operatorname*{arg\,min}_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_k(A, \mathbf{r}_0)} \|\mathbf{b} - A\mathbf{x}\|_2. \tag{6.7}$$

Writing $\mathbf{x} = \mathbf{x}_0 + Q_k \mathbf{y}$ with $\mathbf{y} \in \mathbb{R}^k$ and using (6.6), the large minimization problem collapses to a small one:

$$\mathbf{y}_k = \arg\min_{\mathbf{y}\in\mathbb{R}^k} \left\| \beta\, \mathbf{e}_1 - \widetilde{H}_k \mathbf{y} \right\|_2, \qquad \beta = \|\mathbf{r}_0\|_2, \tag{6.8}$$

which is a $(k+1)\times k$ least-squares problem.

A few properties follow directly from (6.7):

- Since $\mathcal{K}_k \subseteq \mathcal{K}_{k+1}$, the residual norm $\|\mathbf{r}_k\|_2$ never increases from one iteration to the next.
- In exact arithmetic GMRES finds the exact solution in at most $n$ steps. This is useless as a practical bound ($n = N^3$) – what matters is how fast the residual drops in the first few iterations.
- The residual can be written as $\mathbf{r}_k = p_k(A)\mathbf{r}_0$, where $p_k$ is the polynomial of degree $k$ with $p_k(0) = 1$ that minimizes $\|p_k(A)\mathbf{r}_0\|_2$.

### 6.3.1 Cost and restarting

Each GMRES iteration costs one application of $A$ plus the orthogonalization of the new vector against *all* previous basis vectors, noting that every basis vector must be stored. Both the work and the memory therefore grow with the iteration count $k$. The common remedy is *restarted* GMRES, written GMRES($m$): run $m$ iterations, form $\mathbf{x}_m$, and start over using $\mathbf{x}_m$ as the new initial guess. Restarting caps the memory at $m+1$ basis vectors, but it throws away information and can slow convergence or stall it entirely. In MATLAB the third argument of `gmres` is the restart length $m$.

**Exercise 6.4** For $N = 128$, how many bytes does one basis vector of length $N^3$ occupy in double precision? How much memory could the Krylov basis occupy with a restart length of 500? Compare with the memory used by the sparse matrix $L$ (count its nonzeros).

## 6.4 Initial Guesses

Because GMRES is iterative, it must start somewhere. The default in both MATLAB and SciPy is $\mathbf{x}_0 = \mathbf{0}$, in which case $\mathbf{r}_0 = \mathbf{b}$ and the search space is exactly $\mathcal{K}_k(A, \mathbf{b})$. However, a good initial guess can save many iterations, as GMRES only has to find the *correction* $\mathbf{x} - \mathbf{x}_0$. Some sources of good guesses include...

- **Time stepping.** When an implicit time integrator requires a linear solve at every step, the solution from the previous time step is usually an excellent guess for the current one.
- **Physics or asymptotics.** An approximate analytical solution, a simplified model, or a known far-field behavior can supply the large scale structure of the solution.
- **Coarser grids.** The solution on a coarse grid, interpolated to the fine grid, already captures the smooth part of the solution.

In MATLAB the initial guess is the eighth argument, `gmres(A, b, restart, tol, maxit, M1, M2, x0)`; in SciPy it is the keyword argument `x0`.

## 6.5 Choosing a Tolerance

GMRES stops when the relative residual drops below a user-supplied tolerance when

$$\frac{\|\mathbf{b} - A\mathbf{x}_k\|_2}{\|\mathbf{b}\|_2} < \texttt{tol} \tag{6.9}$$

or when it runs out of iterations, which you can detect with the output `FLAG`. The discrete solution $\mathbf{u}$ of $L\mathbf{u} = \mathbf{f}$ is itself only an approximation to the PDE solution, which maintains a discretization error of $\mathcal{O}(\Delta x^2)$. The GMRES iterate $\mathbf{u}_k$ adds a second, *solver* error on top of that:

$$\underbrace{u(\mathbf{x}) - \mathbf{u}_k}_{\text{total error}} = \underbrace{u(\mathbf{x}) - \mathbf{u}}_{\text{discretization} \sim \Delta x^2} + \underbrace{\mathbf{u} - \mathbf{u}_k}_{\text{solver}}.$$

Driving the solver error far below the discretization error does not make the total error any smaller, it just costs iterations. If the solver error is *larger* than the discretization error, it dominates and you lose the convergence order of the method.

> *Remark* 7. Choose the solver tolerance only slightly smaller than the next largest source of error in your problem. For the second-order schemes in this text, that is the truncation error, so we usually want $\texttt{tol} \lesssim \Delta x^2$. The code in Section 6.7 uses $\texttt{tol} = 0.1\,\Delta x^2$.

Note that `tol` controls the *residual*, not the error. The two are related through the condition number $\kappa(A)$ by

$$\frac{\|\mathbf{x} - \mathbf{x}_k\|}{\|\mathbf{x}\|} \le \kappa(A)\, \frac{\|\mathbf{b} - A\mathbf{x}_k\|}{\|\mathbf{b}\|}, \tag{6.10}$$

and for the Laplacian $\kappa(L)$ grows as the grid is refined.

## 6.6 Looking Ahead: Preconditioning

The previous section hints at the weakness of GMRES applied directly to $L$: as the grid is refined, the number of iterations grows. This is a consequence of the spectrum of $L$ spreading out as $\Delta x \to 0$ (recall the last bullet of Section 6.3). The real power of GMRES comes from *preconditioning*: replacing $A\mathbf{x} = \mathbf{b}$ with an equivalent system such as $M^{-1}A\mathbf{x} = M^{-1}\mathbf{b}$, where $M$ is cheap to invert and $M^{-1}A$ has a far more favorable spectrum. With a good preconditioner, the iteration count can become nearly independent of $N$. Notice that GMRES still only needs to *apply* $M^{-1}A$ to a vector, so preconditioners can be matrix-free too.

## 6.7 Numerical Implementation

The code below solves (6.1) with the exact solution (6.2) twice: once with GMRES applied to the assembled sparse matrix $L$ from (6.5), and once with GMRES applied to the matrix-free function `Apply_Lap`, which evaluates (6.4) directly. Both should give the same answer to within the solver tolerance.

### 6.7.1 MATLAB

```
% grid: N interior points per direction on [0, 2*pi]
N = 128;
x = linspace(0, 2*pi, N+2)';
x(1) = [];               % remove boundary node x = 0 (Dirichlet, known)
x(end) = [];             % remove boundary node x = 2*pi
dx = x(2) - x(1);

% 1D second-derivative matrix (homogeneous Dirichlet built in)
D2 = spdiags(ones(N,1)*[1 -2 1]./(dx^2), -1:1, N, N);

% 3D Laplacian from Kronecker products
In = speye(N);
tic
Lap = kron(In,kron(In,D2)) + kron(In,kron(D2,In)) + kron(kron(D2,In),In);
toc

tic
[X,Y,Z] = meshgrid(x,x,x);
toc

% right-hand side f = Lap(u_exact) and the exact solution
tic
sx = sin(X);
sy = sin(Y);
sz = sin(Z);
esx = exp(sx.*sy.*sz);
f = -3.0*sx.*sy.*sz.*esx + ...
         sx.^2.*sy.^2.*cos(Z).^2.*esx + ...
         sx.^2.*sz.^2.*cos(Y).^2.*esx + ...
         sz.^2.*sy.^2.*cos(X).^2.*esx;
u_exact = esx - 1;
toc

% direct solve -- try this at small N and watch time and memory as N grows
% tic
% sol = Lap\f(:);
% disp(['backslash took '])
% toc

%%%%% GMRES with the assembled sparse matrix
tic
max_its = N;             % (not used below -- see the maxit argument of gmres)
tol = 0.1*dx*dx;         % slightly below the O(dx^2) truncation error
                         % try tol = 1e-14 and compare
[u_sparse_gmres,FLAG,RELRES,ITER] = gmres(Lap, f(:), 500, tol);
disp(['GMRES converged in ' num2str(ITER) ' iterations and took '])
toc
```

```
u_approx = reshape(u_sparse_gmres, N, N, N);
Rel_L2_err = sqrt(mean((u_exact(:) - u_approx(:)).^2)) ./ ...
             sqrt(mean(u_exact(:).^2));
disp(['sparse relative error: ' num2str(Rel_L2_err)])

%%%%% GMRES with a matrix-free function
tic
max_its = N;
tol = 0.1*dx*dx;
MatVec = @(x) Apply_Lap(x, dx, N);   % gmres only needs x -> Lap*x
[u_matVec_gmres,FLAG,RELRES,ITER] = gmres(MatVec, f(:), 500, tol);
disp(['mat vec GMRES converged in ' num2str(ITER) ' iterations and took '])
toc

u_approx_MV = reshape(u_matVec_gmres, N, N, N);
Rel_L2_err_matVec = sqrt(mean((u_exact(:) - u_approx_MV(:)).^2)) ./ ...
                    sqrt(mean(u_exact(:).^2));
disp(['Mat. vec. relative error: ' num2str(Rel_L2_err_matVec)])

function y = Apply_Lap(u, dx, N)
    % applies the seven-point Laplacian without forming a matrix
    u = reshape(u, N, N, N);   % reshape from vector to 3D array
    y = 0*u;                   % initialize output
    for i = 1:N
        for j = 1:N
            for k = 1:N
                % neighbors outside the grid stay zero (u = 0 on boundary)
                u_E = 0.0;
                u_W = 0.0;
                u_N = 0.0;
                u_S = 0.0;
                u_T = 0.0;
                u_B = 0.0;
                u_P = u(i,j,k);
                if i > 1; u_W = u(i-1,j,k); end
                if i < N; u_E = u(i+1,j,k); end
                if j > 1; u_S = u(i,j-1,k); end
                if j < N; u_N = u(i,j+1,k); end
                if k > 1; u_B = u(i,j,k-1); end
                if k < N; u_T = u(i,j,k+1); end
                y(i,j,k) = (1/(dx^2))*(u_W + u_E + ...
                                       u_S + u_N + ...
                                       u_T + u_B + ...
                                       -6.0*u_P);
            end
        end
    end
    y = y(:);                  % return as a vector
end
```

### 6.7.2 python

In python, element-by-element loops over $N^3$ points are far too slow to be practical, so the matrix-free operator below is written with array slicing. Note that recent versions of SciPy call the relative tolerance `rtol`.

```
import time
import numpy as np
```

```
import scipy.sparse as sp
import scipy.sparse.linalg as spla

# grid: N interior points per direction on [0, 2pi]
N = 128
x = np.linspace(0, 2*np.pi, N + 2)[1:-1]   # drop the two boundary nodes
dx = x[1] - x[0]

# 1D second-derivative matrix (homogeneous Dirichlet built in)
e = np.ones(N)
D2 = sp.diags([e[:-1], -2*e, e[:-1]], [-1, 0, 1], format='csr') / dx**2
In = sp.identity(N, format='csr')

# 3D Laplacian from Kronecker products
t = time.perf_counter()
Lap = (sp.kron(In, sp.kron(In, D2)) + sp.kron(In, sp.kron(D2, In))
       + sp.kron(sp.kron(D2, In), In)).tocsr()
print(f'building Lap took {time.perf_counter() - t:.2f} s')

# right-hand side f = Lap(u_exact) and the exact solution
X, Y, Z = np.meshgrid(x, x, x, indexing='ij')
sx, sy, sz = np.sin(X), np.sin(Y), np.sin(Z)
esx = np.exp(sx*sy*sz)
f = (-3.0*sx*sy*sz*esx
     + sx**2*sy**2*np.cos(Z)**2*esx
     + sx**2*sz**2*np.cos(Y)**2*esx
     + sz**2*sy**2*np.cos(X)**2*esx)
u_exact = esx - 1

tol = 0.1*dx**2   # slightly below the O(dx^2) truncation error


def rel_err(u):
    # relative root-mean-square error against the exact solution
    return (np.sqrt(np.mean((u_exact.ravel() - u)**2))
            / np.sqrt(np.mean(u_exact**2)))

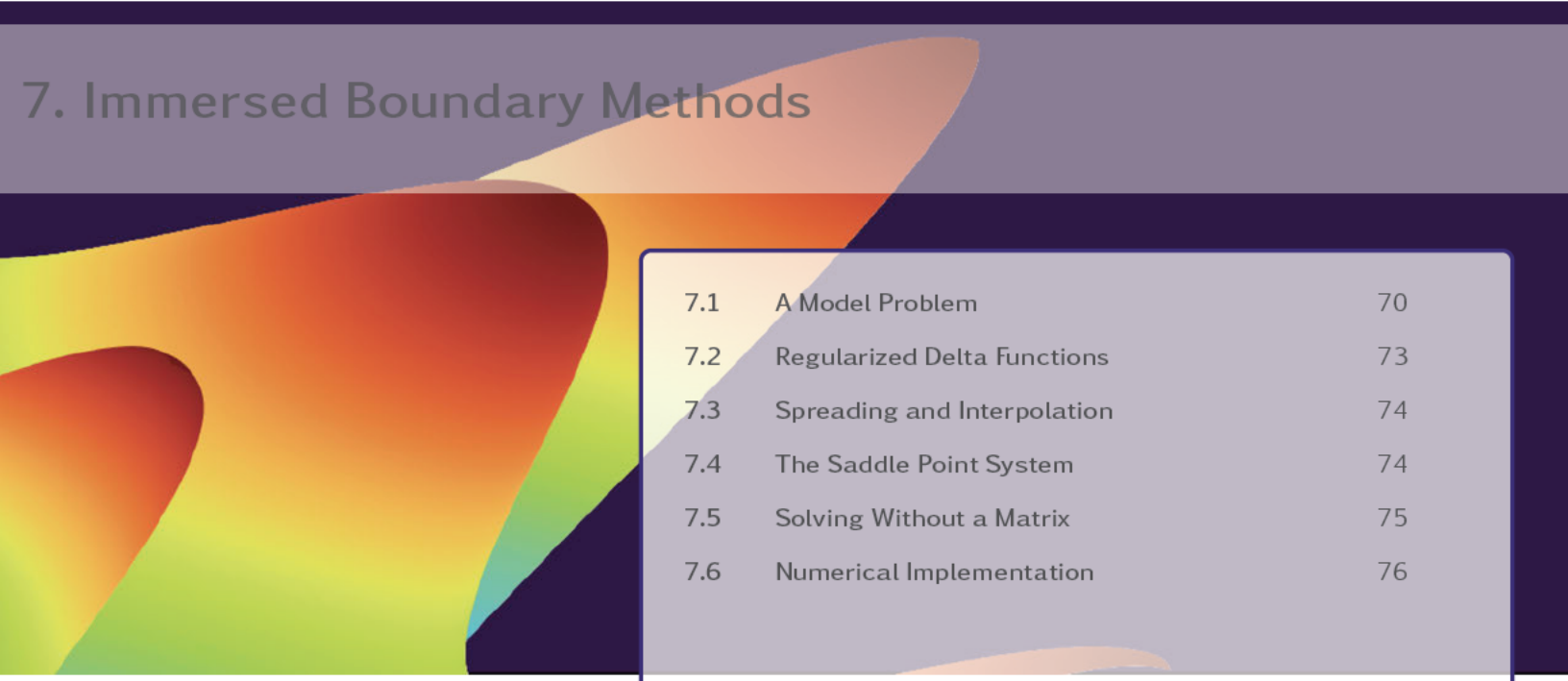

##### GMRES with the assembled sparse matrix
t = time.perf_counter()
u_sparse, flag = spla.gmres(Lap, f.ravel(), rtol=tol, restart=500)
print(f'sparse GMRES: flag {flag}, {time.perf_counter() - t:.2f} s, '
      f'relative error {rel_err(u_sparse):.3e}')


##### GMRES with a matrix-free operator
def apply_lap(u):
    u = u.reshape(N, N, N)
    up = np.pad(u, 1)   # ghost layer of zeros = homogeneous Dirichlet
    y = (up[:-2, 1:-1, 1:-1] + up[2:, 1:-1, 1:-1]     # W + E
         + up[1:-1, :-2, 1:-1] + up[1:-1, 2:, 1:-1]   # S + N
         + up[1:-1, 1:-1, :-2] + up[1:-1, 1:-1, 2:]   # B + T
         - 6.0*u) / dx**2
    return y.ravel()


# gmres only needs to know the shape and how to compute x -> Lap*x
A = spla.LinearOperator((N**3, N**3), matvec=apply_lap, dtype=float)
```

```
t = time.perf_counter()
u_mv, flag = spla.gmres(A, f.ravel(), rtol=tol, restart=500)
print(f'matrix-free GMRES: flag {flag}, {time.perf_counter() - t:.2f} s, '
      f'relative error {rel_err(u_mv):.3e}')
```

# 7. Immersed Boundary Methods




7.1 A Model Problem 70
7.2 Regularized Delta Functions 73
7.3 Spreading and Interpolation 74
7.4 The Saddle Point System 74
7.5 Solving Without a Matrix 75
7.6 Numerical Implementation 76


Immersed boundary methods (IBMs) are built from a numerical framework for *fluid–structure interaction* (FSI) and specifically designed to model interactions where an object is immersed in a viscous, incompressible fluid. The method was introduced by Peskin to simulate blood flow around heart valves [8] and has since been used for swimming organisms, cell mechanics, vesicles, flexible fibers, and many other systems [9, 10].

Let's consider an object (a heart valve leaflet, a red blood cell, a wire frame) completely submerged, or *immersed*, in a fluid domain. If we were to implement the methods of Chapter 2 directly, we need a *body-fitted mesh* (or grid) that conforms to the object so that boundary conditions can be imposed on the immersed object. These meshes are tedious to build for complex geometries and must be rebuilt at every time step if the object moves or deforms. For a system stepping forward at approximately $10^{-4}$ seconds, this remeshing process is expensive and difficult to implement, especially in three dimensions.

IBMs avoid body-fitted meshes entirely through the use of two grids: one for the fluid field and one for the immersed object. The PDEs that describe the fluid system are solved on the same uniform Eulerian grid we have used all along, ignoring the object completely. The object is then described by its own set of curvilinear, Lagrangian points and its effect on the fluid system is simply imposed onto the fluid domain.

Everything we need to implement a basic IBM has appeared in earlier chapters: the five-point Laplacian (Chapter 2), block systems with a constraint (Chapter 5), and matrix-free GMRES (Chapter 6). For those curious, a similar and more in-depth treatment can be found in Peskin's lecture notes [11].

*Remark* 8. Throughout this chapter, the term *immersed boundary* refers to the object inside the domain, not to the edge of the computational domain. The domain edge is handled by ordinary boundary conditions, exactly as in Section 4.5.

## 7.1 A Model Problem

This chapter will work through a model problem to help develop the basic ideas of IBMs.

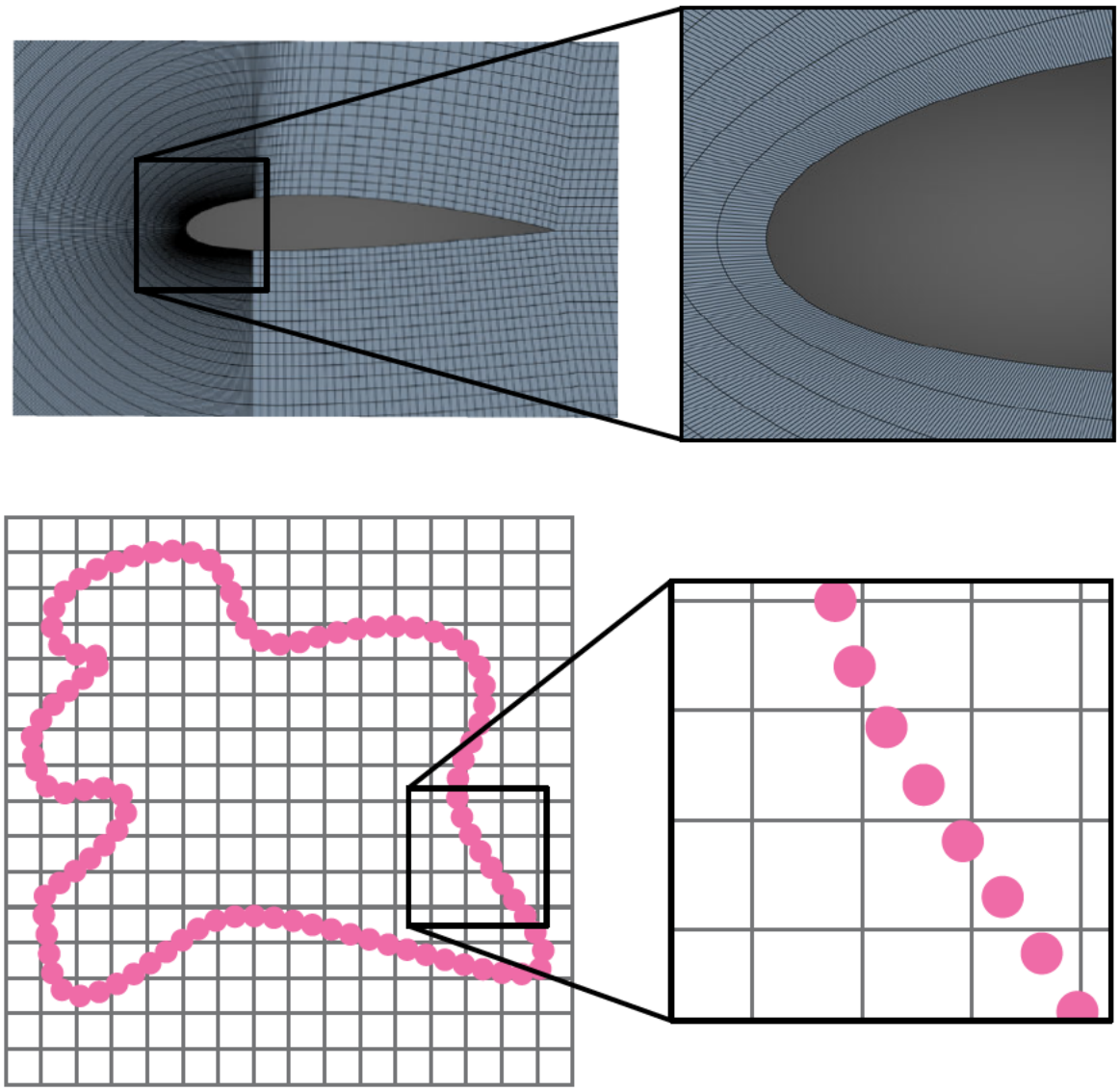


*Figure 7.1: Two ways to discretize a fluid domain containing a closed curve on which a boundary condition is imposed. Top: a body-fitted (unstructured) mesh whose edges conform to the curve (a NACA 2415 airfoil, meshed in Ansys Fluent [12]). Bottom: a uniform Cartesian grid with the curve drawn on top and represented by its own set of points (markers) that do not coincide with grid nodes.*

Let $\Omega = [0, 2\pi] \times [0, 2\pi]$ define the domain and consider

$$\nabla^2 u = f \quad \text{in } \Omega, \qquad u|_{\partial\Omega} = 0. \tag{7.1}$$

Note this is just the Poisson problem of (4.1). Let $\Gamma$ be the ellipse

$$\Gamma: \quad X(\theta) = \pi + 1.5\cos\theta, \qquad Y(\theta) = \pi + 0.75\sin\theta, \qquad 0 \le \theta < 2\pi. \tag{7.2}$$

Let $g(\theta)$ describe the values of $u$ on $\Gamma$, such that

$$u\Big(X(\theta), Y(\theta)\Big) = g(\theta), \qquad 0 \le \theta < 2\pi. \tag{7.3}$$

To enforce this condition, we add a force that acts only along the curve $\Gamma$. We describe it by a force per unit length $\lambda(s)$, where $s$ is arc length along $\Gamma$. (7.1) can then be rewritten as

$$\nabla^2 u = f + \int_\Gamma \lambda(s)\,\delta\Big(\mathbf{x} - \mathbf{X}(s)\Big)\,ds, \tag{7.4}$$

where $\mathbf{X}(s)$ is the point on $\Gamma$ and $\delta(\mathbf{x}) = \delta(x)\,\delta(y)$ is the two-dimensional Dirac delta function. The force density $\lambda$ is a new unknown and (7.3) is the equation that determines it.

### 7.1.1 Two Descriptions

Two different sets of points are used at once, and keeping them separate is the most important bookkeeping in IB methods.

Definition 7.1 **Eulerian variables** are functions of a fixed spatial position $\mathbf{x} \in \Omega$. The solution $u$ and the force $f$ are Eulerian. They are defined on the same grid we have used in previous chapters.

Definition 7.2 **Lagrangian variables** live on the immersed boundary and are functions of a coordinate along it, here the angle $\theta$. The points of the curve $\mathbf{X}(\theta)$, the boundary values $g(\theta)$, and the charges $\lambda$ are Lagrangian.

Now we discretize the problem. Let $\Gamma$ be represented by $N_{ib}$ IB points $\mathbf{X}_k = \Big(X(\theta_k), Y(\theta_k)\Big)$, with $\theta_k = k\,\Delta\theta$ with $\Delta\theta = 2\pi/N_{ib}$ for $k = 0, \ldots, N_{ib} - 1$. The local spacing between IB points is $\Delta s_k = |\mathbf{X}'(\theta_k)|\,\Delta\theta$ and, as spacing on an ellipse is not uniform in arc length, the average space is the perimeter of $\Gamma$ divided by $N_{ib}$. On the IB points, the line integral in (7.4) becomes a sum of point charges,

$$\nabla^2 u = f + \sum_{k=0}^{N_{ib}-1} q_k\,\delta(\mathbf{x} - \mathbf{X}_k), \qquad q_k \approx \lambda(s_k)\,\Delta s_k, \tag{7.5}$$

and the condition (7.3) becomes $u(\mathbf{X}_k) = g_k$ with $g_k = g(\theta_k)$.

## 7.2 Regularized Delta Functions

A classic delta function cannot be represented on a grid. We will replace it the Gaussian

$$\delta_a(x,y) = \frac{1}{2\pi a^2}\exp\left(-\frac{x^2+y^2}{2a^2}\right) = \underbrace{\frac{e^{-x^2/2a^2}}{\sqrt{2\pi}\,a}}_{\delta_a(x)}\ \underbrace{\frac{e^{-y^2/2a^2}}{\sqrt{2\pi}\,a}}_{\delta_a(y)}, \tag{7.6}$$

where the width $a$ is tied to the grid spacing, $a = c\,\Delta x$ for a constant $c$ of order one. In this chapter we take $a = 1.2\,\Delta x$. As $\Delta x \to 0$, $\delta_a$ becomes narrower and taller and approaches the true delta function. Using a Gaussian in this way is the idea behind the *force coupling method* [13], and the Gaussian delta function is the only part of this chapter that differs from Peskin's version of the method.

On the grid, integrals become sums. The property we need most is that the discrete version of $\int \delta_a(x-X)\,dx = 1$ holds no matter where $X$ sits relative to the grid:

$$\sum_i \delta_a(x_i - X)\,\Delta x \approx 1. \tag{7.7}$$

> *Remark* 9. The Gaussian is never exactly zero, so in principle every IB point touches every grid point. To make this more realistic and cost-effective, we would want to define a *support* for each delta function so it only impacts subsets of the domain.

### 7.2.1 Peskin's Four-Point Kernel

Peskin's original method uses a kernel with *compact support*. His four-point kernel [9] is written in terms of the distance to a grid point measured in grid cells, $r = x/\Delta x$, as $\phi(r) = \Delta x\,\delta_h(x)$ with

$$\phi(r) = \begin{cases} \dfrac{3 - 2|r| + \sqrt{1 + 4|r| - 4r^2}}{8}, & 0 \le |r| \le 1, \\[2ex] \dfrac{5 - 2|r| - \sqrt{-7 + 12|r| - 4r^2}}{8}, & 1 \le |r| \le 2, \\[2ex] 0, & 2 \le |r|, \end{cases} \tag{7.8}$$

and $\delta_h(x,y) = \delta_h(x)\,\delta_h(y)$ in two dimensions. Rather than being chosen for its shape, (7.8) is the unique function that satisfies the following conditions, where the sums run over all integers $j$ and must hold for every real $r$:

(i) $\phi(r)$ is continuous;

(ii) $\phi(r) = 0$ for $|r| \ge 2$;

(iii) $\displaystyle\sum_{j\ \text{even}} \phi(r-j) = \sum_{j\ \text{odd}} \phi(r-j) = \frac{1}{2}$;

(iv) $\displaystyle\sum_j (r-j)\,\phi(r-j) = 0;$

(v) $\displaystyle\sum_j \phi(r-j)^2 = \frac{3}{8}.$

Each condition has a purpose. Continuity (i) keeps the interaction smooth as an IB point slides across the grid. Compact support (ii) means each IB point touches exactly a $4\times4$ block of grid points, which makes spreading and interpolation cheap. Adding the two sums in (iii) gives (7.7) exactly, so the total charge is conserved when it is spread; requiring the even and odd sums separately also prevents a checkerboard pattern that centered differences cannot see. Condition (iv) makes interpolation exact for linear functions. Condition (v) makes the interaction between two IB points depend (approximately) only on their distance apart, not on where they sit relative to the grid. The Gaussian satisfies (i), (iii), (iv), and (v) only approximately and does not satisfy (ii) at all; the choice between the two kernels trades smoothness against cost.

## 7.3 Spreading and Interpolation

We use the Type I grid of Section 4.4 with $N$ interior points in each direction, $x_i = i\,\Delta x$ and $y_j = j\,\Delta x$ for $i, j = 1, \dots, N$, with $\Delta x = \Delta y = 2\pi/(N+1)$. The values on the walls are known, so only interior points are unknowns. Two operators connect the Eulerian grid to the IB points.

**Definition 7.3** The **spreading operator** $S$ spreads force of the charges $q_k$ onto a grid function:

$$(S\mathbf{q})_{i,j} = \sum_{k=0}^{N_{ib}-1} q_k\,\delta_a(x_i - X_k,\, y_j - Y_k). \tag{7.9}$$

**Definition 7.4** The **interpolation operator** $J$ interpolates a grid function onto the immersed object:

$$(J\mathbf{u})_k = \sum_{i=1}^{N}\sum_{j=1}^{N} u_{i,j}\,\delta_a(x_i - X_k,\, y_j - Y_k)\,\Delta x^2. \tag{7.10}$$

Spreading is the discrete version of the sum of delta functions in (7.5). Interpolation is the discrete version of $u(\mathbf{X}_k) = \iint u(\mathbf{x})\,\delta(\mathbf{x}-\mathbf{X}_k)\,d\mathbf{x}$. Spreading turns the charges on $\Gamma$ into a source on the grid, and interpolation reads the solution from the grid back at $\Gamma$. Both operators are linear. With the grid values flattened row-wise as in (4.8), $S$ is an $N^2 \times N_{ib}$ matrix and $J$ is $N_{ib} \times N^2$.

## 7.4 The Saddle Point System

Now we can put the pieces together. We discretize (7.5) with the five-point Laplacian of Section 4.1, replace the sum of delta functions with spreading, and replace the condition on

$\Gamma$, $u(\mathbf{X}_k) = g_k$, with interpolation. Let $L$ be the $N^2 \times N^2$ Laplacian (4.11) with the wall values removed. Then the solution $\mathbf{u}$ and the charges $\mathbf{q}$ must satisfy

$$L\mathbf{u} - S\mathbf{q} = \mathbf{f} + \mathbf{b}, \tag{7.11}$$

$$J\mathbf{u} = \mathbf{g}, \tag{7.12}$$

where $\mathbf{b}$ holds the boundary condition contributions of the wall values, as in (4.15) and $\mathbf{f}$ holds $f$. The first equation says the solution on the grid is produced by the forcing $f$ and the charges on $\Gamma$. The second says that the solution read back at $\Gamma$ must equal $g$. In our problem the walls are held at zero, so $\mathbf{b} = \mathbf{0}$. Stacking the two equations gives the block system

$$\begin{bmatrix} L & -S \\ J & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{q} \end{bmatrix} = \begin{bmatrix} \mathbf{f} + \mathbf{b} \\ \mathbf{g} \end{bmatrix}, \tag{7.13}$$

with $N^2 + N_{ib}$ unknowns: $N^2$ values of the solution on the grid and $N_{ib}$ charges on $\Gamma$.

This system should look familiar. Compare (7.13) with the Stokes system (5.12). In both, one of the unknowns (the pressure there, the charges here) has no equation of its own. It appears only in the other equations and its job is to enforce a constraint. An unknown of this kind is called a *Lagrange multiplier* and the zero block on the diagonal is its signature. Systems with this structure are called *saddle point systems*.

### 7.4.1 How Far Apart Should IB Points Be?

The spacing of the IB points affects both the accuracy of the solution and how hard the system (7.13) is to solve. If two IB points are much closer together than the kernel width $a$, a charge at one creates almost the same solution on the grid as a charge at the other. Their columns of $S$ are nearly identical, so the system cannot decide how to split the charge between them, and (7.13) becomes badly conditioned. On the other hand, each IB point controls the solution only in a neighborhood about as wide as its kernel. If the IB points are too far apart, their kernels do not overlap and the solution can leak through the gaps between them. Choosing the average spacing $\Delta s$ relative to $\Delta x$ (and to $a$) is therefore a balance between these two effects.

> *Remark* 10. Starting with $\Delta s \approx \Delta x$ is reasonable, but it is worth checking for yourself how the number of GMRES iterations and the accuracy of the solution change as $\Delta s/\Delta x$ is varied.

## 7.5 Solving Without a Matrix

The system (7.13) has $N^2 + N_{ib}$ unknowns, and since the Gaussian is never zero, the blocks $S$ and $J$ are effectively dense. We do not want to build this matrix. Luckily, as discussed in Section 6.2.1, GMRES only needs a function that applies the matrix to a vector. For (7.13),

that function takes a stacked vector $\mathbf{z} = [\mathbf{u}; \mathbf{q}]$ and returns

$$\mathbf{z} \mapsto \begin{bmatrix} L\mathbf{u} - S\mathbf{q} \\ J\mathbf{u} \end{bmatrix}.$$

Each piece is something we already know how to apply without a matrix: $L$ with the five-point stencil (as in `Apply_Lap` of Section 6.7, in two dimensions instead of three), and $S$ and $J$ directly from (7.9) and (7.10).

*Remark* 11. GMRES assumes the function it is given is *linear*. Known data, such as $\mathbf{f}$, the wall values, and $\mathbf{g}$, belong on the right-hand side of (7.13) and should never appear inside the matrix-apply function.

*Remark* 12. A matrix-free function is hard to inspect. For small $N$, we can build the matrix it represents one column at a time by applying the function to each column of the identity. Looking at its sparsity plot (Section 5.3.2) or checking that $J = \Delta x^2 S^T$ is the single most useful debugging step in this chapter.

The choices discussed in Chapter 6 still apply here: the tolerance (Section 6.5), the restart length (Section 6.3.1), and the initial guess (Section 6.4). Keep in mind that saddle point systems are harder for GMRES than the Laplacian alone. Recall from (6.10) that the condition number links the residual GMRES controls to the error we actually care about. If restarting stalls the iteration, try running GMRES without restarts by setting the restart length to the size of the system (or as large as memory allows) and allowing a single outer iteration.

## 7.6 Numerical Implementation

The code below sets up our problem with $f = 0$ and $g = 1$, places IB points around $\Gamma$ about $\Delta x$ apart, defines the Gaussian delta function, spreading, interpolation, and a matrix-free five-point Laplacian, and calls GMRES without restarts. The function that applies the saddle point matrix is left for you to write. The commented lines build the matrix column by column for debugging.

### 7.6.1 MATLAB

```
% grid: N interior points per direction on [0, 2*pi] (Type I)
N = 63;
x = linspace(0, 2*pi, N+2)';
x(1) = [];                % remove boundary node x = 0 (Dirichlet, known)
x(end) = [];              % remove boundary node x = 2*pi
dx = x(2) - x(1);
[X, Y] = ndgrid(x, x);    % X(i,j) = x_i and Y(i,j) = y_j
% the grid values are stored as U(i,j), so U(:) stacks the rows
% and k = i + (j-1)*N, as in the text

% load: f = 0 (the walls are held at u = 0)
f = zeros(N, N);
```

```
% immersed boundary: an elliptical ring held at height g = 1
Ax = 1.5;                % semi-axes of the ellipse
Ay = 0.75;
% measure the perimeter with a fine sampling of the ellipse
th = linspace(0, 2*pi, 10001);
perimeter = sum(hypot(diff(Ax*cos(th)), diff(Ay*sin(th))));
ds_target = dx;          % average spacing of IB points
Nib = ceil(perimeter/ds_target);
theta = 2*pi*(0:Nib-1)'/Nib;
Xib = pi + Ax*cos(theta);
Yib = pi + Ay*sin(theta);
g = ones(Nib, 1);

a = 1.2*dx;              % width of the delta function

rhs = [f(:); g];

% GMRES without restarts: restart = size of the system, one outer iteration
Afun = @(z) apply_A(z, X, Y, dx, Xib, Yib, a);
tol = 0.1*dx^2;
[z, FLAG, RELRES, ITER] = gmres(Afun, rhs, N*N + Nib, tol, 1);
U = reshape(z(1:N*N), N, N);
q = z(N*N+1:end);
disp(['GMRES flag ' num2str(FLAG) ', ' num2str(Nib) ' IB points'])

% debugging: build the matrix one column at a time (small N only!)
% n = N*N + Nib;
% Amat = zeros(n);
% I = eye(n);
% for c = 1:n
%     Amat(:,c) = Afun(I(:,c));
% end
% spy(abs(Amat) > 1e-8*max(abs(Amat(:))))

Uib = interp_J(U, X, Y, dx, Xib, Yib, a);
figure
surf(X, Y, U, 'EdgeColor', 'none')
hold on
plot3([Xib; Xib(1)], [Yib; Yib(1)], [Uib; Uib(1)], 'k.-')
hold off
xlabel('x')
ylabel('y')
zlabel('u')

function d = delta_a(rx, ry, a)
    % two-dimensional Gaussian regularized delta function
    d = exp(-(rx.^2 + ry.^2)/(2*a^2))/(2*pi*a^2);
end

function F = spread_S(q, X, Y, Xib, Yib, a)
    % S q: charges at the IB points -> N x N grid function
    F = zeros(size(X));
    for k = 1:length(Xib)
        F = F + q(k)*delta_a(X - Xib(k), Y - Yib(k), a);
    end
end

function Uib = interp_J(U, X, Y, dx, Xib, Yib, a)
```

```
    % J u: N x N grid function -> values at the IB points
    Uib = zeros(length(Xib), 1);
    for k = 1:length(Xib)
        Uib(k) = sum(sum(U.*delta_a(X - Xib(k), Y - Yib(k), a)))*dx^2;
    end
end

function V = apply_L(U, dx)
    % five-point Laplacian, homogeneous Dirichlet (zero ghost layer)
    N = size(U, 1);
    Up = zeros(N+2, N+2);
    Up(2:end-1, 2:end-1) = U;
    V = (Up(1:end-2, 2:end-1) + Up(3:end, 2:end-1) ...   % W + E
       + Up(2:end-1, 1:end-2) + Up(2:end-1, 3:end) ...   % S + N
       - 4*U)/dx^2;
end

function y = apply_A(z, X, Y, dx, Xib, Yib, a)
    N = size(X, 1);
    y = zeros(size(z));
    %%%
    % IMPLEMENT THE SADDLE POINT OPERATOR HERE
    % z holds the N*N grid values (stacked as U(:)) followed by the
    % Nib charges. Reshape the grid part to N x N, apply L, S, and J,
    % and return the stacked vector [L u - S q ; J u]
    %%%
end
```

### 7.6.2 python

```
import numpy as np
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt

# N interior points per direction on [0, 2pi] (Type I)
N = 63
x = np.linspace(0, 2*np.pi, N + 2)[1:-1]   # drop the two boundary nodes
dx = x[1] - x[0]
X, Y = np.meshgrid(x, x)    # X[j, i] = x_i and Y[j, i] = y_j
# the grid values are stored as U[j, i], so U.ravel() stacks the rows
# and k = i + j*N, as in the text

# f = 0 (the walls are held at u = 0)
f = np.zeros((N, N))

# immersed boundary: an elliptical ring held at height g = 1
Ax, Ay = 1.5, 0.75                          # semi-axes of the ellipse
# measure the perimeter with a fine sampling of the ellipse
th = np.linspace(0, 2*np.pi, 10001)
perimeter = np.sum(np.hypot(np.diff(Ax*np.cos(th)), np.diff(Ay*np.sin(th))))
ds_target = dx                              # average spacing of IB points
Nib = int(np.ceil(perimeter/ds_target))
theta = 2*np.pi*np.arange(Nib)/Nib
Xib = np.pi + Ax*np.cos(theta)
Yib = np.pi + Ay*np.sin(theta)
g = np.ones(Nib)

a = 1.2*dx                                  # width of the delta function
```

```
def delta_a(rx, ry):
    # two-dimensional Gaussian regularized delta function
    return np.exp(-(rx**2 + ry**2)/(2*a**2))/(2*np.pi*a**2)


def spread(q):
    # charges at the IB points
    F = np.zeros((N, N))
    for k in range(Nib):
        F += q[k]*delta_a(X - Xib[k], Y - Yib[k])
    return F


def interp(U):
    # values at the IB points
    out = np.zeros(Nib)
    for k in range(Nib):
        out[k] = np.sum(U*delta_a(X - Xib[k], Y - Yib[k]))*dx**2
    return out


def apply_L(U):
    # five-point Laplacian, homogeneous Dirichlet
    Up = np.pad(U, 1)
    return (Up[1:-1, :-2] + Up[1:-1, 2:]       # W + E
            + Up[:-2, 1:-1] + Up[2:, 1:-1]     # S + N
            - 4*U)/dx**2


def apply_A(z):
    ###
    # IMPLEMENT THE SADDLE POINT OPERATOR HERE
    # z holds the N*N grid values (flattened row-wise) followed by the
    # Nib charges. Reshape the grid part to N x N, apply L, S, and J,
    # and return the flattened, stacked vector [L u - S q ; J u]
    ###
    return np.zeros(N*N + Nib)


rhs = np.concatenate([f.ravel(), g])

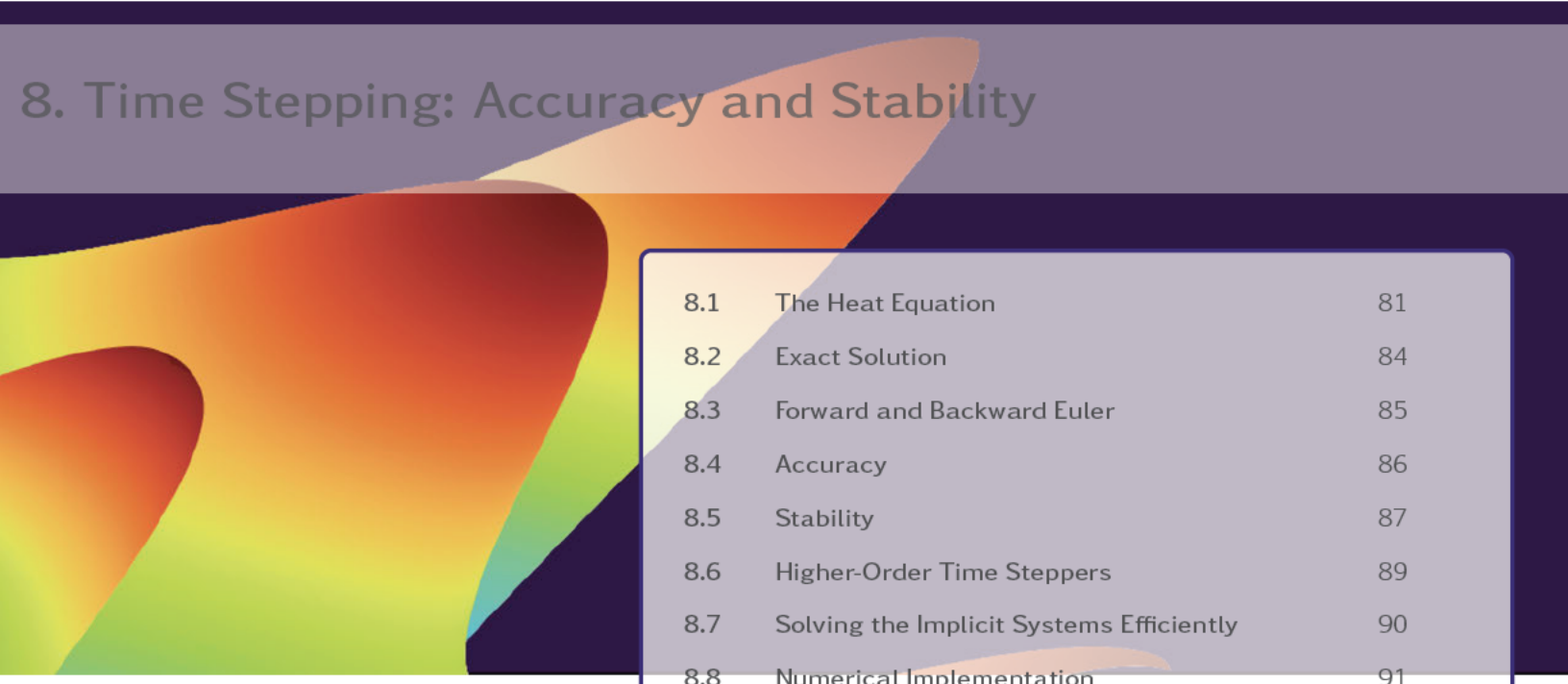
# GMRES without restarts: restart = size of the system, one outer iteration
A = spla.LinearOperator((N*N + Nib, N*N + Nib), matvec=apply_A, dtype=float)
tol = 0.1*dx**2
z, flag = spla.gmres(A, rhs, rtol=tol, restart=N*N + Nib, maxiter=1)
U = z[:N*N].reshape(N, N)
q = z[N*N:]
print(f'GMRES flag {flag}, {Nib} IB points')

# debugging option -- build the matrix one column at a time (small N only!)
# Amat = np.column_stack([apply_A(e) for e in np.eye(N*N + Nib)])
# plt.spy(Amat, precision=1e-8)
# plt.show()

Uib = interp(U)
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X, Y, U, cmap='viridis', linewidth=0)
```

```
ax.plot(np.r_[Xib, Xib[0]], np.r_[Yib, Yib[0]], np.r_[Uib, Uib[0]], 'k.-')
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.set_zlabel('$u$')
plt.show()
```

# 8. Time Stepping: Accuracy and Stability




| | | |
|---|---|---|
| 8.1 | The Heat Equation | 81 |
| 8.2 | Exact Solution | 84 |
| 8.3 | Forward and Backward Euler | 85 |
| 8.4 | Accuracy | 86 |
| 8.5 | Stability | 87 |
| 8.6 | Higher-Order Time Steppers | 89 |
| 8.7 | Solving the Implicit Systems Efficiently | 90 |
| 8.8 | Numerical Implementation | 91 |


Every problem we have seen so far has been *steady*, which means the unknown $\phi$ does not change with time. In this chapter, we will see how to handle problems where $\phi$ changes in time using *time stepping* or *time integration*. There are many time stepping methods, and each comes with its own costs. Two questions guide our comparison of them:

1. **Accuracy.** How fast does the error shrink as the time step $\Delta t$ is made smaller?
2. **Stability.** Does the numerical solution stay bounded at all, or can it blow up even when the true solution decays?

Answering both lets us weigh the cost of each method against what it gives us. Our model problem will be the heat equation, the parabolic member of the three classic PDEs introduced in Chapter 1.

## 8.1 The Heat Equation

Let $\Omega = [a,b]\times[c,d] \subset \mathbb{R}^2$ and let $\phi(x,y,t)$ be temperature. We solve for $\phi$ satisfying

$$\frac{\partial \phi}{\partial t} = k\nabla^2\phi + f(x,y,t), \qquad (x,y)\in\Omega, \quad t>0, \tag{8.1}$$

where $k>0$ is the diffusivity (i.e., how quickly the heat spreads) and $f$ is a known forcing function (i.e., a heat source/sink). Unlike the steady problems of earlier chapters, the solution requires

- an **initial condition**, $\phi(x,y,0)=\phi_0(x,y)$ for all $(x,y)\in\Omega$, and
- **boundary conditions** on $\partial\Omega$ for every $t>0$, exactly as in Section 4.5, except the boundary data may now depend on time.

Equation (8.1) is also called the *diffusion equation*, because temperature is only one example. The quantity $\phi$ can be many things: temperature (heat), the concentration of a dissolved

chemical, the density of a pollutant in the air, or even the density of people wandering randomly around a room. In every case, the Laplacian moves $\phi$ from where there is more of it to where there is less, so the heat equation *smooths things out*. Sharp features are rounded off, peaks flatten, and in some cases the solution relaxes toward a steady state.

### 8.1.1 A Random Walk

Why does diffusion look like a Laplacian? Consider a person who has had a bit too much to drink and is walking along a line, starting from the point $x_0$. Every $\Delta t$ seconds they stumble a distance $\Delta x$, either to the right with probability $p = 0.5$, or to the left with probability $1-p = 0.5$. A single walker's path is random and jagged, as seen in the left panel of Figure 8.1. However, if we start $N_w$ walkers at the same point $x_0$ and let them all stumble independently, we can describe the *distribution* of their positions at a given time. Let $P(t,x)$ be the probability of finding a walker at position $x$ at time $t$. To be at $x$ at time $t+\Delta t$, a walker must have been at $x-\Delta x$ and stumbled right, or at $x+\Delta x$ and stumbled left, so

$$P(t+\Delta t,x) = \tfrac{1}{2}P(t,x-\Delta x) + \tfrac{1}{2}P(t,x+\Delta x). \tag{8.2}$$

Taylor expanding both sides of (8.2), the way we did for finite differences in Section 2.3, the left-hand side expands in time,

$$P(t+\Delta t,x) = P + \Delta t\,\frac{\partial P}{\partial t} + \frac{\Delta t^2}{2}\frac{\partial^2 P}{\partial t^2} + \mathcal{O}(\Delta t^3),$$

and the two terms on the right-hand side expand in space,

$$\begin{aligned} P(t,x+\Delta x) &= P + \Delta x\,\frac{\partial P}{\partial x} + \frac{\Delta x^2}{2}\frac{\partial^2 P}{\partial x^2} + \frac{\Delta x^3}{6}\frac{\partial^3 P}{\partial x^3} + \frac{\Delta x^4}{24}\frac{\partial^4 P}{\partial x^4} + \mathcal{O}(\Delta x^5),\\ P(t,x-\Delta x) &= P - \Delta x\,\frac{\partial P}{\partial x} + \frac{\Delta x^2}{2}\frac{\partial^2 P}{\partial x^2} - \frac{\Delta x^3}{6}\frac{\partial^3 P}{\partial x^3} + \frac{\Delta x^4}{24}\frac{\partial^4 P}{\partial x^4} + \mathcal{O}(\Delta x^5), \end{aligned}$$

where $P$ and its derivatives on the right are all evaluated at $(t,x)$. Averaging the two spatial expansions, the odd derivatives cancel, just as for the central difference (2.7):

$$\tfrac{1}{2}P(t,x-\Delta x) + \tfrac{1}{2}P(t,x+\Delta x) = P + \frac{\Delta x^2}{2}\frac{\partial^2 P}{\partial x^2} + \frac{\Delta x^4}{24}\frac{\partial^4 P}{\partial x^4} + \mathcal{O}(\Delta x^6).$$

Setting the two sides equal, cancelling $P$, and dividing by $\Delta t$ gives

$$\frac{\partial P}{\partial t} = \frac{\Delta x^2}{2\Delta t}\frac{\partial^2 P}{\partial x^2} \underbrace{- \frac{\Delta t}{2}\frac{\partial^2 P}{\partial t^2} + \frac{\Delta x^4}{24\Delta t}\frac{\partial^4 P}{\partial x^4} + \cdots}_{\text{higher-order terms}} = \frac{\Delta x^2}{2\Delta t}\frac{\partial^2 P}{\partial x^2} + (\text{higher-order terms}). \tag{8.3}$$

Apart from the higher-order terms, this is the heat equation in one dimension, with diffusivity $k = \Delta x^2/(2\Delta t)$.

The higher-order terms of (8.3) are negligible when $P$ barely changes over a single step.

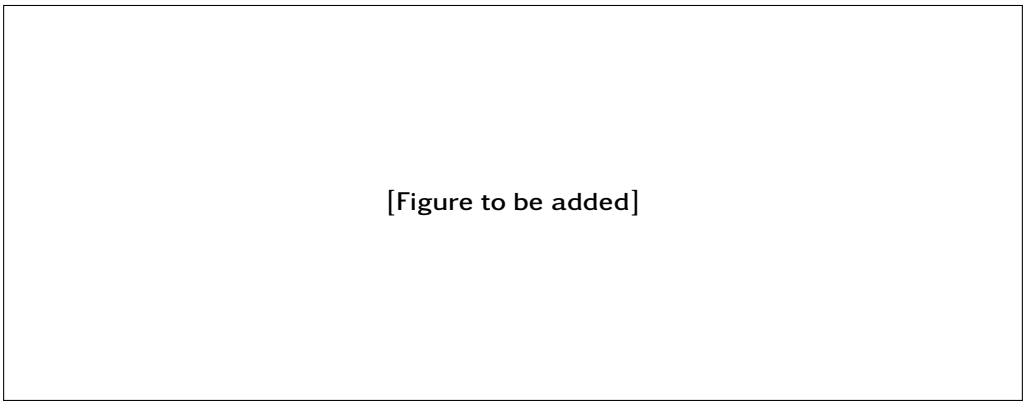



*Figure 8.1: Left: the position of a few random walkers plotted against time, each taking steps of $\pm\Delta x$ every $\Delta t$, with vertical lines marking $t = 1$ and $t = 10$. Right: the distribution $P(t,x)$ of many walkers' positions (as histograms, i.e. data) at $t = 1$ and $t = 10$, with the corresponding solutions of the heat equation (8.3) drawn as curves. The later distribution should be visibly wider and shorter.*

When does that happen? We can think of our drunk walkers are molecules moving randomly. At the macroscopic level, the amount the individual molecules change with space ($\Delta x$) with respect to time ($\Delta t$) are very small. So let's consider (8.3) as $\Delta x \to 0$ and $\Delta t \to 0$.

- If $\Delta t \propto \Delta x$, the coefficient behaves like $\Delta x \to 0$ and the limit is $\partial P/\partial t = 0$: the walkers never spread.
- If $\Delta t \propto \Delta x^2$, the coefficient stays fixed, and choosing $\Delta t = \Delta x^2/(2k)$ gives the heat equation with diffusivity $k$.
- If $\Delta t \propto \Delta x^3$, the coefficient behaves like $1/\Delta x \to \infty$: the walkers spread infinitely far in any positive time.

Only $\Delta t \propto \Delta x^2$ gives a finite, nonzero limit. At this scaling, the higher-order terms of (8.3) go to zero.

*Remark* 13. Keep the scaling $\Delta t \sim \Delta x^2$ in mind; it will reappear in Section 8.5.

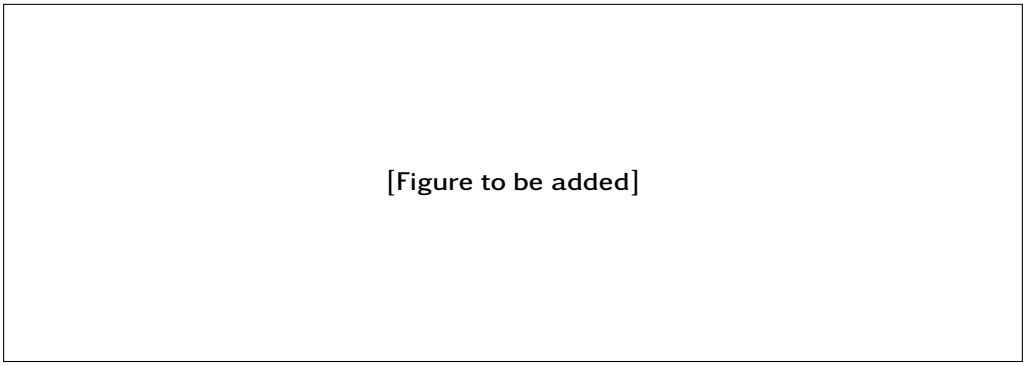



*Figure 8.2: A probability distribution $P(t,x)$ obtained from many random walkers, plotted at three times (e.g. $t = 0$, $t \approx 1.3$, $t \approx 11$). The earliest distribution is narrow and noisy; later ones are progressively wider, flatter, and smoother. Each time should use a distinct color* and *line style.*

## 8.2 Exact Solution

With the random walk as motivation, we return to the two-dimensional heat equation (8.1). We already know how to spatially discretize the Laplacian $\nabla^2$. Let's consider this problem on a Type I grid with Dirichlet boundary conditions on the domain $\Omega = [0,\ell]^2$ such that for $N$ points,

$$x_i = i\,\Delta x, \quad y_j = j\,\Delta x, \qquad i,j = 1,\dots,N, \qquad \Delta x = \frac{\ell}{N+1}. \tag{8.4}$$

We can discretize (8.1) in terms of the Laplacian $L$ and flatten the unknown field $\boldsymbol{\phi}$ to obtain a large system of ordinary differential equations (ODEs),

$$\frac{d\boldsymbol{\phi}}{dt} = k\,L\,\boldsymbol{\phi} + \mathbf{g}(t), \qquad \boldsymbol{\phi}(0) = \boldsymbol{\phi}_0, \tag{8.5}$$

where $\mathbf{g}(t)$ collects the forcing $f$ evaluated at the grid points and the boundary contributions (which now may change in time).

> *Remark* 14. Note we have separated the problem into two independent choices: how to discretize space and how to step in time. The spatial error and the temporal error then add together, just as the discretization and solver errors did in Section 6.5.

Before discretizing time, it helps to look at the exact solution of (8.5). Let $k = 1$ and $\mathbf{g} = \mathbf{0}$. For a scalar ODE $y' = \lambda y$, we would write $y(t) = e^{\lambda t}y(0)$. The exact solution to (8.5) is then

$$\boldsymbol{\phi}(t) = e^{tL}\boldsymbol{\phi}_0, \qquad e^{tL} = I + tL + \frac{t^2}{2}L^2 + \frac{t^3}{3!}L^3 + \cdots, \tag{8.6}$$

where the matrix exponential is defined by its Taylor series, which converges far too slowly for this to be a practical solution. Since $L$ is symmetric, it can be decomposed as

$$L = P\,D\,P^{\mathsf{T}}, \qquad D = \operatorname{diag}(\lambda_1,\lambda_2,\dots,\lambda_n), \tag{8.7}$$

where the columns of $P$ are the eigenvectors and $\lambda_m$ the eigenvalues of $L$. Let $\mathbf{y} = P^{\mathsf{T}}\boldsymbol{\phi}$. Multiplying $\boldsymbol{\phi}' = kL\boldsymbol{\phi}$ by $P^{\mathsf{T}}$ gives $\mathbf{y}' = kD\mathbf{y}$. Each row of this system represents the scalar equation

$$y_m' = k\lambda_m y_m$$

. Defining $\hat{\lambda}_m = k\lambda_m$, we can consider one of these scalar equations as

$$\frac{dy_m}{dt} = \hat{\lambda}_m y_m, \qquad y_m(0) = y_m^0, \qquad \text{with exact solution} \qquad y_m(t) = e^{\hat{\lambda}_m t}\,y_m^0. \tag{8.8}$$

*Remark* 15. If we understand what a time stepper does to (8.8) for one eigenvalue $\lambda_m$, we understand what it does to the whole system, one eigenvalue at a time. Each of the following time steppers, applied to $\boldsymbol{\phi}' = kL\boldsymbol{\phi}$, is the same as applying it to every $y_m$ separately.

This reduces the question to what the biggest $\lambda$ is. For the system we have defined thus far, the eigenvalues are

$$\lambda_{p,q} = -\frac{4}{\Delta x^2}\left[\sin^2\left(\frac{p\pi\Delta x}{2\ell}\right) + \sin^2\left(\frac{q\pi\Delta x}{2\ell}\right)\right], \qquad p,q = 1,\ldots,N. \tag{8.9}$$

Every eigenvalue is real and negative, so every mode of the true solution decays. The eigenvalue of largest magnitude belongs to the most oscillatory mode $p = q = N$ (a checkerboard), and since the $\sin^2$ terms are then nearly one,

$$\lambda_{\max} \approx -\frac{8}{\Delta x^2}. \tag{8.10}$$

We have thus defined the largest possible eigenvalue, which we help us with our stability analysis for each temporal scheme.

## 8.3 Forward and Backward Euler

Discretization of time is very similar to that for space. Consider the time interval $[0,T]$ evaluated over $N_t$ equally spaced points, such that $\Delta t = T/N_t$. Then, we can define time levels, or evaluations of the system at specific times, as

$$t^n = n\Delta t, \qquad n = 0,1,\ldots,N_t, \qquad \boldsymbol{\phi}^n \approx \boldsymbol{\phi}(t^n). \tag{8.11}$$

Superscripts denote the time level, while subscripts still denote the grid point, so $\phi^n_{i,j} \approx \phi(x_i, y_j, t^n)$. The simplest approximation of the time derivative is the first-order difference

$$\frac{d\boldsymbol{\phi}}{dt} \approx \frac{\boldsymbol{\phi}^{n+1} - \boldsymbol{\phi}^n}{\Delta t},$$

and identical to Section 2.2), we will define forward and backwards approximations of this derivative.

**Forward Euler.** Evaluate $\mathbf{F}$ at the *known* level $t^n$:

$$\frac{\boldsymbol{\phi}^{n+1} - \boldsymbol{\phi}^n}{\Delta t} = \mathbf{F}(t^n, \boldsymbol{\phi}^n) \quad \Longrightarrow \quad \boxed{\boldsymbol{\phi}^{n+1} = (I + \Delta t\, kL)\,\boldsymbol{\phi}^n + \Delta t\, \mathbf{g}(t^n).} \tag{8.12}$$

Each step is a sparse matrix-vector multiplication.

**Backward Euler.** Evaluate $\mathbf{F}$ at the *unknown* level $t^{n+1}$:

$$\frac{\boldsymbol{\phi}^{n+1}-\boldsymbol{\phi}^{n}}{\Delta t}=\mathbf{F}(t^{n+1},\boldsymbol{\phi}^{n+1}) \quad\Longrightarrow\quad \boxed{(I-\Delta t\,kL)\,\boldsymbol{\phi}^{n+1}=\boldsymbol{\phi}^{n}+\Delta t\,\mathbf{g}(t^{n+1}).} \tag{8.13}$$

Now each step requires a linear *solve*. A solve is going to be more expensive than a multiplication, so there must be a tradeoff that makes backward Euler useful. Finding that tradeoff is the goal of the next two sections.

*Remark* 16. A method in which the new value is given directly by known values is called *explicit*. A method whose new value is defined only by unknown values is called *implicit*.

*Remark* 17. Starting from $\boldsymbol{\phi}^0=\boldsymbol{\phi}_0$, it takes exactly $N_t$ steps to reach $\boldsymbol{\phi}^{N_t}\approx\boldsymbol{\phi}(T)$. A loop that runs one step too many or too few compares your solution at $T\pm\Delta t$ with the exact solution at $T$, an error that looks like a first-order method no matter what you implemented. Likewise, if $\Delta t$ does not divide $T$ evenly, you can never land on $T$. Compute the time of each level as $t^n=n\Delta t$ rather than repeatedly adding $\Delta t$, which accumulates round-off.

## 8.4 Accuracy

To compare the two methods, apply both to the test equation (8.8). Forward Euler gives $y^{n+1}=(1+\Delta t\lambda)\,y^n$ and backward Euler gives $(1-\Delta t\lambda)\,y^{n+1}=y^n$. Repeating either $n$ times, we can write the *exact* solutions of the two approximations,

$$\text{exact:}\qquad y(t^n)=e^{\lambda t^n}\,y^0, \tag{8.14}$$

$$\text{FE:}\qquad y^n=(1+\Delta t\,\lambda)^n\,y^0, \tag{8.15}$$

$$\text{BE:}\qquad y^n=\left(\frac{1}{1-\Delta t\,\lambda}\right)^n y^0. \tag{8.16}$$

The error of forward Euler at $t^n$, measured relative to the size of the initial data, is

$$\frac{|y^n-y(t^n)|}{|y^0|}=\left|(1+\Delta t\lambda)^n-e^{\lambda t^n}\right|=\frac{1}{2}\,\lambda^2\,t^n\,e^{\lambda t^n}\,\Delta t+\mathcal{O}(\Delta t^2). \tag{8.17}$$

We will see in the next section what can go wrong. The error is proportional to $\Delta t$, so forward Euler is first-order accurate.

*Remark* 18. The test equation gives us exact formulas, but for a general method we use the method of manufactured solutions.

Definition 8.1 **(Local truncation error)** The *local truncation error* $\tau^n$ of a time stepper is the error accumulated from the iterative scheme starting from an exact solution per unit of time.

For forward Euler, $\tau^n = \frac{\phi(t^{n+1})-\phi(t^n)}{\Delta t} - \mathbf{F}(t^n, \phi(t^n)) = \frac{\Delta t}{2}\phi''(t^n) + \cdots$, which is exactly the forward difference error (2.2).

The local errors made at each of the $N_t = T/\Delta t$ steps accumulate into *global error*. If $\tau^n = \mathcal{O}(\Delta t^p)$ and the method is **stable**, then the *global error* at a fixed time $T$ is also $\mathcal{O}(\Delta t^p)$ and the method is said to be *p-th order accurate*. If the method is unstable, global error becomes unbounded and your system will not converge.

### 8.4.1 Measuring the Order of Accuracy

In practice, we check the order of a time stepper numerically. We use a manufactured solution that now also depends on time, compute the solution at a fixed final time $T$ for a sequence of time steps, and measure the relative error

$$\mathrm{Err}(\Delta t) = \frac{\left\|\phi^{N_t} - \phi_{\mathrm{exact}}(T)\right\|}{\left\|\phi_{\mathrm{exact}}(T)\right\|}. \tag{8.18}$$

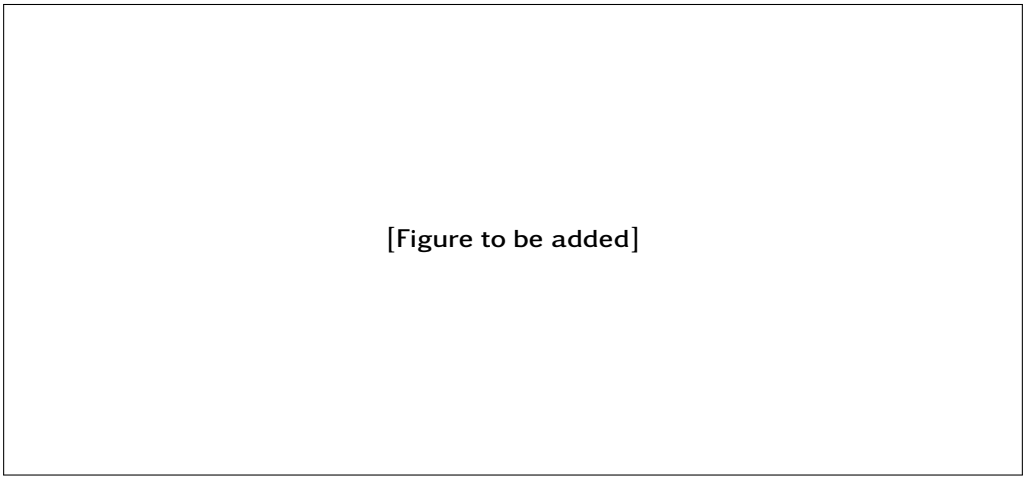



*Figure 8.3: A convergence study for backward Euler on the unforced test problem $\phi = e^{-2kt}\sin(x)\sin(y)$ on $[0, 2\pi]^2$. Relative error (8.18) at a fixed $T$ versus $\Delta t$ on log-log axes for two spatial resolutions $N$, drawn with distinct markers and colors, with a dashed $\mathcal{O}(\Delta t)$ reference line. For small $\Delta t$ each curve should flatten at a level set by the spatial error, with the finer grid flattening lower.*

## 8.5 Stability

Look again at the error (8.17) and the test equation with $\lambda < 0$, which we know is the case for every eigenvalue of the Laplacian. The exact solution $e^{\lambda t}y^0$ decays to zero as $t \to \infty$, so we expect the numerical approximation to as well.

Consider the forward Euler case, we only see a decay in the solution when

$$|1 + \Delta t\, \lambda| < 1. \tag{8.19}$$

This is called a *stability restriction* and it must hold for *every* eigenvalue. Recalling that we

determined the maximum eigenvalue as $\lambda = k\lambda_{\max} \approx -8k/\Delta x^2$ in (8.10),

$$\begin{aligned} -1 &< 1 + \Delta t\left(\frac{-8k}{\Delta x^2}\right) < 1 \\ -2 &< \Delta t\left(\frac{-8k}{\Delta x^2}\right) < 0 \\ \frac{1}{4} &> \frac{k\,\Delta t}{\Delta x^2} > 0, \end{aligned}$$

so forward Euler for the two-dimensional heat equation requires

$$\boxed{\frac{4k\,\Delta t}{\Delta x^2} < 1.} \tag{8.20}$$

If this relation holds, the error (8.17) decays along with the solution for every mode. If it does not, the most oscillatory mode is multiplied by a number less than $-1$ every step, flips sign, grows, and soon swamps the solution with a growing checkerboard pattern. Notice the scaling: $\Delta t$ must shrink like $\Delta x^2$, the diffusive scaling of Section 8.1.1.

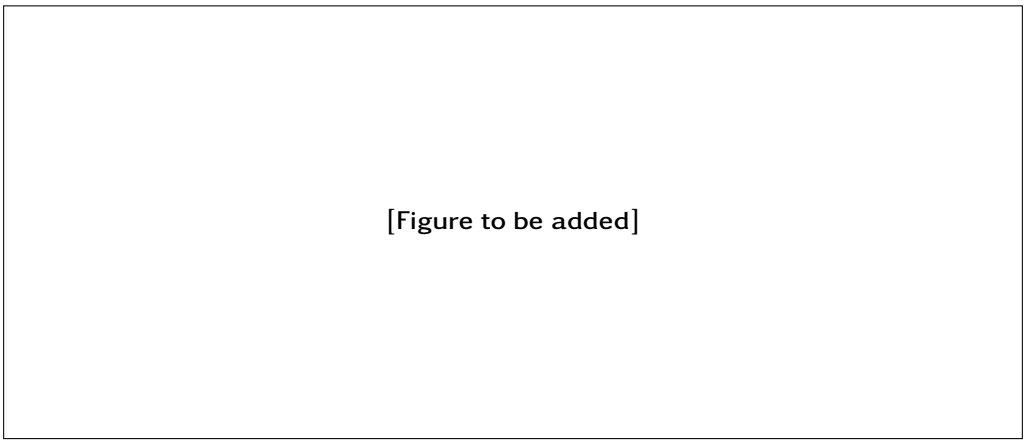



*Figure 8.4: Forward Euler applied to the test equation $y' = \lambda y$ with $\lambda < 0$. The exact solution (curve) is compared with forward Euler (symbols) for three time steps: $|1 + \Delta t\lambda|$ well below one (smooth decay), $-1 < 1 + \Delta t\lambda < 0$ (decay with alternating sign), and $1 + \Delta t\lambda < -1$ (oscillating growth). Backward Euler at the largest time step may be added for comparison.*

**Backward Euler.** Repeating the argument with (8.16), backward Euler decays only if

$$\left|\frac{1}{1 - \Delta t\,\lambda}\right| < 1. \tag{8.21}$$

For $\lambda < 0$ and any $\Delta t > 0$, the denominator $1 - \Delta t\lambda$ is larger than one, so (8.21) always holds. We end up with *no* restriction on $\Delta t$ at all, which we refer to as *unconditionally stable*.

### 8.5.1 The Tradeoff Between Forward and Backward Euler

Recall that backward Euler requires a linear solve, making it more expensive than forward. There is a tradeoff between the two, however, and it is related to their stabilities.

- Backward Euler is an **implicit** method. Each step needs a linear solve, but there are no (or few) restrictions on $\Delta t$ for stability.
- Forward Euler is an **explicit** method. There is no system to solve, so each step is very fast, but $\Delta t$ is restricted by (8.20).

*Stability is not accuracy.* Backward Euler is stable for an enormous $\Delta t$, but its answer will be inaccurate. The point is that with an implicit method, $\Delta t$ can be chosen by the accuracy you need, while with an explicit method $\Delta t$ may be forced far smaller than accuracy requires.

## 8.6 Higher-Order Time Steppers

Stability tells us which time steps we may take, and accuracy tells us how good the answer is. Both Euler methods are only first-order accurate. Just as the central difference of Section 2.2.3 improved on the one-sided differences, we can do better in time. We describe four methods for the general system $\boldsymbol{\phi}' = \mathbf{F}(t, \boldsymbol{\phi})$, where we have defined $\mathbf{F}(t, \boldsymbol{\phi}) = kL\boldsymbol{\phi} + \mathbf{g}(t)$ for the heat equation.

### 8.6.1 Crank–Nicolson

Instead of evaluating $\mathbf{F}$ at one end of the step, average the two ends:

$$\frac{\boldsymbol{\phi}^{n+1} - \boldsymbol{\phi}^n}{\Delta t} = \frac{1}{2}\Big[\mathbf{F}(t^n, \boldsymbol{\phi}^n) + \mathbf{F}(t^{n+1}, \boldsymbol{\phi}^{n+1})\Big]. \tag{8.22}$$

This is the trapezoidal rule, known as the *Crank–Nicolson* method when applied to PDEs. The difference quotient is now *centered* at $t^{n+1/2}$, which is the source of its extra order of accuracy (recall (2.7)!). Since $\boldsymbol{\phi}^{n+1}$ appears on the right-hand side, it is implicit.

### 8.6.2 Explicit Midpoint (RK2)

Another way to center the step at $t^{n+1/2}$ is to first take half a forward Euler step to estimate the solution at the midpoint, and then use the slope there for the full step:

$$\boldsymbol{\phi}^* = \boldsymbol{\phi}^n + \frac{\Delta t}{2}\,\mathbf{F}(t^n, \boldsymbol{\phi}^n), \tag{8.23}$$

$$\boldsymbol{\phi}^{n+1} = \boldsymbol{\phi}^n + \Delta t\,\mathbf{F}\Big(t^n + \tfrac{\Delta t}{2}, \boldsymbol{\phi}^*\Big). \tag{8.24}$$

The intermediate value $\boldsymbol{\phi}^*$ is called a *stage*. Methods built from several stages are *Runge–Kutta* (RK) methods. This one, the explicit midpoint method, is a second-order RK method (RK2). It requires no solve, so it is explicit.

### 8.6.3 BDF2

Runge–Kutta methods use extra stages within one step. *Multistep* methods instead reuse the solution from earlier steps. The *backward differentiation formulas* (BDF) approximate $d\boldsymbol{\phi}/dt$ at the new level $t^{n+1}$ by a one-sided difference through several past levels, and evaluate $\mathbf{F}$ at $t^{n+1}$, as backward Euler does (backward Euler is BDF1). The second-order member is

$$\frac{3\boldsymbol{\phi}^{n+1} - 4\boldsymbol{\phi}^{n} + \boldsymbol{\phi}^{n-1}}{2\,\Delta t} = \mathbf{F}(t^{n+1}, \boldsymbol{\phi}^{n+1}). \tag{8.25}$$

BDF2 cannot start on its own. To compute $\boldsymbol{\phi}^{1}$ it needs $\boldsymbol{\phi}^{0}$ and $\boldsymbol{\phi}^{-1}$, and the latter does not exist. The first step must be taken with a one-step method (or, when testing with a manufactured solution, $\boldsymbol{\phi}^{1}$ can be taken from the exact solution).

### 8.6.4 Adaptive Runge–Kutta (RK45)

Rather than fixing $\Delta t$, *adaptive* methods choose it as they go. The popular RK45 (Dormand–Prince) pair computes a fourth- and a fifth-order approximation from the same stages. The difference of these approximations estimates the error of the step, and $\Delta t$ is grown or shrunk to keep that estimate below a user-supplied tolerance. Writing one is tedious, so we use the open implementations `ode45` in MATLAB and `scipy.integrate.solve_ivp(..., method='RK45')` in python. Both take a function that evaluates $\mathbf{F}(t, \boldsymbol{\phi})$, the time interval, the initial condition, and relative and absolute tolerances, and both can report how many times $\mathbf{F}$ was evaluated.

| *Method* | *Type* | *Order* | *Stable for all $z < 0$?* |
|---|---|---|---|
| Forward Euler (8.12) | explicit | 1 | no, only $-2 \le z \le 0$ |
| Backward Euler (8.13) | implicit | 1 | yes |
| Crank–Nicolson (8.22) | implicit | 2 | yes |
| Explicit midpoint (8.24) | explicit | 2 | no, only $-2 \le z \le 0$ |
| BDF2 (8.25) | implicit | 2 | yes |
| RK45 | explicit | 4/5 (adaptive) | no, only about $-3.3 \le z \le 0$ |

*Table 8.1: Summary of the time steppers in this chapter. Here $z = \Delta t\,\lambda$, where $\lambda$ is an eigenvalue of $kL$.*

## 8.7 Solving the Implicit Systems Efficiently

Implicit methods buy their stability with a linear solve at every step, so the cost of that solve deserves attention. Backward Euler, Crank–Nicolson, and BDF2 all reduce each time step to

a linear system of the form

$$A\boldsymbol{\phi}^{n+1} = \mathbf{b}^n, \qquad n = 0, \ldots, N_t - 1, \tag{8.26}$$

where $\mathbf{b}^n$ changes every step but, as long as $\Delta t$ is fixed, the matrix $A$ does *not*. For backward Euler, $A = I - \Delta t\, kL$. We have to solve (8.26) thousands of times, so it pays to think about how.

**1) Use GMRES.** Solve each step with GMRES as in Chapter 6, using the previous time level as the initial guess (Section 6.4),

$$\boldsymbol{\phi}^{n+1} = \texttt{gmres}(A,\ \mathbf{b}^n,\ \text{guess} = \boldsymbol{\phi}^n).$$

If $\Delta t$ is small, $\boldsymbol{\phi}^n$ is already very close to $\boldsymbol{\phi}^{n+1}$ and only a few iterations are needed. Recall from Section 6.5 that the tolerance should sit just below the other errors in the problem.

**2) Factor once, solve many times.** Since $A$ never changes, we can pay to factor it once, before the time loop, and then reuse the factors at every step. If $A$ is *symmetric positive definite* (SPD), it has a *Cholesky factorization*

$$A = RR^{\mathsf{T}}, \tag{8.27}$$

where $R$ is lower triangular and called the *Cholesky factor*. Then each solve $RR^{\mathsf{T}}\boldsymbol{\phi}^{n+1} = \mathbf{b}^n$ splits into two triangular solves,

$$\begin{cases} R\mathbf{z}^n = \mathbf{b}^n & (R \text{ lower triangular: forward substitution}), \\ R^{\mathsf{T}}\boldsymbol{\phi}^{n+1} = \mathbf{z}^n & (R^{\mathsf{T}} \text{ upper triangular: back substitution}), \end{cases} \tag{8.28}$$

each of which costs about as much as a multiplication by $R$.

For the five-point Laplacian on an $N \times N$ grid with $n = N^2$ unknowns, a good factorization costs about $n^{3/2}$ operations and produces a factor with about $n \log n$ nonzeros, so each pair of triangular solves costs about $n \log n$. The factorization is paid once; the solves are paid every step.

> *Remark* 19. The factor is only reusable while $A$ is unchanged. If $\Delta t$ changes, as in an adaptive method, or $k$ depends on time, the factorization must be redone.

## 8.8 Numerical Implementation

The code below sets up the two-dimensional heat equation on $[0, 2\pi]^2$ with Dirichlet conditions, using $N$ interior points per direction as in Chapter 6. It builds the Laplacian, factors an implicit matrix once before the time loop, and lays out the time loop with the bookkeeping of Section 8.3. The manufactured solution, the forcing (with boundary contributions), and the update performed in each time step are left for you to write. The backward Euler matrix is factored as an example; other implicit methods need their own matrix.

### 8.8.1 MATLAB

```
% grid: N interior points per direction on [0, 2*pi] (Type I, Dirichlet)
N = 64;
ell = 2*pi;
x = linspace(0, ell, N+2)';
x(1) = [];                % remove boundary node x = 0 (Dirichlet, known)
x(end) = [];              % remove boundary node x = 2*pi
dx = x(2) - x(1);
[X, Y] = ndgrid(x, x);    % X(i,j) = x_i and Y(i,j) = y_j
% grid values are stored as U(i,j), so U(:) stacks the rows,
% matching the row-wise ordering of the text

k = 1.0;                  % diffusivity

% 2D five-point Laplacian on the interior unknowns (Kronecker build)
e = ones(N, 1);
D2 = spdiags([e, -2*e, e], [-1, 0, 1], N, N) / dx^2;
I_N = speye(N);
L = kron(I_N, D2) + kron(D2, I_N);
I_n = speye(N*N);

% right-hand side of the semi-discrete system d(phi)/dt = F(t, phi)
F = @(t, phi) k*(L*phi) + g(t, X, Y);

% time stepping parameters
T = 2*pi;                 % final time
Nt = 2000;                % number of steps; dt = T/Nt divides T exactly
dt = T / Nt;

% implicit methods: the matrix does not change from step to step,
% so factor it ONCE, before the time loop.
% example: the backward Euler matrix (other methods need their own matrix)
A_BE = I_n - dt*k*L;
dA_BE = decomposition(A_BE, 'chol');   % sparse Cholesky; dA_BE \ b solves

phi = phi_exact(0.0, X, Y);   % initial condition, phi^0
for n = 0:Nt-1            % n = 0, ..., Nt-1 takes phi^n to phi^(n+1)
    t = n*dt;             % time of the KNOWN level phi^n
                          % (compute from n; do not accumulate t = t + dt)
    %%%
    % IMPLEMENT ONE TIME STEP HERE: phi^n -> phi^(n+1)
    % e.g. for an implicit method, build the right-hand side b
    % and solve with phi = dA_BE \ b;
    %%%
end

% after the loop, phi approximates the solution at t = Nt*dt = T
err = rel_err(phi, phi_exact(T, X, Y));
disp(['N = ' num2str(N) ', dt = ' num2str(dt) ...
      ', relative error = ' num2str(err)])

% RK45: an open implementation with adaptive steps
% opts = odeset('RelTol', 1e-6, 'AbsTol', 1e-9, 'Stats', 'on');
% [tout, phiout] = ode45(F, [0 T], phi_exact(0.0, X, Y), opts);
% phi_RK45 = phiout(end, :)';

function phi = phi_exact(t, X, Y)
    %%%
```

```
    % IMPLEMENT: the manufactured solution at time t on the grid,
    % returned as a column vector of length N*N
    %%%
    phi = zeros(numel(X), 1);
end

function gt = g(t, X, Y)
    %%%
    % IMPLEMENT: the forcing f at time t on the grid, plus any boundary
    % contributions (Section on boundary conditions), as a column vector
    %%%
    gt = zeros(numel(X), 1);
end

function err = rel_err(phi_num, phi_ex)
    % relative error in the discrete 2-norm
    nrm = norm(phi_ex);
    if nrm == 0
        disp('phi_exact is zero -- implement it first')
        err = NaN;
        return
    end
    err = norm(phi_num - phi_ex) / nrm;
end
```

### 8.8.2 python

SciPy does not include a sparse Cholesky factorization, so instead we use the sparse *LU* factorization package `scipy.sparse.linalg.factorized` that returns a function for repeated solves. Its cost behaves similarly for this problem.

```
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

# grid: N interior points per direction on [0, 2pi] (Type I, Dirichlet)
N = 64
ell = 2*np.pi
x = np.linspace(0, ell, N + 2)[1:-1]   # drop the two boundary nodes
dx = x[1] - x[0]
X, Y = np.meshgrid(x, x)    # X[j, i] = x_i and Y[j, i] = y_j
# grid values are stored as U[j, i], so U.ravel() stacks the rows,
# matching the row-wise ordering of the text

k = 1.0                     # diffusivity

# 2D five-point Laplacian on the interior unknowns (Kronecker build)
e = np.ones(N)
D2 = sp.diags([e[:-1], -2*e, e[:-1]], [-1, 0, 1]) / dx**2
I_N = sp.identity(N)
L = (sp.kron(I_N, D2) + sp.kron(D2, I_N)).tocsr()
I_n = sp.identity(N*N, format='csr')


def phi_exact(t):
    ###
    # IMPLEMENT: the manufactured solution at time t on the grid,
    # returned as a flattened vector of length N*N
    ###
```

```
    return np.zeros(N*N)


def g(t):
    ###
    # IMPLEMENT: the forcing f at time t on the grid, plus any boundary
    # contributions (Section on boundary conditions), flattened
    ###
    return np.zeros(N*N)


def F(t, phi):
    # right-hand side of the semi-discrete system d(phi)/dt = F(t, phi)
    return k*(L @ phi) + g(t)


def rel_err(phi_num, phi_ex):
    # relative error in the discrete 2-norm
    nrm = np.linalg.norm(phi_ex)
    if nrm == 0.0:
        print('phi_exact is zero -- implement it first')
        return np.nan
    return np.linalg.norm(phi_num - phi_ex) / nrm


# time stepping parameters
T = 2*np.pi                 # final time
Nt = 2000                   # number of steps; dt = T/Nt divides T exactly
dt = T / Nt

# implicit methods: the matrix does not change from step to step,
# so factor it ONCE, before the time loop.
# example: the backward Euler matrix (other methods need their own matrix)
A_BE = (I_n - dt*k*L).tocsc()
solve_BE = spla.factorized(A_BE)   # returns a function b -> A_BE^{-1} b

phi = phi_exact(0.0)        # initial condition, phi^0
for n in range(Nt):         # n = 0, ..., Nt-1 takes phi^n to phi^(n+1)
    t = n*dt                # time of the KNOWN level phi^n
                            # (compute from n; do not accumulate t += dt)
    ###
    # IMPLEMENT ONE TIME STEP HERE: phi^n -> phi^(n+1)
    # e.g. for an implicit method, build the right-hand side b
    # and call phi = solve_BE(b)
    ###
    pass

# after the loop, phi approximates the solution at t = Nt*dt = T
print(f'N = {N}, dt = {dt:.3e}, relative error = {rel_err(phi, phi_exact(T)):.3e}'
    )

# RK45: an open implementation with adaptive steps
# from scipy.integrate import solve_ivp
# sol = solve_ivp(F, (0.0, T), phi_exact(0.0), method='RK45',
#                 t_eval=[T], rtol=1e-6, atol=1e-9)
# print(sol.nfev, 'evaluations of F')
```

# Bibliography




[1] Timothy Sipkens and Benoît Pasquier. *cmap: Perceptually uniform colormaps for MATLAB, compiled from multiple sources*. https://github.com/tsipkens/cmap.

[2] Kristen Thyng et al. *cmocean: True colors of oceanography*. https://github.com/matplotlib/cmocean.

[3] Kristen M. Thyng et al. "True Colors of Oceanography: Guidelines for Effective and Accurate Colormap Selection". In: *Oceanography* 29.3 (2016), pp. 9–13. DOI: 10.5670/oceanog.2016.66.

[4] Ellert van der Velden. "CMasher: Scientific colormaps for making accessible, informative and 'cmashing' plots". In: *The Journal of Open Source Software* 5.46 (Feb. 2020), p. 2004. DOI: 10.21105/joss.02004. arXiv: 2003.01069 [eess.IV].

[5] Michael L. Waskom. "seaborn: statistical data visualization". In: *Journal of Open Source Software* 6.60 (2021), p. 3021. DOI: 10.21105/joss.03021.

[6] Fabio Crameri. *Scientific colour maps*. Version 8.0.1. Oct. 2023. DOI: 10.5281/zenodo.8409685.

[7] Anton Mikhailov. *Turbo, An Improved Rainbow Colormap for Visualization*. Google Research Blog, Aug. 2019. URL: https://research.google/blog/turbo-an-improved-rainbow-colormap-for-visualization/.

[8] Charles S. Peskin. "Flow patterns around heart valves: A numerical method". In: *Journal of Computational Physics* 10.2 (1972), pp. 252–271. DOI: 10.1016/0021-9991(72)90065-4.

[9] Charles S. Peskin. "The immersed boundary method". In: *Acta Numerica* 11 (2002), pp. 479–517. DOI: 10.1017/S0962492902000077.

[10] Rajat Mittal and Gianluca Iaccarino. "Immersed boundary methods". In: *Annual Review of Fluid Mechanics* 37 (2005), pp. 239–261. DOI: 10.1146/annurev.fluid.37.061903.175743.

[11] Charles S. Peskin. *Lectures on the Immersed Boundary Method*. https://math.nyu.edu/~peskin/ib_lecture_notes/index.html. 2024.

[12] *Ansys Fluent*. Canonsburg, PA, 2025. URL: https://www.ansys.com/products/fluids/ansys-fluent.

[13] M. R. Maxey and B. K. Patel. "Localized force representations for particles sedimenting in Stokes flow". In: *International Journal of Multiphase Flow* 27.9 (2001), pp. 1603–1626. DOI: 10.1016/S0301-9322(01)00014-3.

# Index




adaptive time stepping, 90
Arnoldi process, 63
array flattening, 39

backward Euler, 85
BDF methods, 90
boundary conditions, 27

Cayley–Hamilton theorem, 62
cell-centered grid, 29, 41
checkerboard problem, 51
Cholesky factorization, 91
collocated grid, 51
compact support, 73
convergence study, 87
Crank–Nicolson, 89

diffusion equation, 81
diffusive scaling, 83
Dirichlet boundary condition, 30, 42

Eulerian description, 72
explicit method, 85
explicit midpoint method, 89

figure making, 7
finite difference methods, 23
    first derivatives, 24
        backward difference, 24
        central difference, 25
        forward difference, 24
    second derivatives
        central difference, 26, 38
    two dimensions, 37
five-point stencil, 38
forward Euler, 85

ghost points, 28
global error, 87
GMRES, 60
    initial guess, 64
    least-squares problem, 64
    restarted, 64
    tolerance, 65
grid
    Type I, 29, 41
    Type II, 29, 41

heat equation, 81

immersed boundary method, 70
    four-point kernel, 73
    interpolation, 74
    regularized delta function, 73
    spreading, 74
implicit method, 86
    linear solves, 90
initial condition, 81
iterative methods, 60

Kronecker product, 40, 55, 62

Krylov subspace, 62

Lagrange multiplier, 75
Lagrangian description, 72
Laplacian, 37
    eigenvalues, 85
local truncation error, 87

MAC grid, 53
manufactured solution, 37, 60
matrix-free methods, 63, 75
multistep methods, 90

Neumann boundary condition, 32, 43
nodal grid, 29, 41

order of accuracy, 86

periodic boundary condition, 30, 44
plotting, 7
preconditioning, 65

random walk, 82
Runge–Kutta methods, 89

saddle point system, 74
seven-point Laplacian, 61
stability, 87
stability restriction, 88
staggered grid, 51
    divergence operator, 52
    gradient operator, 52
    MAC, 53

Taylor series, 23
time stepping, 81

unconditional stability, 88